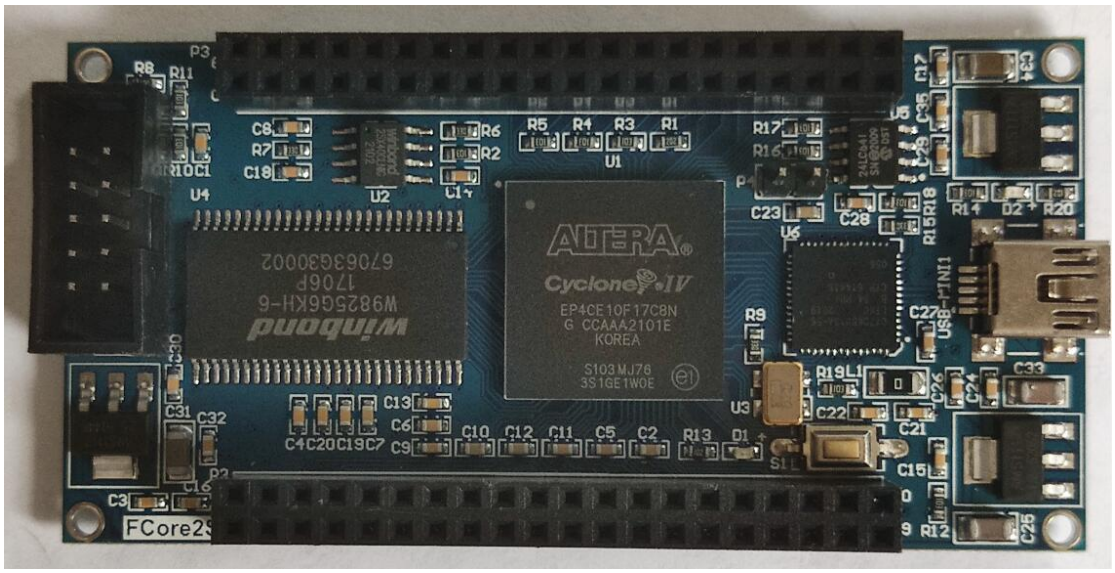
FPGA 实例教程
[FCore2S]



作者：ETree
日期：2022-10-9
版本：v1.4

Copyright (C) 2017-20xx ETree Corporation
版权归<益树科技工作室>所有，转载请注明出处！

版本	内容
V1.0	初始版本，2020.7.4
V1.1	增加《Verilog 数字系统篇》“VGA 彩条显示”章节，2021.10.23 增加《Verilog 数字系统篇》“OV7670 VGA 显示器显示”章节，2021.10.23 增加《Verilog 数字系统篇》“双目 MT9V011 USB2.0 显示”章节，2021.10.23
V1.2	增加《Verilog 数字系统篇》“双目 MT9V011 VGA 显示器显示”章节，2021.10.24
V1.3	增加《Verilog 数字系统篇》“USB Slave FIFO 读”章节，2022.10.7 增加《Verilog 数字系统篇》“2.4 寸液晶屏彩条显示”，2022.10.7
V1.4	增加《Verilog 数字系统篇》“OV7670 2.4 寸液晶屏显示”章节，2022.10.9 增加《Verilog 数字系统篇》“USB 2.4 寸图片显示”章节，2022.10.9

目录

FPGA 实例教程.....1
[FCore2S].....1
Verilog HDL 数字系统设计篇.....7
第一章 板卡介绍.....8
第二章 软件安装.....9
2.1. Quartus II.....9
2.2. USB-Blaster 驱动安装.....15
第三章 组合逻辑门建模.....18
3.1. 组合逻辑电路的门级建模.....18
3.2. 多输入门.....18
3.3. 多输出门.....18
3.4. 三态门.....19
第四章 2 线--4 线译码器.....20
4.1. Verilog 代码结构.....20
4.2. 门级建模.....20
4.3. 数据流建模.....21
第五章 D 锁存器/触发器与时序逻辑电路.....22
5.1. D 锁存器/触发器.....22
5.2. 时序逻辑电路(10 进制计数器).....22
第六章 LED 闪烁.....24
6.1. 硬件介绍.....24
6.2. 新建工程.....24
6.3. HDL 设计.....27
6.4. 设置 Dual-Purpose Pins.....29
6.5. 编译工程与管脚约束.....29
6.6. 下载 sof 文件.....30
6.7. jic 文件生成.....31
6.8. 下载 jic 文件.....32
第七章 按键检测.....34
7.1. 硬件介绍.....34
7.2. HDL 设计.....34
7.3. 管脚分配.....36
7.4. 实验操作.....36
第八章 PLL 锁相环.....37
8.1. 硬件介绍.....37
8.2. MegaWizard.....37
8.3. HDL 设计.....40
8.4. 实验操作.....42
第九章 数码管秒表.....43
9.1. 硬件介绍.....43
9.2. HDL 设计.....44
➢ 系统框架设计.....44

- 秒表进位计数.....45
9.3. 管脚分配.....46
9.4. 实验操作.....46
第十章 VGA 彩条显示.....47
10.1. 硬件介绍.....47
10.2. VGA 时序.....48
10.3. HDL 设计.....48
- 系统框架.....48
- VGA 模块.....49
10.4. 管脚分配.....52
10.5. 实验操作.....53
第十一章 数字频率计.....55
11.1. 硬件介绍.....55
11.2. HDL 设计.....55
11.3. 管脚约束.....57
11.4. SignalTap 使用.....57
- 打开 SignalTap II Logic Analyzer.....57
- 选择 JTAG Chain Configuration.....58
- 设置采样时钟和采样深度.....58
- 添加被观察信号.....59
- 保存 SinalTap II 文件.....60
- 选择信号的触发条件.....61
- 编译工程.....62
- 运行 SignalTap II.....62
11.5. 实验操作.....62
第十二章 OV7670 VGA 显示器显示.....65
12.1. 硬件介绍.....65
12.2. OV7670 摄像头.....65
12.3. CMOS 工作原理.....67
12.4. HDL 设计.....69
- 系统框架.....69
- VGA 驱动.....70
- SDRAM 驱动.....72
12.5. 管脚约束.....72
12.6. 实验操作.....73
第十三章 USB Slave FIFO 写.....75
13.1. 硬件介绍.....75
13.2. CY7C68013A 芯片简介.....75
13.3. USB Slave FIFO 传输.....76
13.4. USB 固件开发.....78
13.5. Keil 搭建工程.....80
- 安装软件.....80
13.6. HDL 设计.....84
13.7. 管教约束.....86

13.8. 实验操作.....87
- 安装 USB2.0 驱动.....87
- 下载 USB 固件代码.....90
- 下载 FPGA 配置代码.....93
- 上位机 USB 接受演示.....93

第十四章 USB Slave FIFO 读.....95
14.1. 硬件介绍.....95
14.2. USB 固件开发.....95
14.3. HDL 设计.....95
14.4. 管教约束.....98
14.5. 实验操作.....98
第十五章 USB 上位机数据显示.....100
15.1. 硬件介绍.....100
15.2. USB 固件开发.....100
15.3. HDL 设计.....100
- 系统框架.....100
- vid_test 设计.....101
- usb_ctrl 设计.....101

15.4. 管脚约束.....102
15.5. 实验操作.....104
第十六章 OV7670 USB 上位机显示.....106
16.1. 硬件介绍.....106
16.2. HDL 设计.....106
16.3. 管脚约束.....106
16.4. 实验操作.....108
第十七章 液晶屏彩条显示[2.4 寸].....110
17.1. 硬件介绍.....110
17.2. TFT-LCD 简介.....111
17.3. HDL 设计.....113
17.4. 管脚分配.....116
17.5. 实验操作.....117
第十八章 OV7670 液晶屏彩条显示[2.4 寸].....118
18.1. 硬件介绍.....118
18.2. HDL 设计.....118
- 系统框架.....118
- SDRAM 驱动.....118

18.3. 管脚约束.....119
18.4. 实验操作.....120
第十九章 USB 液晶屏图片显示[2.4 寸].....122
19.1. 硬件介绍.....122
19.2. HDL 设计.....122
- 系统框架.....122
- SDRAM 驱动.....122

19.3. 管脚分配.....123

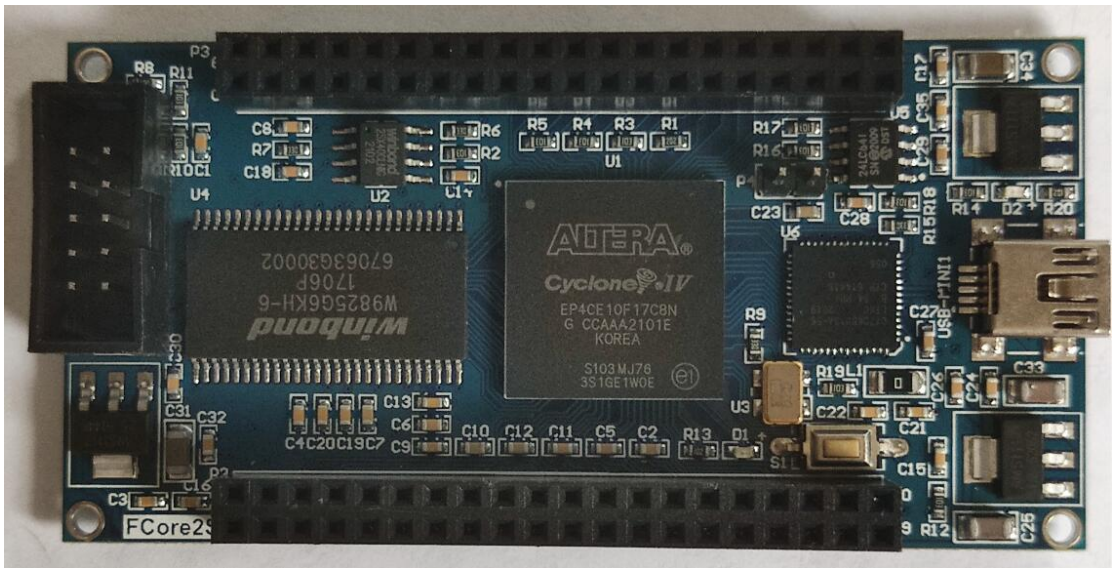
19.4. 实验操作.....123
第二十章 USB 图片 VGA 显示.....127
END.....128

Verilog HDL 数字系统设计篇

第一章 板卡介绍

ETree FCore2S 是结合 FPGA 和 USB2.0(CY7C68013A)的核心板卡， 具体资源如下：

- FPGA: EP4CE6E22C8N or EP4CE10E22C8N；
- SDRAM: W9825G6KH-6；
- SPI Flash: W25X40；
- USB2.0: CY7C68013A；
- EEPROM: 24LC64
- LED；
- 独立按键；
- JTAG 下载接口；
- GPIO 接口



FPGA 板卡可以在 https://shop397545458.taobao.com 购买，期待朋友们的加入。

第二章 软件安装

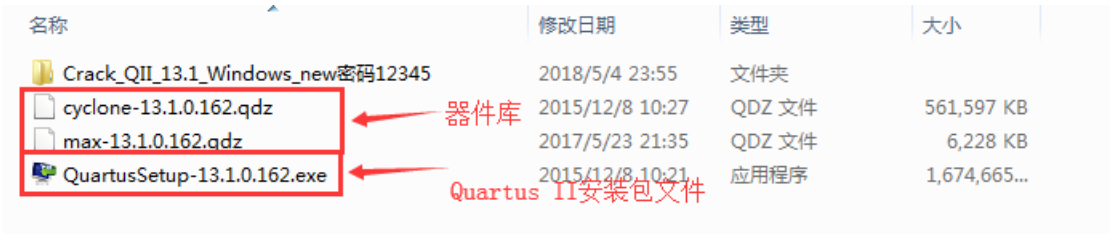
2.1. Quartus II

Quartus II 是 ALTERA 公司的一种可编程的设计环境，由于其功能强大和直观易用，越来越受到数字系统设计者的欢迎。

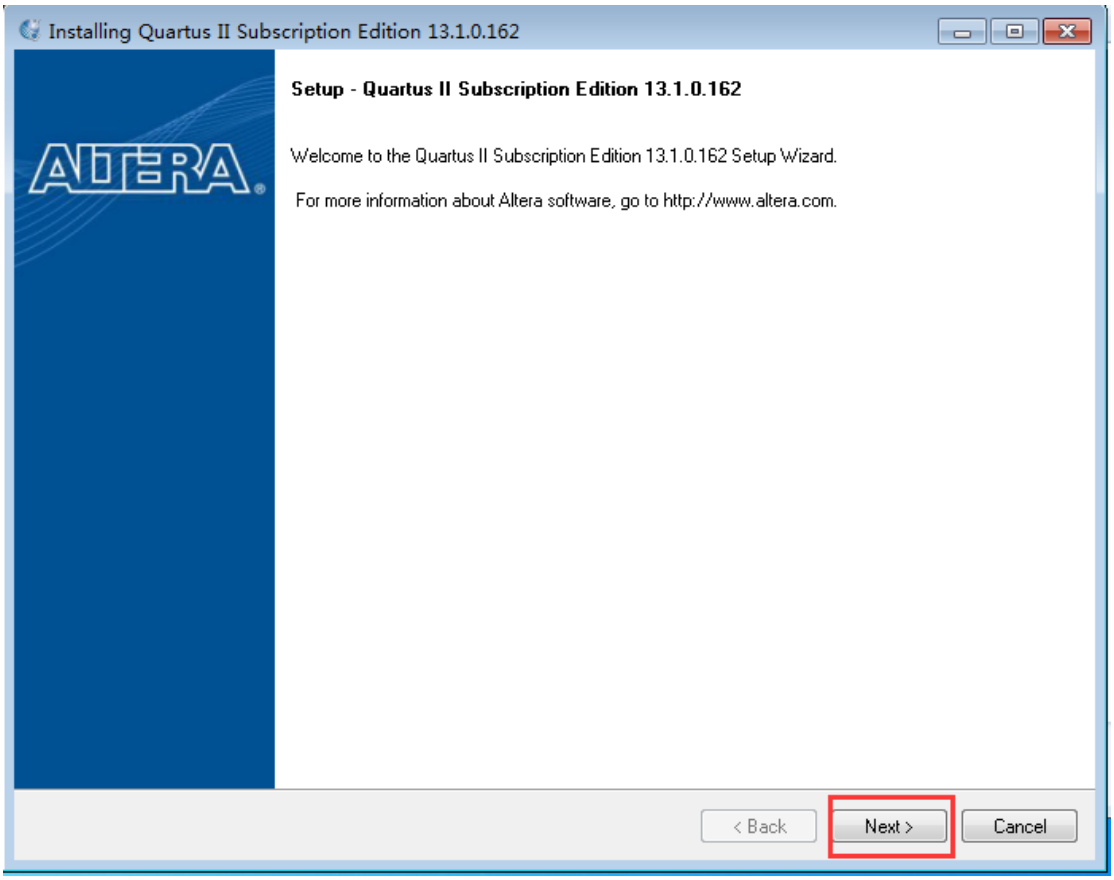
这里我们使用的 Quartus II 版本是 13.1.0.162，具体安装流程可以参考我们录制的视频教程。



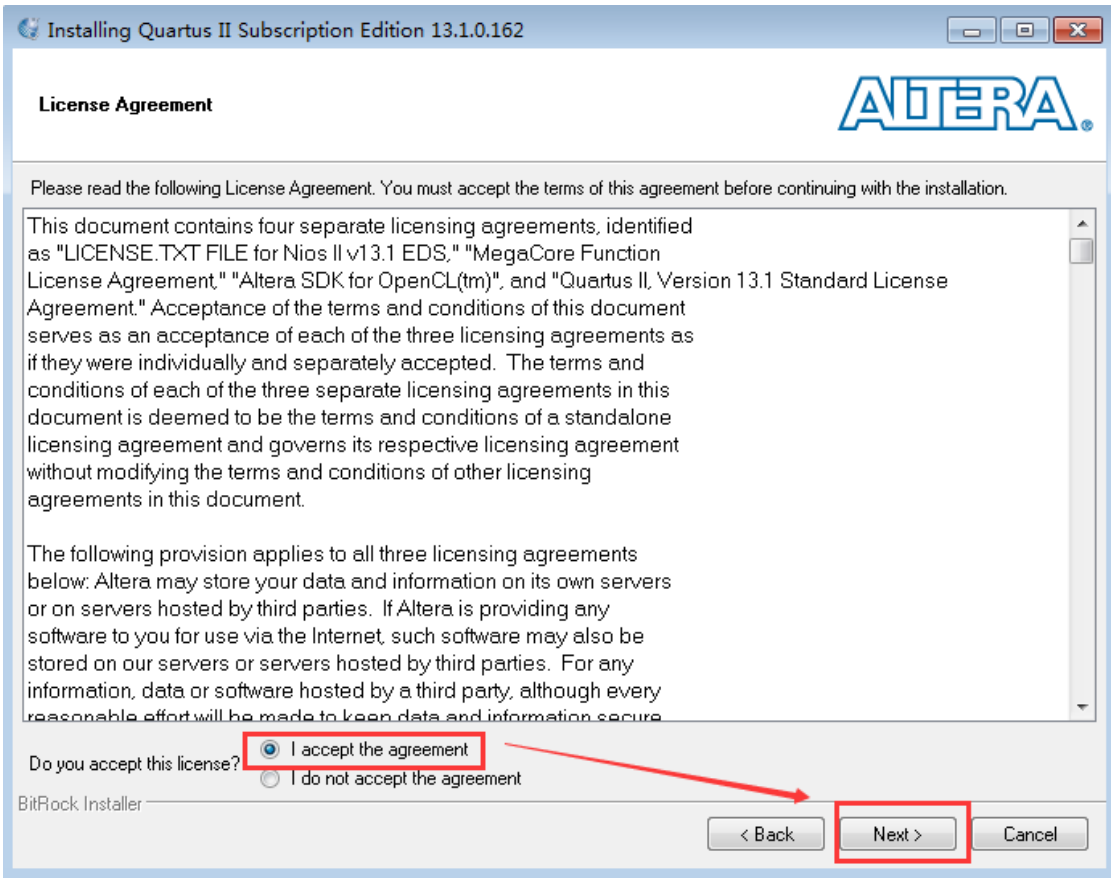
打开 software 目录下的“QuartusSetup-13.1.0.162.exe”安装包文件。



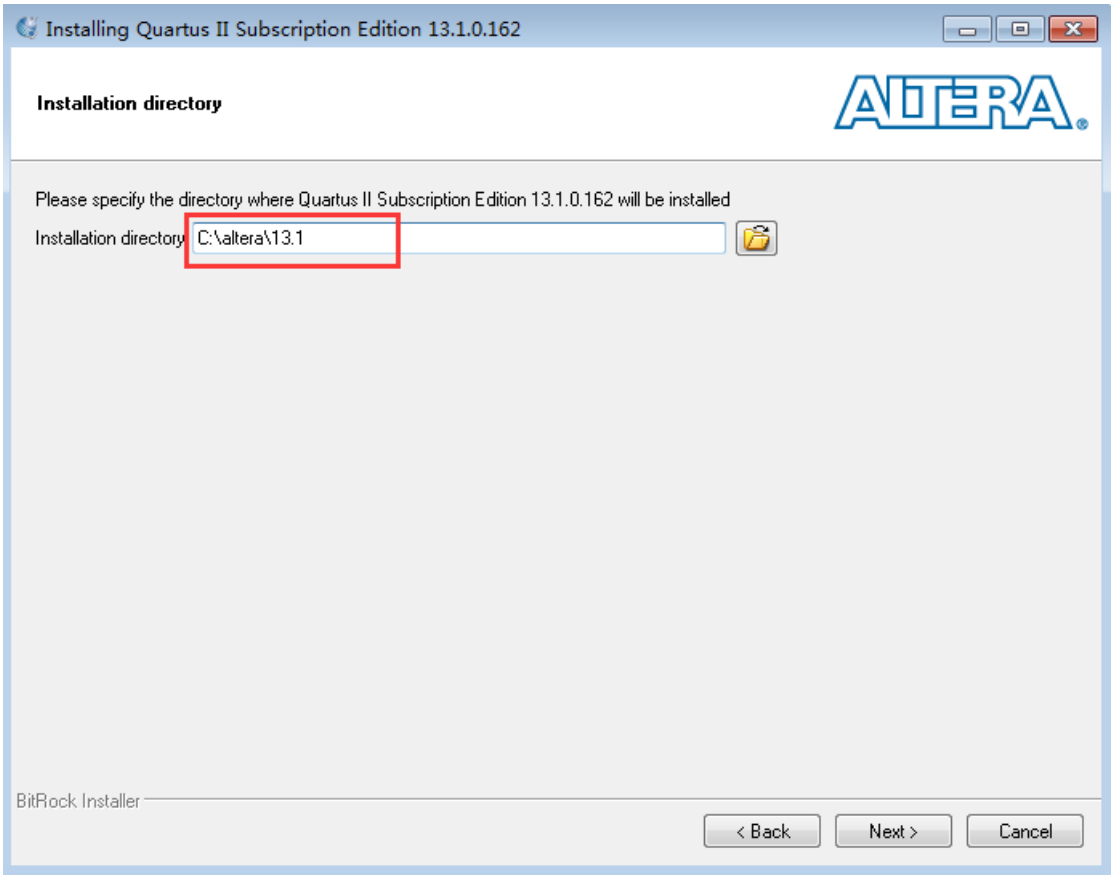
弹出安装对话框，点击“Next”继续安装。



选择“I accept the agreement”，然后点击“Next”继续安装。

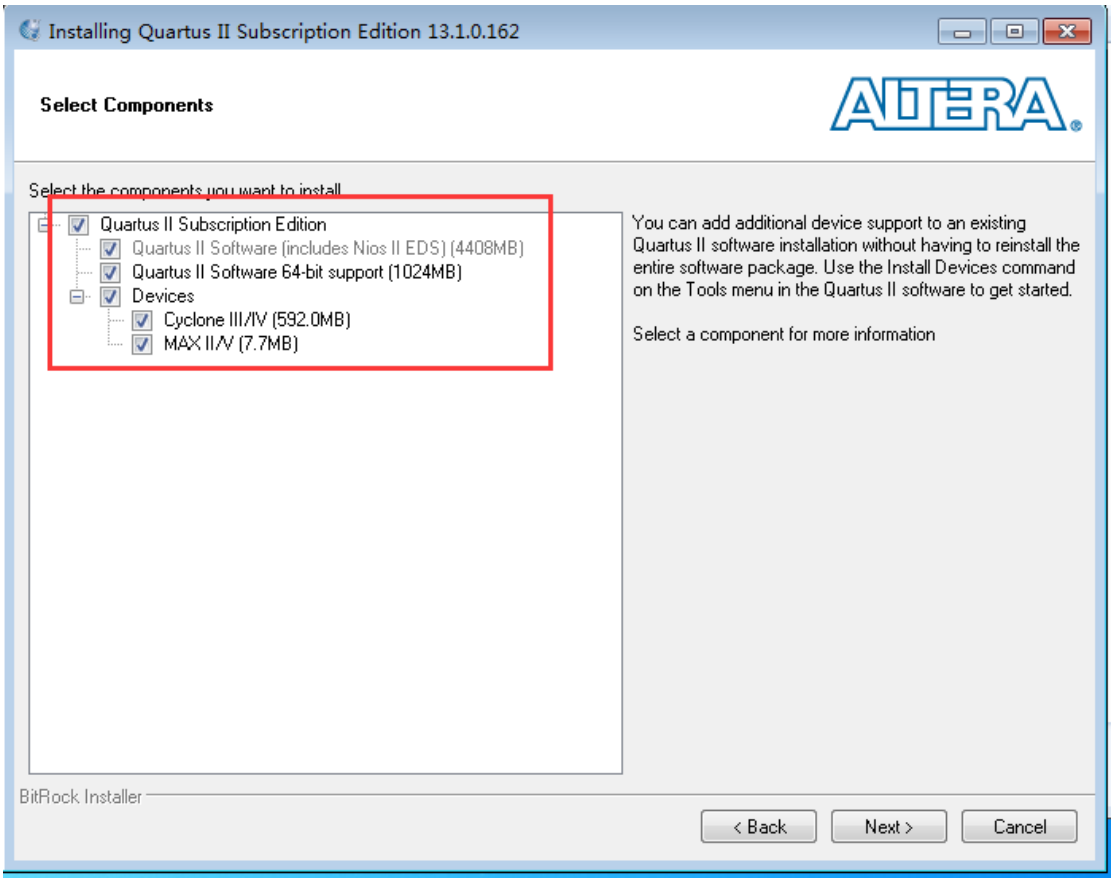


选择安装目录，默认安装在 C 盘根目录下，然后点击“Next”继续安装。

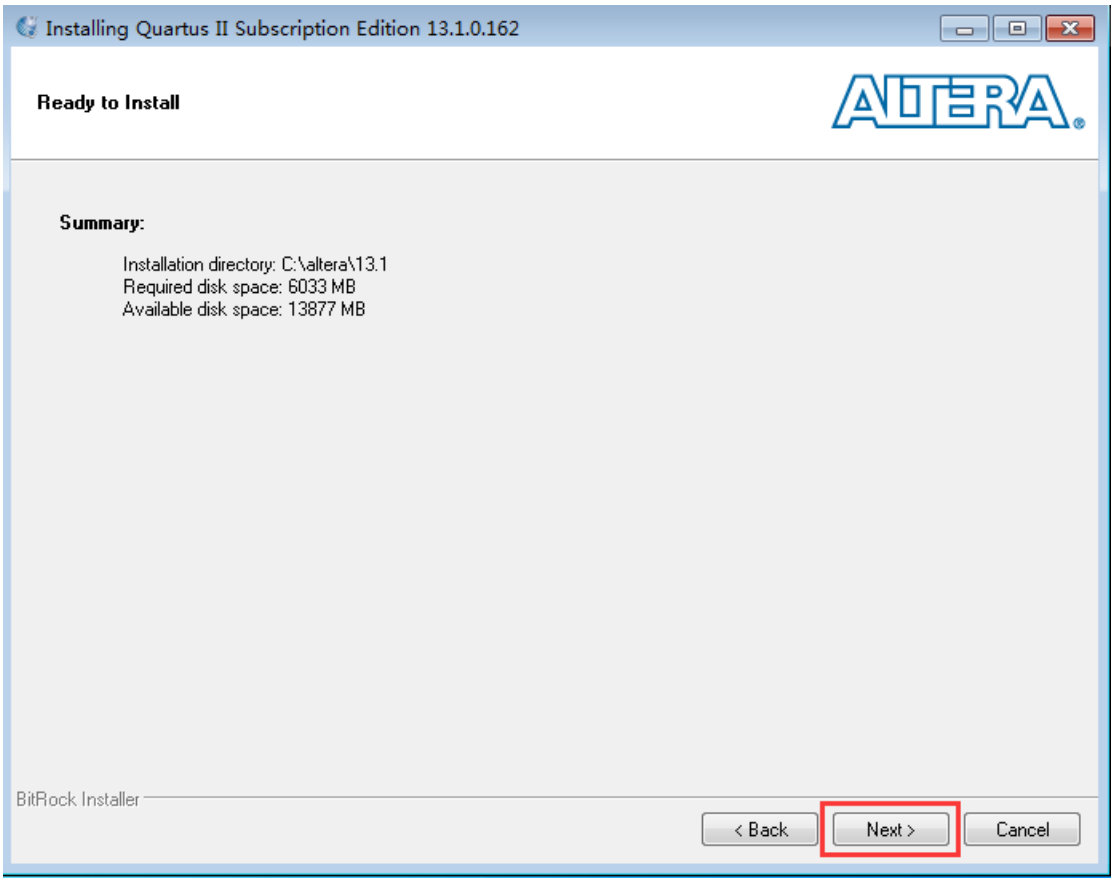


选择安装 Quartus II 类型，默认 32 位必须安装，我们再勾选 64 位版本。选择支持的器

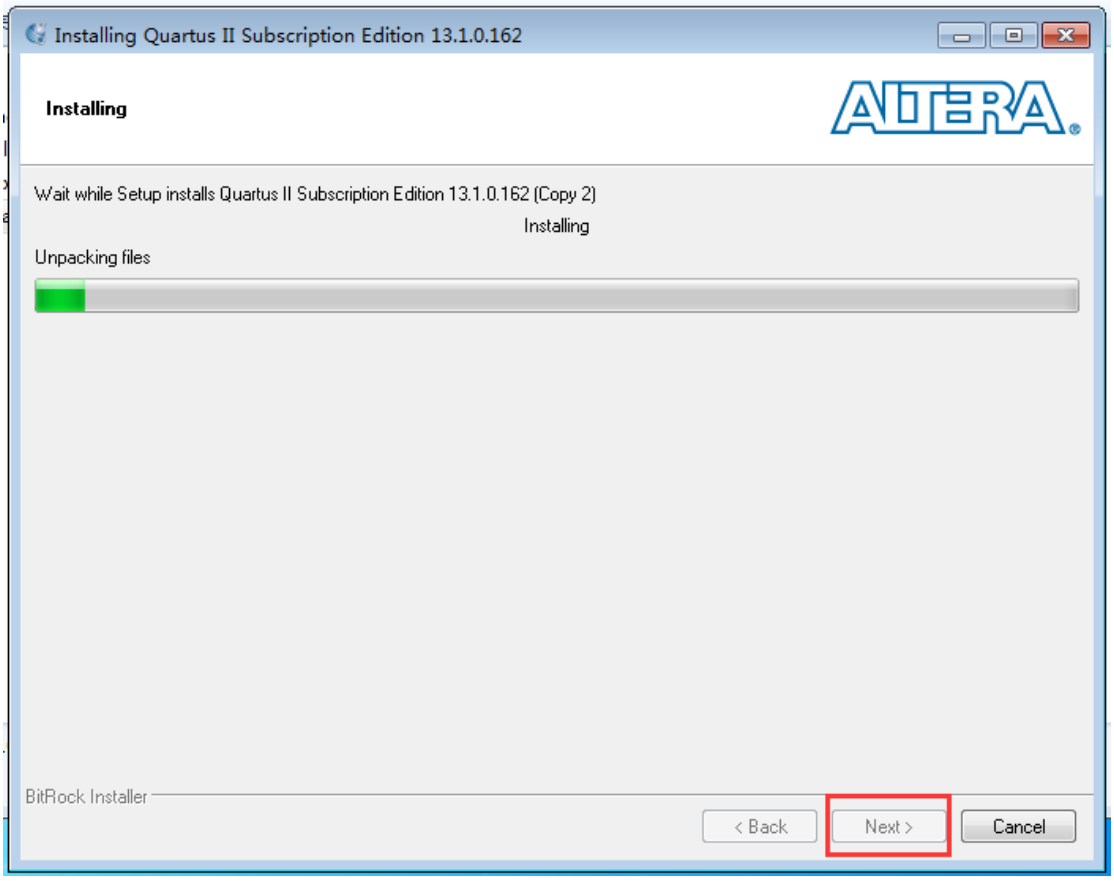
件类型为 Cyclone III/IV 和 MAX II/V。然后点击“Next”继续安装。



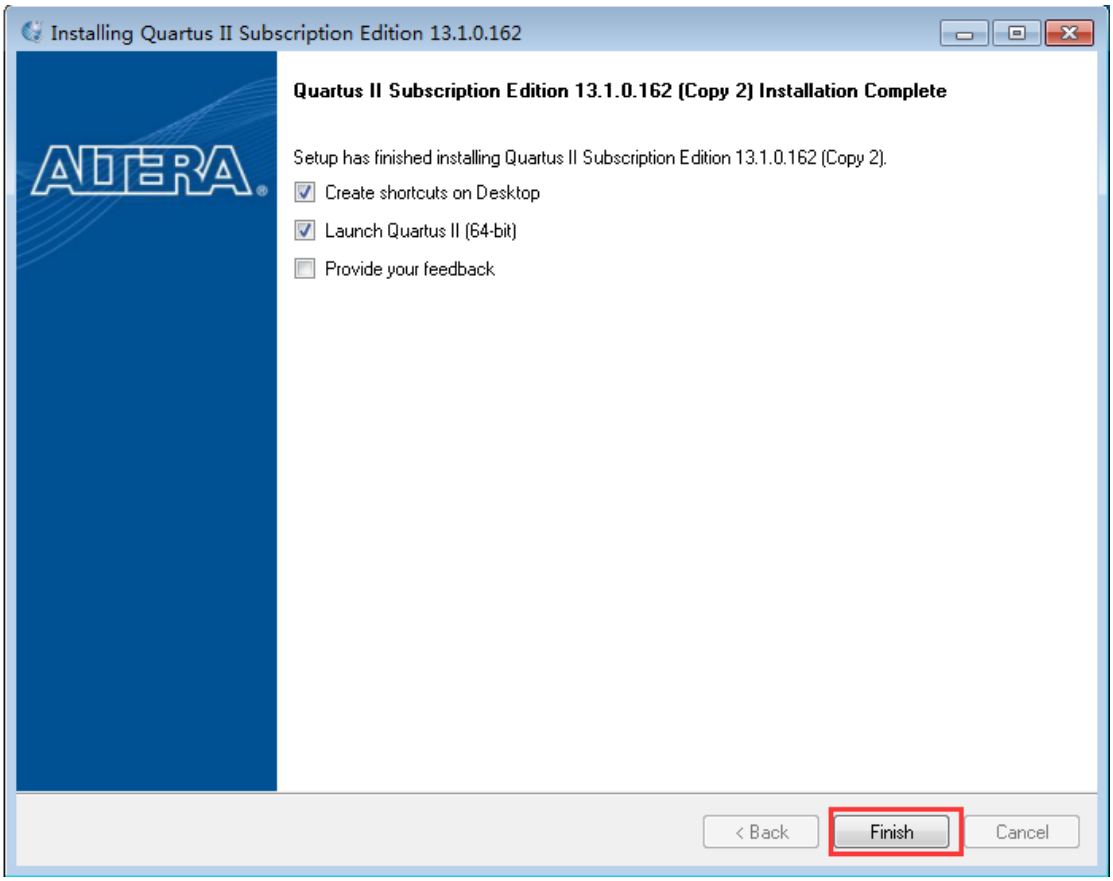
点击“Next”继续安装。



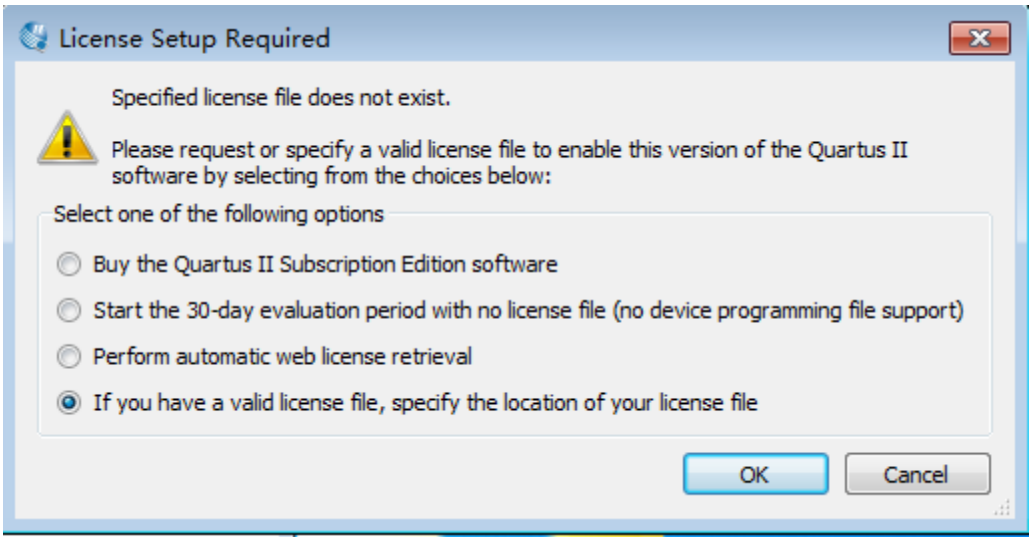
下图表示正在安装，时间会稍微长一点，当绿色进度为 100%时，点击“Next”继续安装。



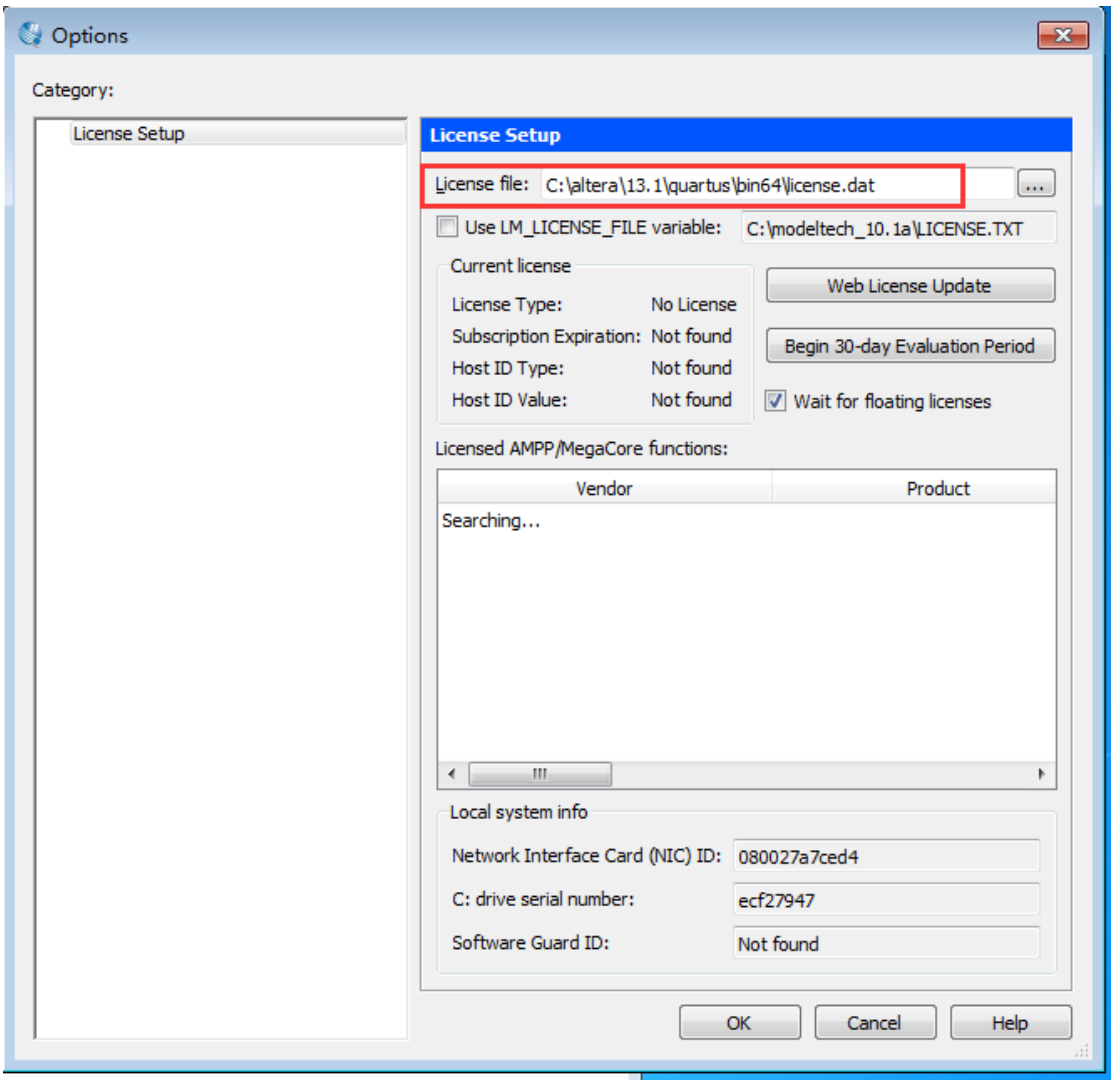
点击“Finish”完成安装。



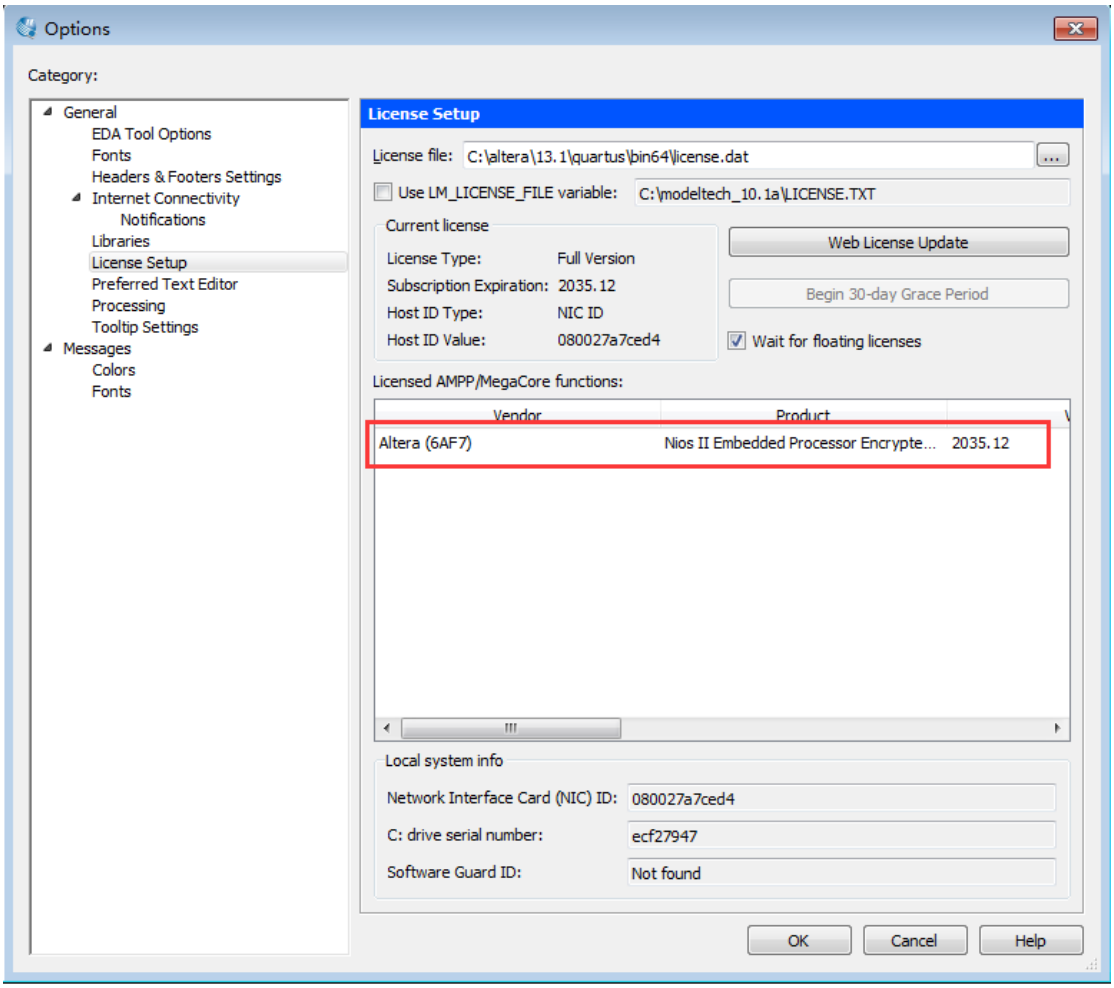
在弹出的对话框中选择“If you have a valid license file, specify the location of your license file”，点击 OK。



定位到 license.dat 存放的目录，点击 OK。



在 Options | General | license Setup 中显示破解成功到 2035 年 12 月，如下图所示。



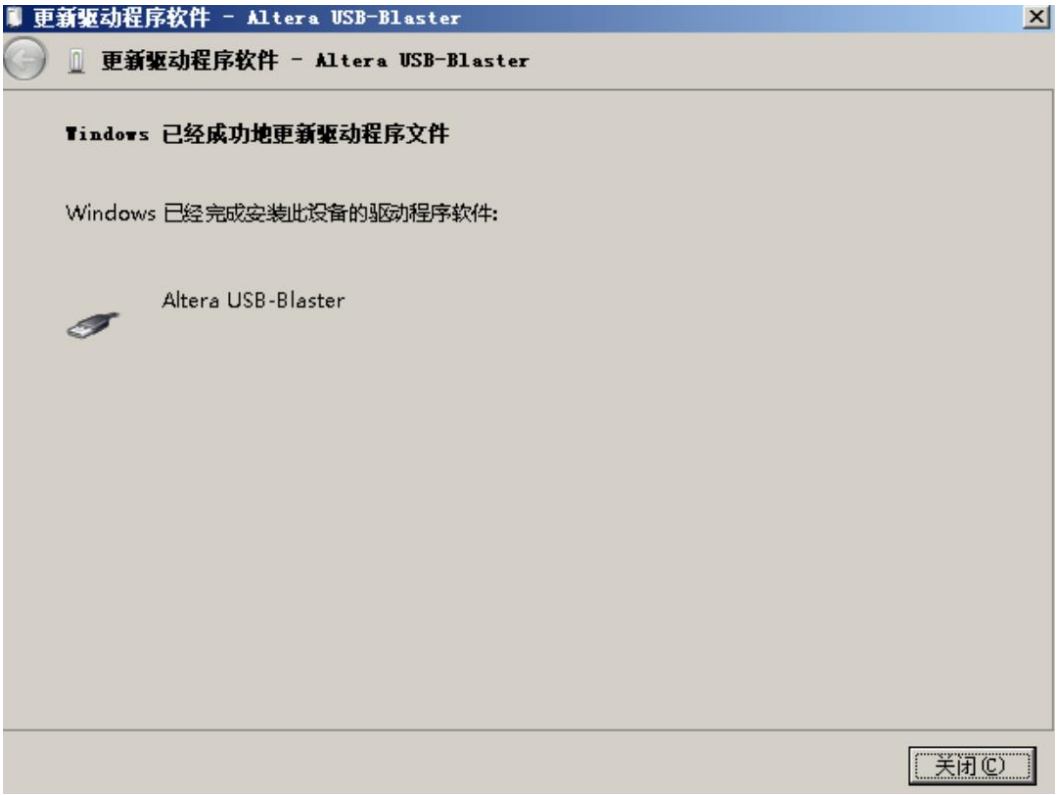
2.2. USB-Blaster 驱动安装

使用 Quartus II 软件对开发板下载和调试前，需要安装 USB Blaster 驱动程序。驱动程序在资料盘"Tools\usb-blaster"，注意不要再往下选择。

在设备管理器中找到对应的位置，选择"下一步"，在弹出的安全对话框中选择"始终信任"，再点击"安装"按钮。



驱动安装完成，点击"关闭"。



在设备管理器中显示安装好的 USB-Blaster 设备。


鼠标和其他指针设备
通用串行总线控制器
Altera USB-Blaster
USB Composite Device
英特尔(R) USB 3.0 根集线器
英特尔(R) USB 3.0 可扩展主机控制器
网络适配器
MERCURY Wireless N Adapter
Microsoft Virtual WiFi Miniport Adapter #3

第三章 组合逻辑门建模

本章介绍 verilog 的组合逻辑门建模基础知识。

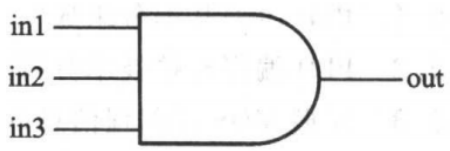
3.1. 组合逻辑电路的门级建模

门级建模就是将逻辑电路图用 verilog HDL 规定的语言描述出来。Verilog 语言中内置 12 个基本门级模型，如下表所示。

元件符号	功能
and	多输入端与门
or	多输入端或门
xor	多输入端异或门
buf	多输出端缓冲器
nand	多输入端与非门
nor	多输入端或非门
xnor	多输入端异或非门
not	多输出端反相器
bufif1	控制信号高电平有效三态缓冲器
bufif0	控制信号低电平有效三态缓冲器
notif1	控制信号高电平有效三态反相器
notif0	控制信号低电平有效三态反相器

3.2. 多输入门

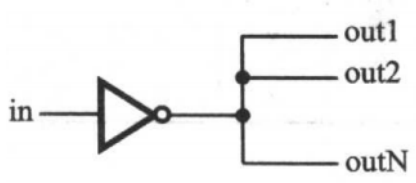
多输入门共同的特点是只有一个输出。下图为与门(and)的模型及其描述方法。




```
and a1(out,in1,in2,in3);
```

3.3. 多输出门

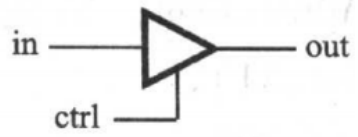
多输出门的共同特点是只有一个输入。下图为输出反相器的模型及其描述方法。




```
not n1(out1,out2,...,in);
```

3.4. 三态门

三态门有一个输入、一个输出和一个控制输入端，如果输入控制信号无效，则三态门输出为高阻态 z。下图为 bufif1 的模型和描述方法。




```
bufif1 b1(out,in,ctrl);
```

第四章 2 线--4 线译码器

本章基于 verilog 逻辑门基础设计一款 2 线--4 线译码器电路。

4.1. Verilog 代码结构

在设计译码器之前先介绍一下 verilog 逻辑代码的结构。

```
//File name: top_fpga.v
//Fuction  : 顶层文件
//Log      :
module top_fpga(    //顶层模块名
    input clk,      //模块端口
    input rst_n,    //模块端口
    output led      //模块端口
);
//信号定义
wire a0,a1;
wire [3:0] y;
//模块例化
_2to4decoder inst0(
    .A1(a1), //信号映射
    .A0(a0), //信号映射
    .Y(y)    //信号映射
);
//逻辑代码
assign led = ~Y;
endmodule//模块结束
```

Verilog 逻辑之间的关系通过一个顶层 module 模块来组织关系，顶层模块把需要的 module 模块例化(按照模块名+端口名)到自己的模块内，上面的例子就是顶层模块 top_fpga 把“5.2 节”的“_2to4decoder”模块例化到了自己的模块内。同样，其他模块也可以利用同样的方法把需要的模块例化到自己的模块内。

4.2. 门级建模

假设 2 线输入为 A 和 B，四线为 Y[3:0]，则译码的真值表为：

{A1,A0}	Y
{0,0}	{0,0,0,1}
{0,1}	{0,0,1,0}
{1,0}	{0,1,0,0}
{1,1}	{1,0,0,0}

根据真值表得到下面的设计代码：

```
module _2to4decoder(
    input A1,       //输入信号
```

```
    input A0,       //输入信号
    output [3:0] Y  //输出信号
);

wire A1_n,A0_n; //电路内部节点信号

not n1(A1_n,A1);
not n2(A0_n,A0);

and n4(Y[0],A1_n,A0_n);
and n5(Y[1],A1_n,A0);
and n6(Y[2],A1,A0_n);
and n7(Y[3],A1,A0);

endmodule
```

4.3. 数据流建模

上一节使用的是门级建模，这章我们使用数据流建模方式实现 2 线--4 线译码器电路。

设计的输入输出接口延用上一章，这里我们可以直接用 assign 语句实现这个组合逻辑功能，具体代码如下。

```
module _2to4decoder(
    input A1,       //输入信号
    input A0,       //输入信号
    output [3:0] Y  //输出信号
);

assign Y[0] = ~A1 & ~A0;
assign Y[1] = ~A1 & A0;
assign Y[2] =  A1 & ~A0;
assign Y[3] =  A1 & A0;

endmodule
```

上面的代码看起来是不是比上一章的代码简单很多，这也是 verilog 设计中使用最多的设计方法。

第五章 D 锁存器/触发器与时序逻辑电路

5.1. D 锁存器/触发器

行为级建模就是描述逻辑电路的功能和算法，一般使用 always 结构实现。本章实现 D 锁存器和 D 触发器的行为级建模。

下面为 D 锁存器的建模代码。

```
module D_latch(
    input D,
    input E,
    output reg Q
);

always @(E or D)
    if(E==1)
        Q <= D;

endmodule
```

如果输入信号 E 或 D 发生变化，就会执行一次 if 语句。

下面为 D 触发器的建模代码。

```
module DFF(
    input D,        //输入数据信号
    input clk,      //输入时钟信号
    output reg Q    //输出数据信号
);

always @(posedge clk)
    Q <= D;

endmodule
```

当时钟 clk 的上升沿到来时，D 的值被传送到 Q。

5.2. 时序逻辑电路(10 进制计数器)

时序逻辑电路和组合逻辑电路的最大区别就是时序逻辑电路是以时钟方式驱动的，以时钟为节拍处理数据信号，本章基于时序逻辑电路的行为级建模方式设计一个 10 进制计数器。

假设输入时钟信号为 clk，输出数据为 Q[3:0](因为 10 进制中最大的位数为 9，而 9 的二进制表示为 1001，这样需要 4 位表示)，具体代码如下。

```
module DFF(
    input clk,      //输入时钟信号
    output reg [3:0] Q  //输出数据信号
);
```

```
always @(posedge clk)
    if(Q>=4'b1001)
        Q <= 4'b0000;
    else
        Q <= Q + 1'B1;

endmodule
```

当 clk 的上升沿到来是，always 结构里的语句被执行一次；当 Q 的值还没有到达 9(4'b1001)时，Q 执行累加 1 操作；当 Q 的值为 9 时，Q 被清零；这样就实现了一个 10 进制的计数器。

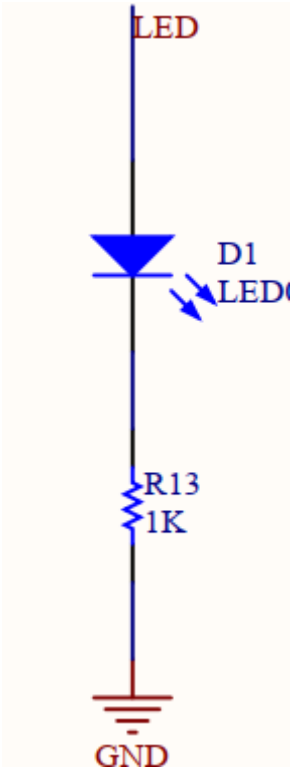
第六章 LED 闪烁

本章是第一个 FPGA 项目，将详细介绍创建新工程的流程，如果用户已经熟悉 Quartus II 开发流程可以略过或作参考。

本设计目的是让 LED 灯每 1 秒闪烁一次。

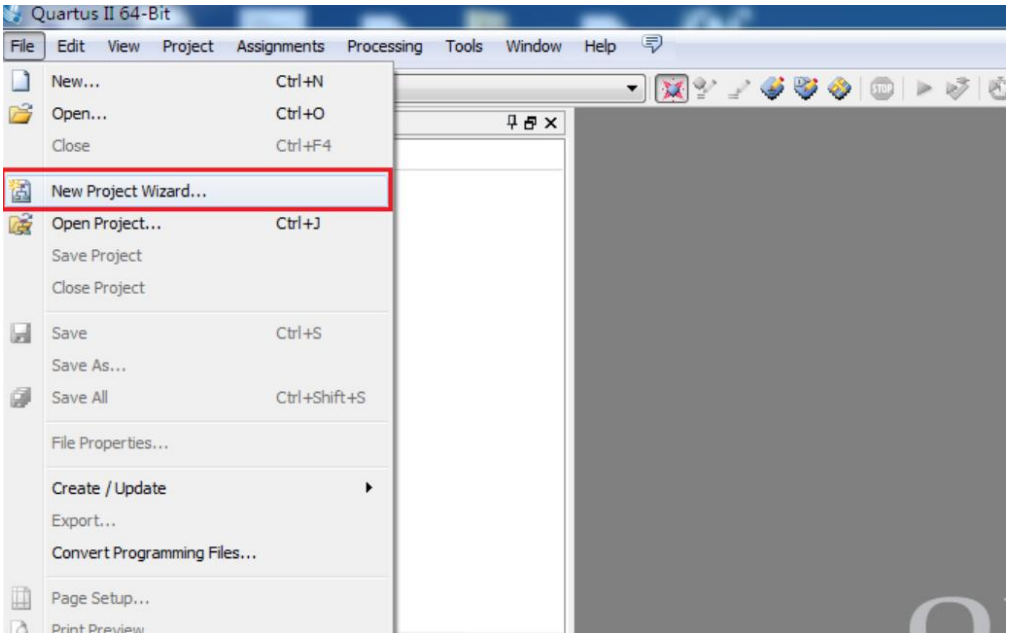
6.1. 硬件介绍

开发板上有 2 盏 LED 灯，一盏为电源指示灯，一盏为 FPGA 可控制的 LED 灯(高电平点亮)。

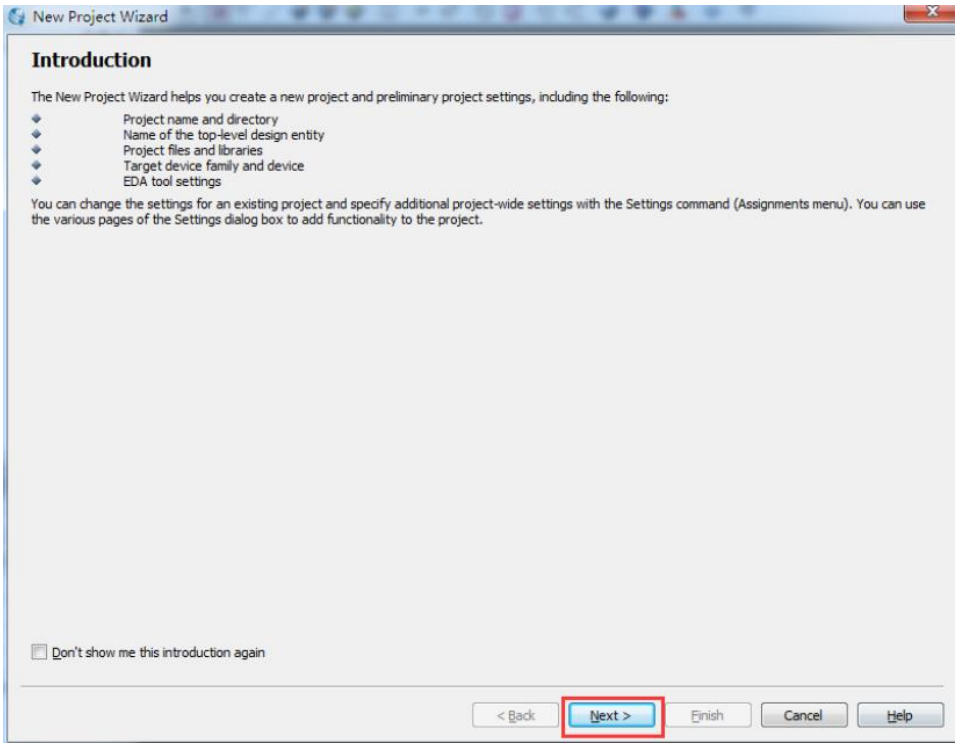


6.2. 新建工程

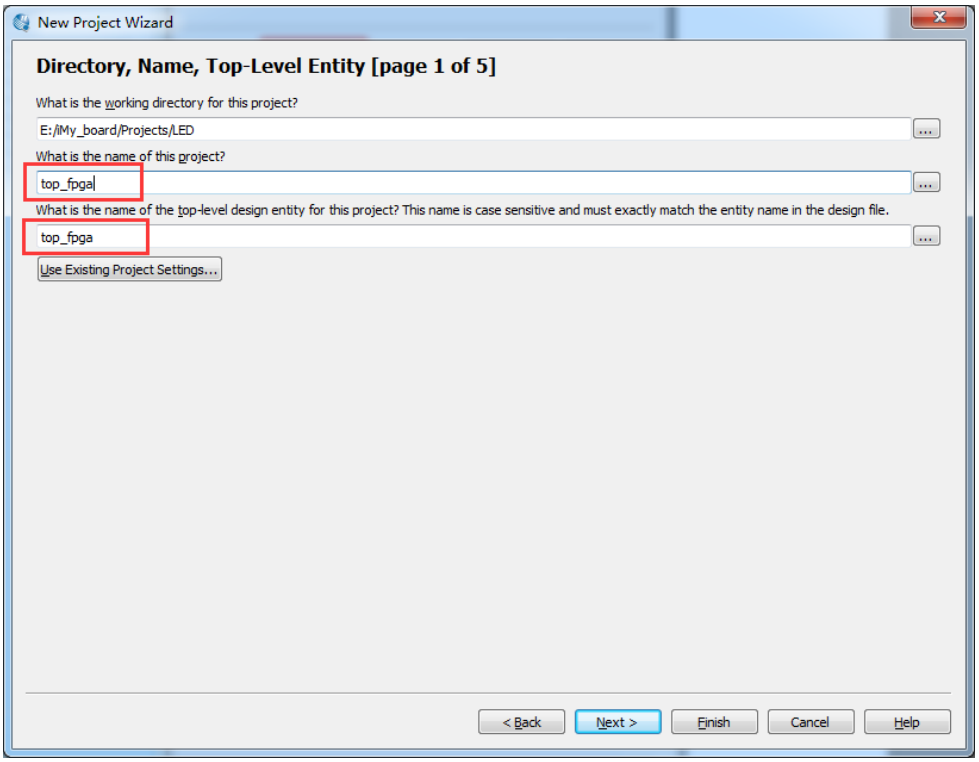
启动 Quartus II，选择菜单 File -> New Project Wizard。



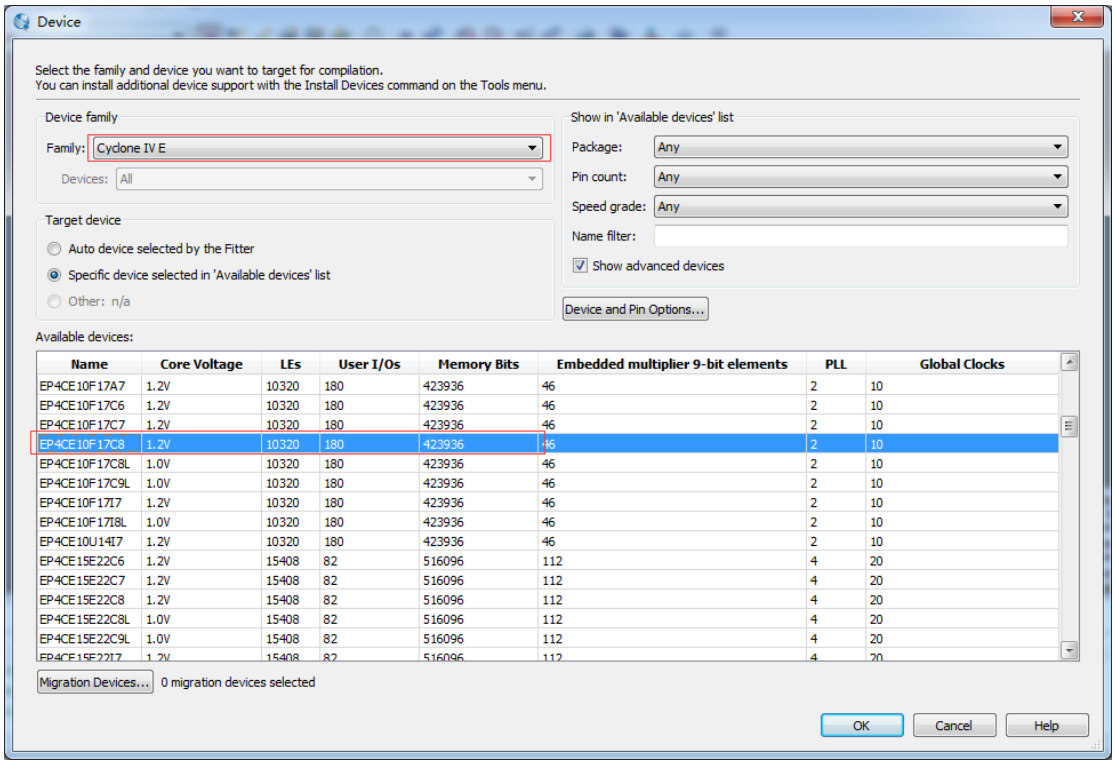
选择"Next'。



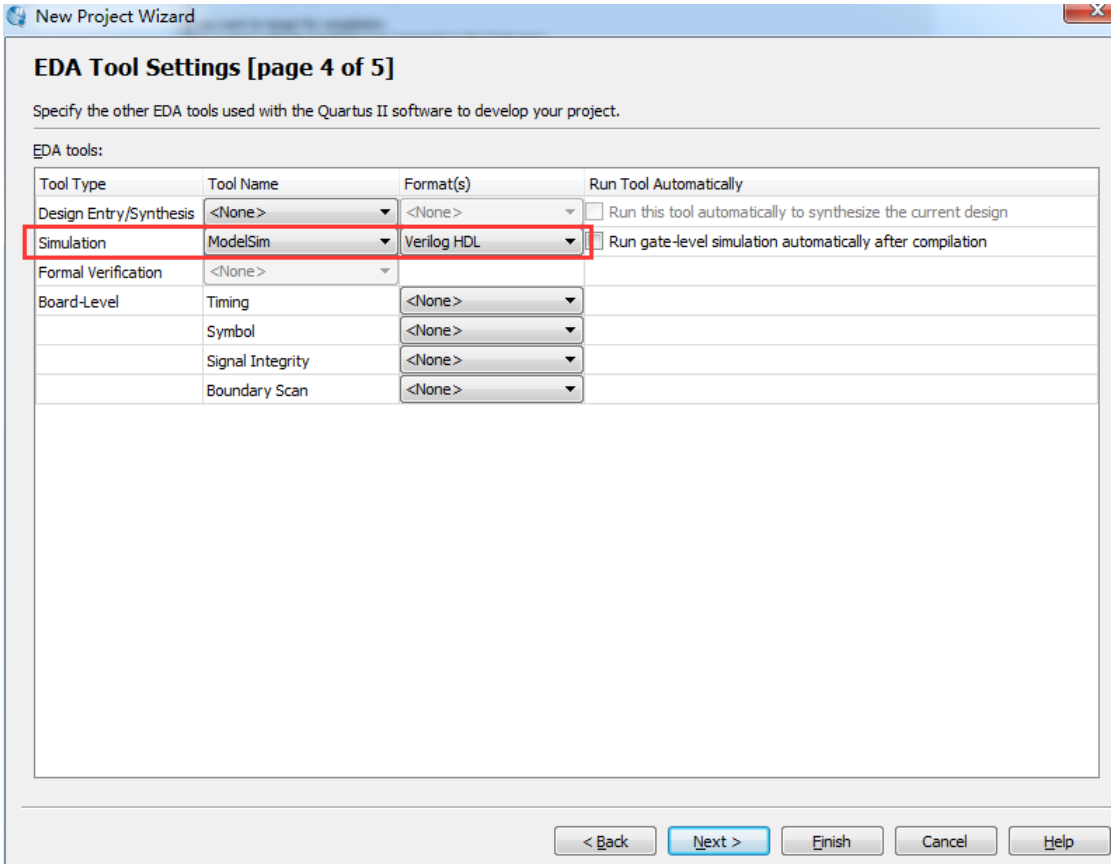
在工程目录下输入工程名(top_fpga)和顶层设计名(top_fpga)，这两个名字一样。



选择器件类型和型号。



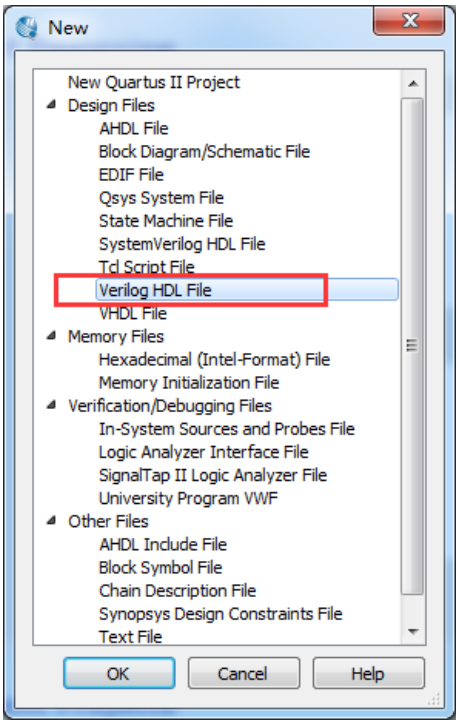
选择仿真软件和仿真语言，这里选择 Modelsim/verilog。



点击"Finish"完成工程创建。

6.3. HDL 设计

新建 top_fpga.v 文件，点击菜单 File -> New，选择 Verilog HDL File。



top_fpga.v 代码设计如下：

```
//file name:      top_fpga.v
//author:         ETree
//date:           2017.5.4
//function:       top file of project
//log:            led light

module top_fpga(
    //global signal
    input clk,  //24MHz
    input rst_n,//low active

    //led
    output  led //high active
);

//寄存器定义
reg [31:0] timer;
reg led_r;

assign led = led_r;

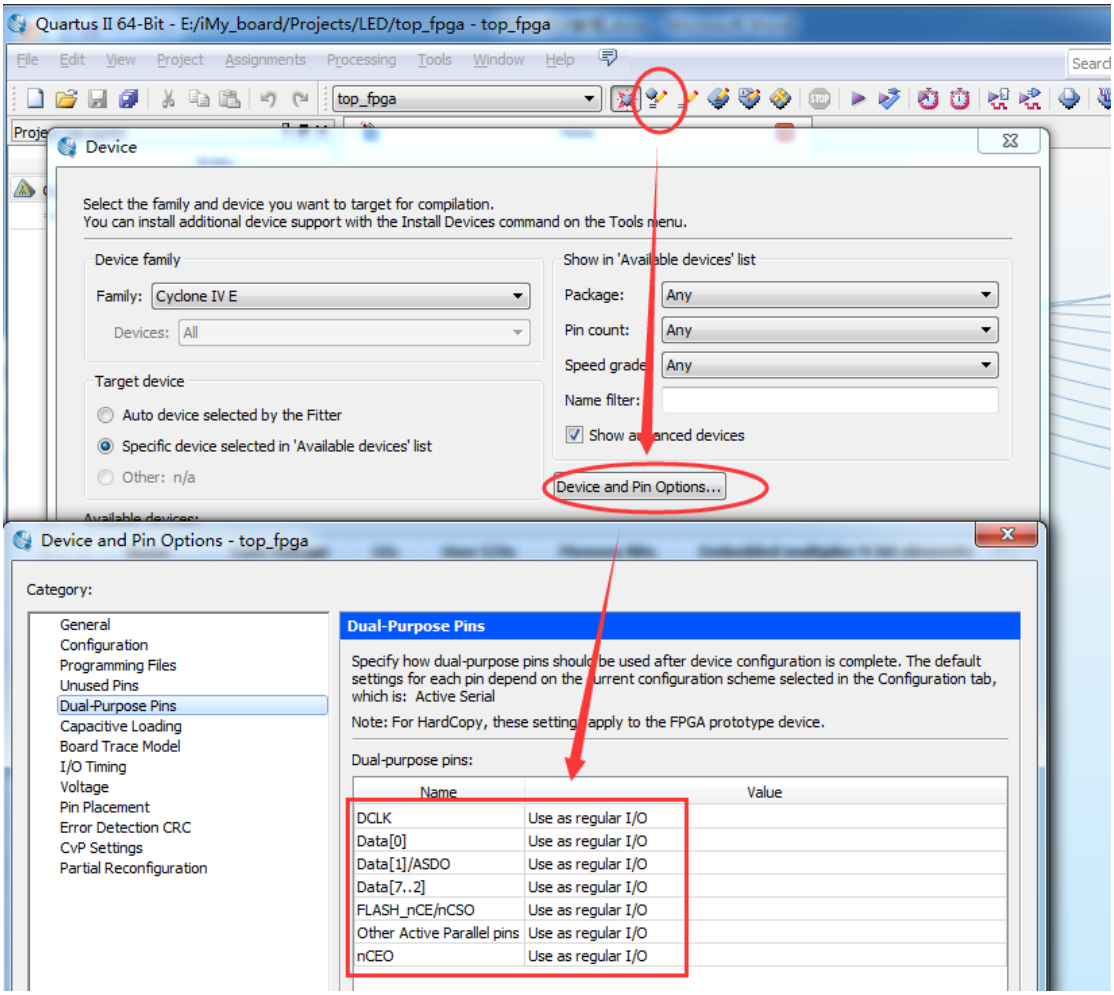
always @(posedge clk or negedge rst_n)
    if (~rst_n)
        timer <= 0; //计数器清零
    else if (timer == 32'd23_999_999) //0.5 秒计数(50M-1=49_999_999)
        timer <= 0; //计数器清零
    else
        timer <= timer + 1'b1; //计数器加 1

//LED 控制
always @(posedge clk or negedge rst_n)
    if (~rst_n)
        led_r <= 1'b0; //LED 灯灭
    else if (timer == 32'd12_999_999) //计数器计到 0.5 秒，
        led_r <= 1'b1; //LED 点亮
    else if (timer == 32'd23_999_999) //计数器计到 1 秒，
        led_r <= 1'b0; //LED 灯灭
    else
        led_r <= led_r;

endmodule
```

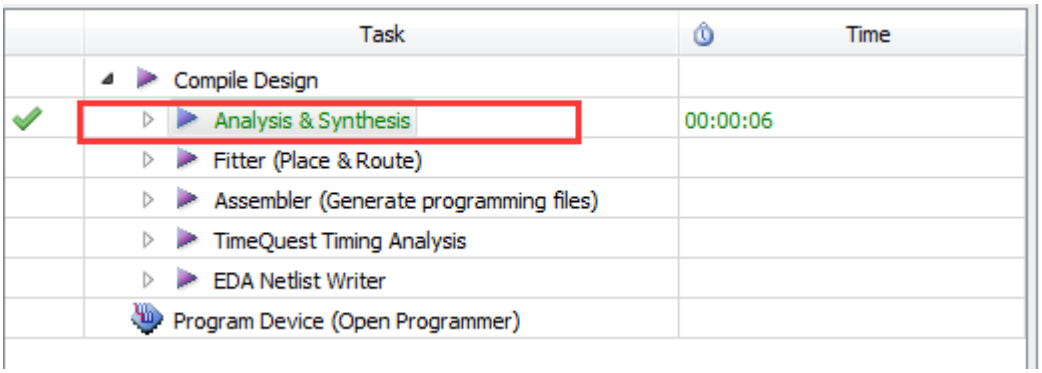
编写好代码后，保存为 top_fpga.v。

6.4. 设置 Dual-Purpose Pins



6.5. 编译工程与管脚约束

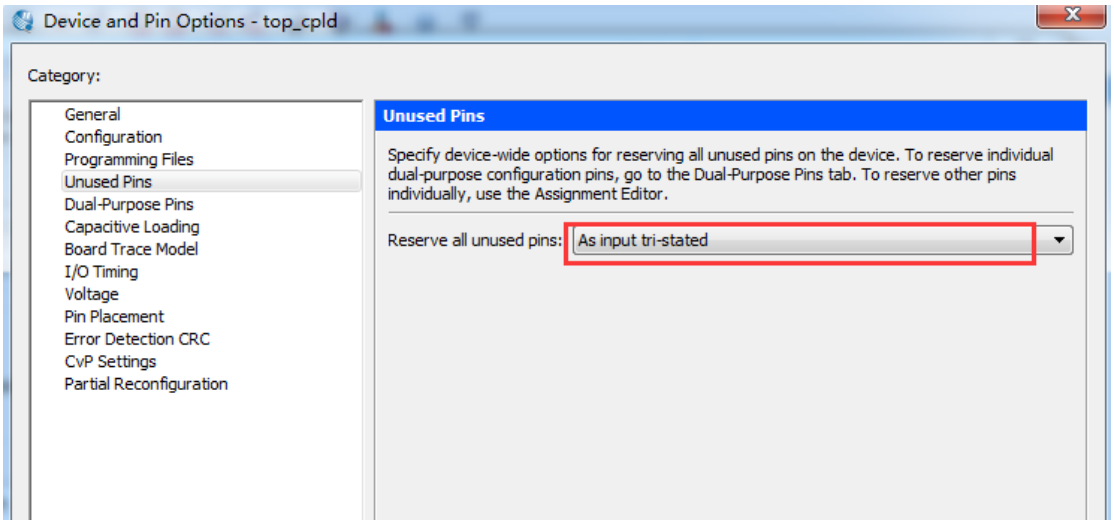
点击 Analysis & Synthesis 按钮，开始综合代码。



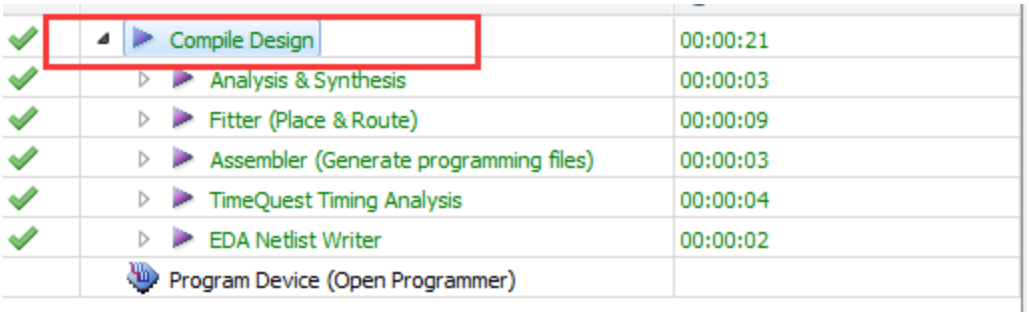
选择菜单 Assignment -> Pin Planner，添加管脚约束。

Node Name	Direction	Location	I/O Bank
clk	Input	PIN_M16	5
led	Output	PIN_L12	5
rst_n	Input	PIN_M12	5

设置 Unused Pins 为输入三态。



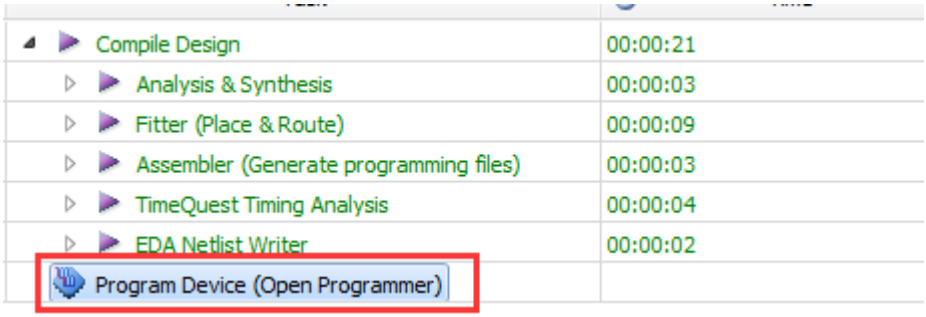
点击 Compile Design 按钮，全编译工程。



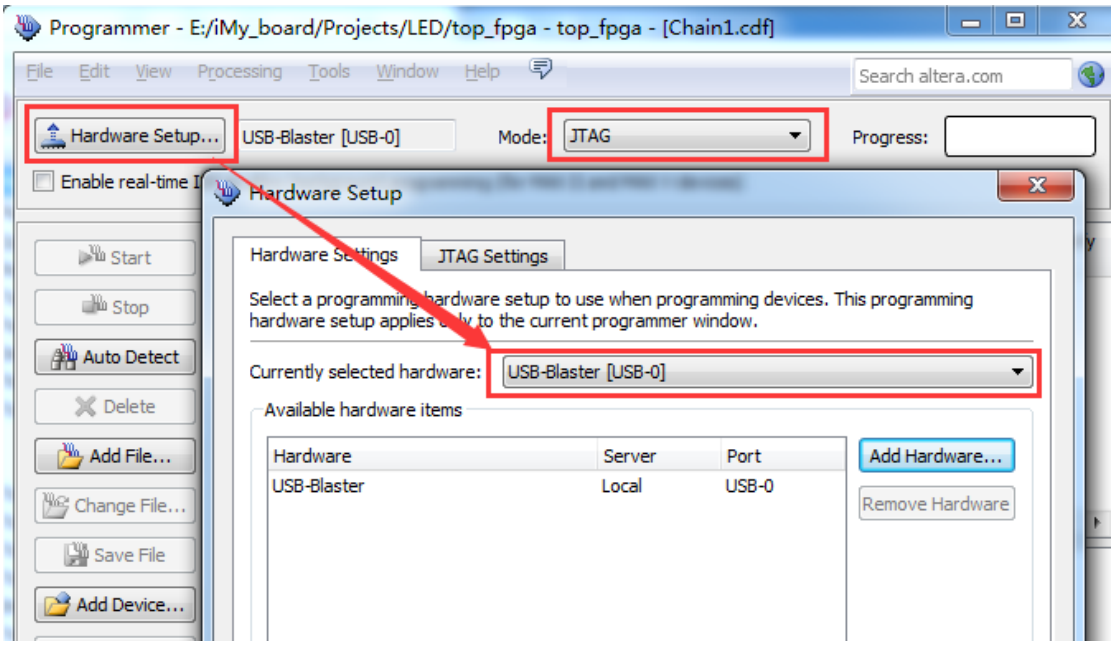
6.6. 下载 sof 文件

一般我们用.sof 文件来验证我们的设计是否正确，.sof 文件用来在线调试 FPGA，因为 FPGA 断电后电路就恢复初始状态了，所以断电后.sof 文件也就没有了。

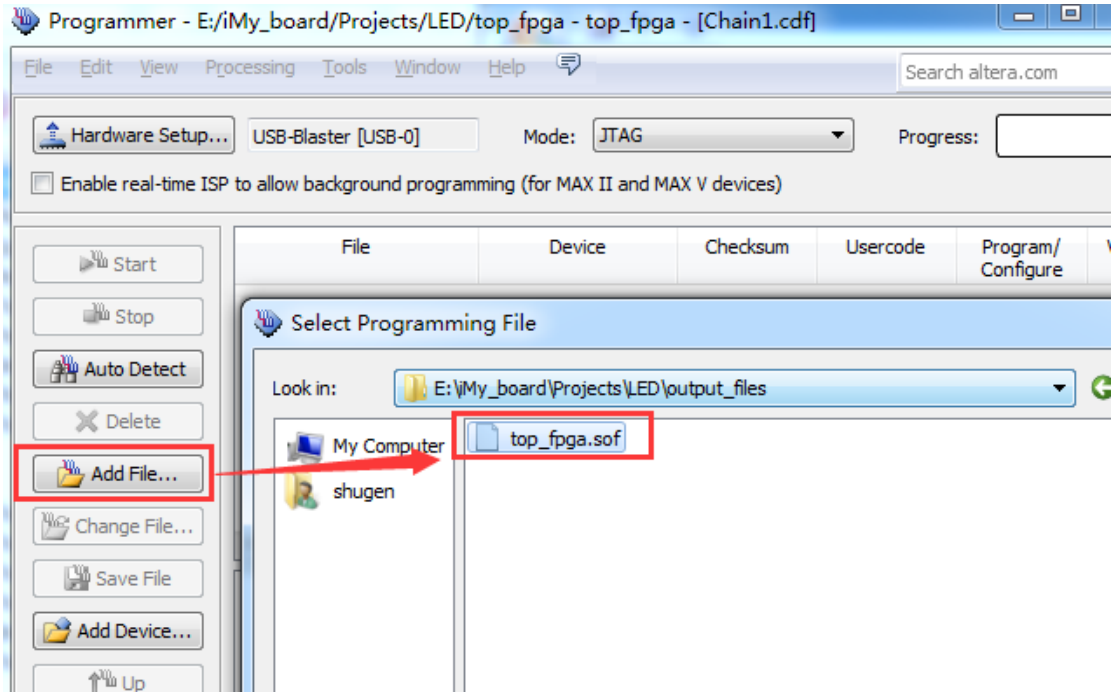
点击 Program Device(Open Programmer)。



点击 Hardware Setup 选择 USB-Blaster[USB-0]，选择 JTAG Mode。



选择下载文件 top_fpga.sof，在目录 output_files 下。

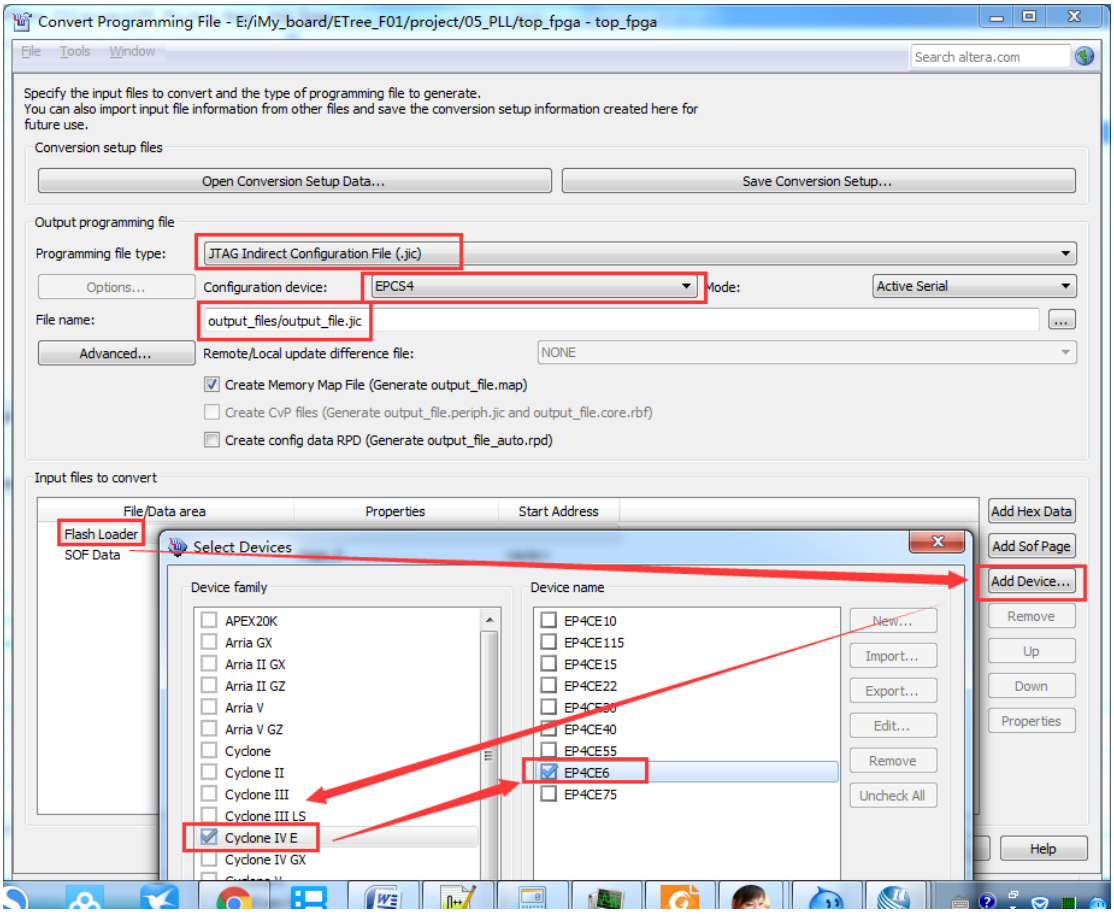


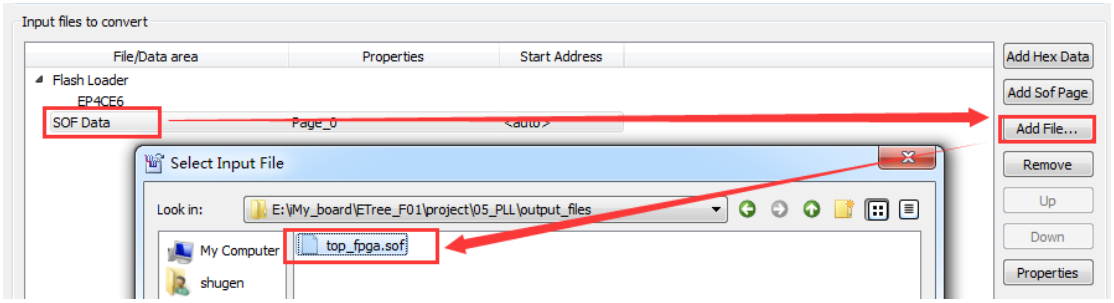
点击"Start"按钮，下载成功后可以看到 100%的绿色精度条，LED 开始闪烁。

6.7. jic 文件生成

在代码设计验证完成后，我们需要将编译后的配置文件固化到外部 flash，这样每次上电后 flash 会自动配置 FPGA。JIC 文件用于固化配置芯片 EPCS4(SPI Flash)，保存 FPGA 的配置文件，这样在断电重启后不需要再下载代码。

点击 File | Convert Programming File 弹出下面对话框，按照图示设置，Device Family 为 Cyclone IV，Device name 为 **EP4CE10**。

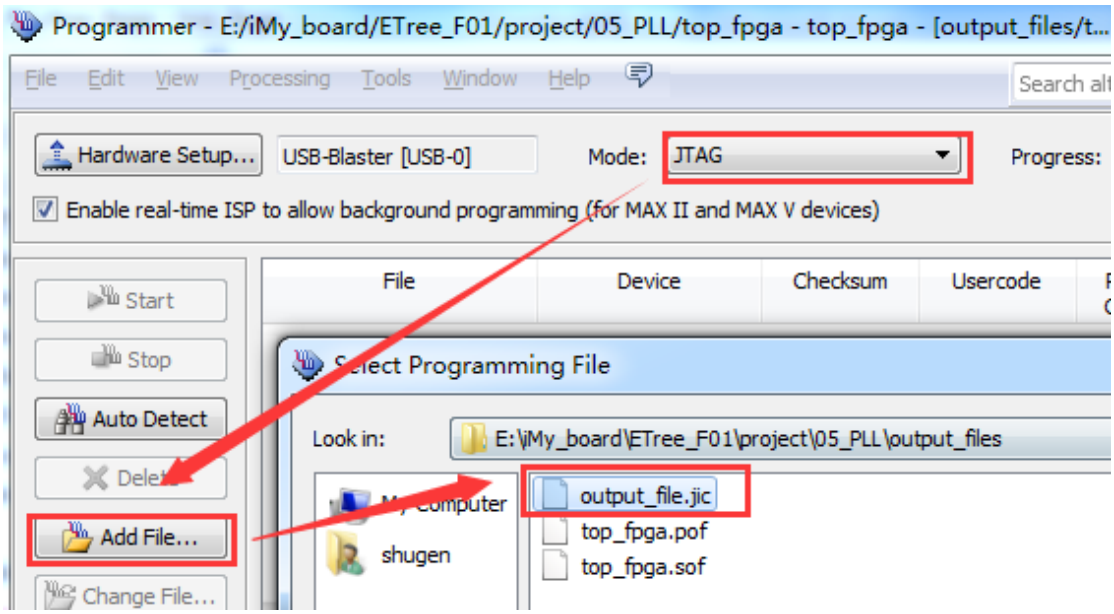




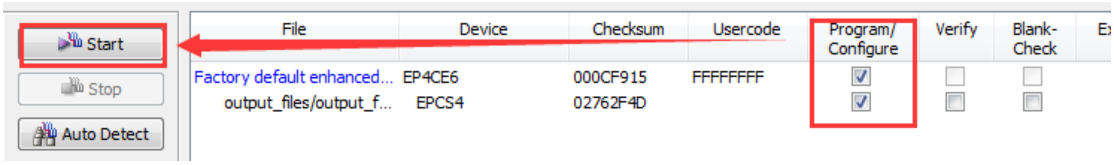
最后点击 Generate 生成 output.jic 文件。

6.8. 下载 jic 文件

这里依然选择 JTAG 模式，然后点击 Add File 添加 output_file.jic 文件，如下图所示。



添加好文件后，需要勾选"Program/Configure"，点击 Start 开始下载。



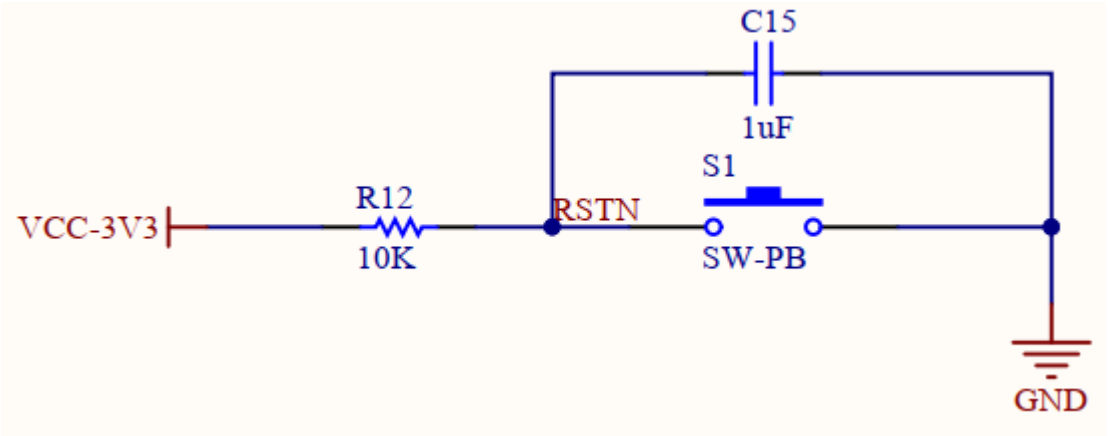
下载成功后可以看到 100%的绿色精度条，断电重启后 LED 不停闪烁。

第七章 按键检测

本章在上一章的基础上，增加按键暂停/开始功能，即按键按下灯停止闪烁，再按下灯继续闪烁。

7.1. 硬件介绍

硬件电路如下图所示，这里我们用到按键“S1”，低电平有效(按键按下时，KEY0 被接到 GND)。

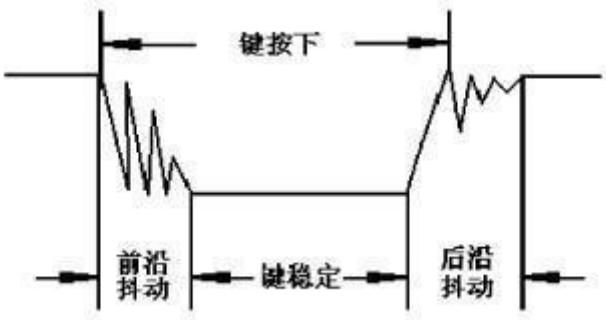


7.2. HDL 设计

由于硬件和操作原因，一般按键按下或释放时，有大约 20ms 的抖动时间，这个时间段电平的状态是不停变化的，FPGA 无法正确捕捉。这里我们要将这 20ms 时间内的抖动信号去除掉。

简单的方法，就是当按键按下后，不停地检测对应 PIN 脚的状态，若能延续超过 20ms 以上为低电平，则认为按键按下一次有效。

按键的抖动示意图如下所示。



下面为按键去抖模块代码：

```
//file name:      key_jitter.v
//author:         ETree
//date:           2017.7.23
```

```
//function:       按键消抖
//log:

module key_jitter(
       input clk,             //24MHz
       input key_n,           //低电平有效
       output reg key_valid //高电平有效
);

reg [19:0] cnt0;

always @(posedge clk)
begin
    cnt0 <= cnt0 + 1'b1;
end

reg [3:0] key_down_check;

always @(posedge clk)
begin
    if(cnt0 == 20'hf0000)
        key_down_check <= {key_down_check[2:0],~key_n};
    else
           key_down_check <= key_down_check;
end

reg key_valid_r0;

always @(posedge clk)
begin
    key_valid_r0 <= (key_down_check==4'b0011)?1'b1:1'b0;
end

reg key_valid_r1;
reg key_valid_r2;

always @(posedge clk)
begin
    key_valid_r1 <= key_valid_r0;
    key_valid_r2 <= key_valid_r1;
end

wire key_valid_pos;
```

```
assign key_valid_pos = ({key_valid_r2,key_valid_r1}==2'b01)?1'b1:1'b0;

always @(posedge clk)
begin
    key_valid <= (key_valid_pos==1'b1)?1'b1:1'b0;
end

endmodule
```

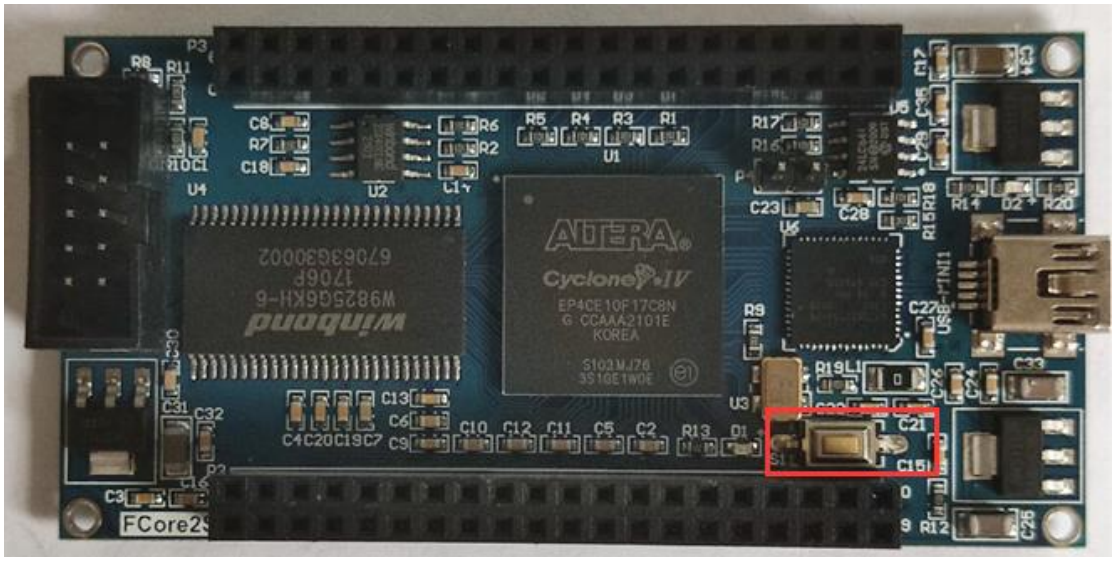
7.3. 管脚分配

选择菜单 Assignment -> Pin Planner，添加管脚约束。

Node Name	Direction	Location	I/O Bank
clk	Input	PIN_M16	5
led	Output	PIN_L12	5
rst_n	Input	PIN_M12	5

7.4. 实验操作

将 FPGA 配置文件下载到板卡后，默认状态 LED 每隔 1 秒闪烁一次，按下板卡下方的按键后，LED 停止闪烁保持不变；再按下按键后，LED 继续每隔 1 秒闪烁，如此周而复始。



第八章 PLL 锁相环

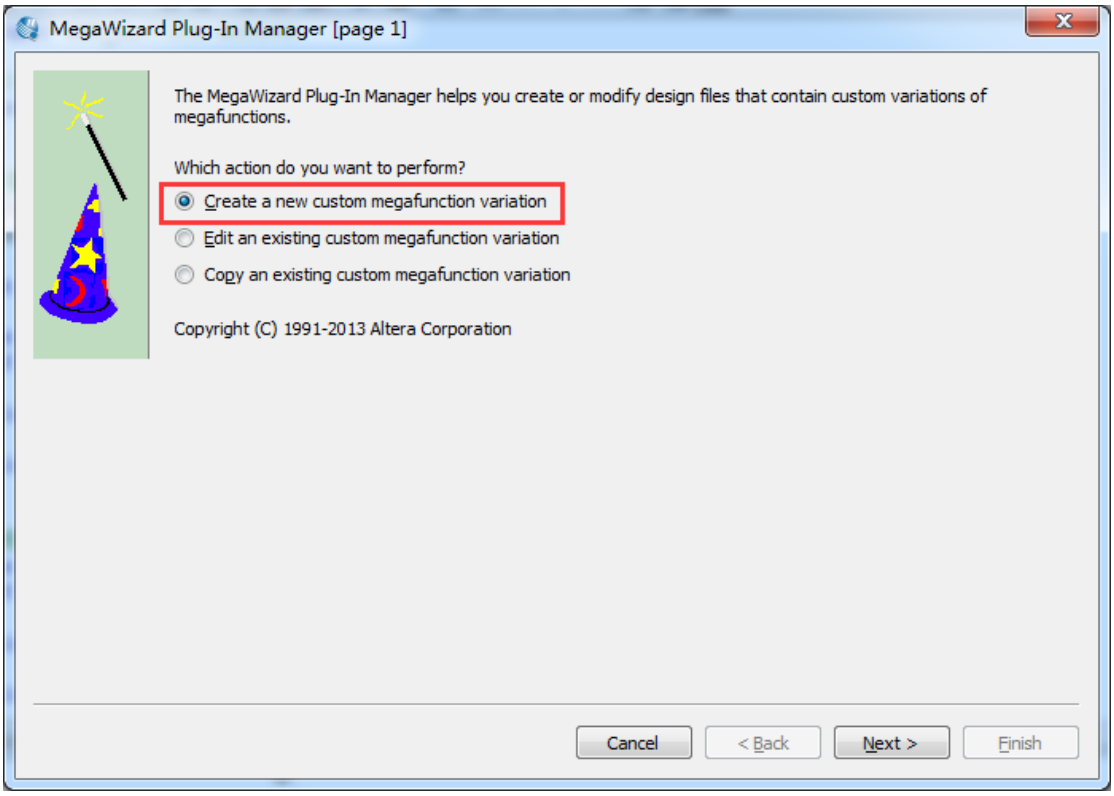
前面几章都没有涉及到 Quarut II 自带的 IP 组件，即 MegaWizard。这章学习如何使用 MegaWizard 创建一个 PLL 锁相环。

8.1. 硬件介绍

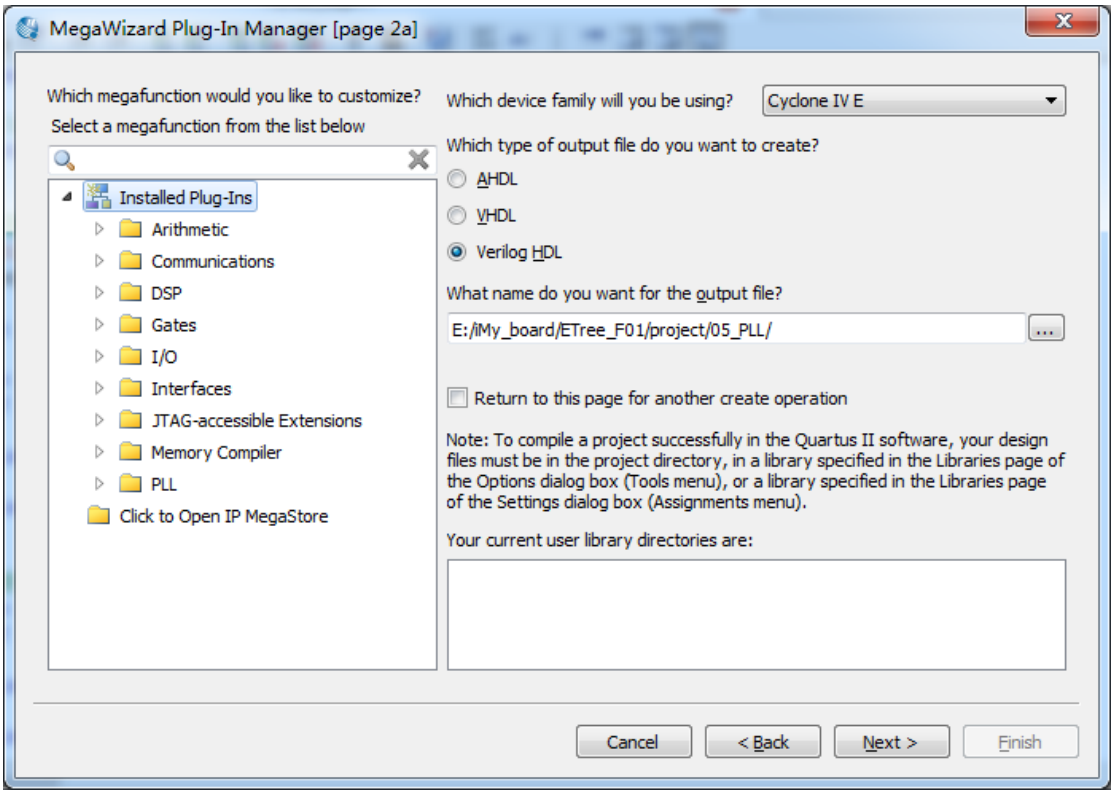
本章使用“第六章 LED 闪烁”的硬件平台，这里不再详述。

8.2. MegaWizard

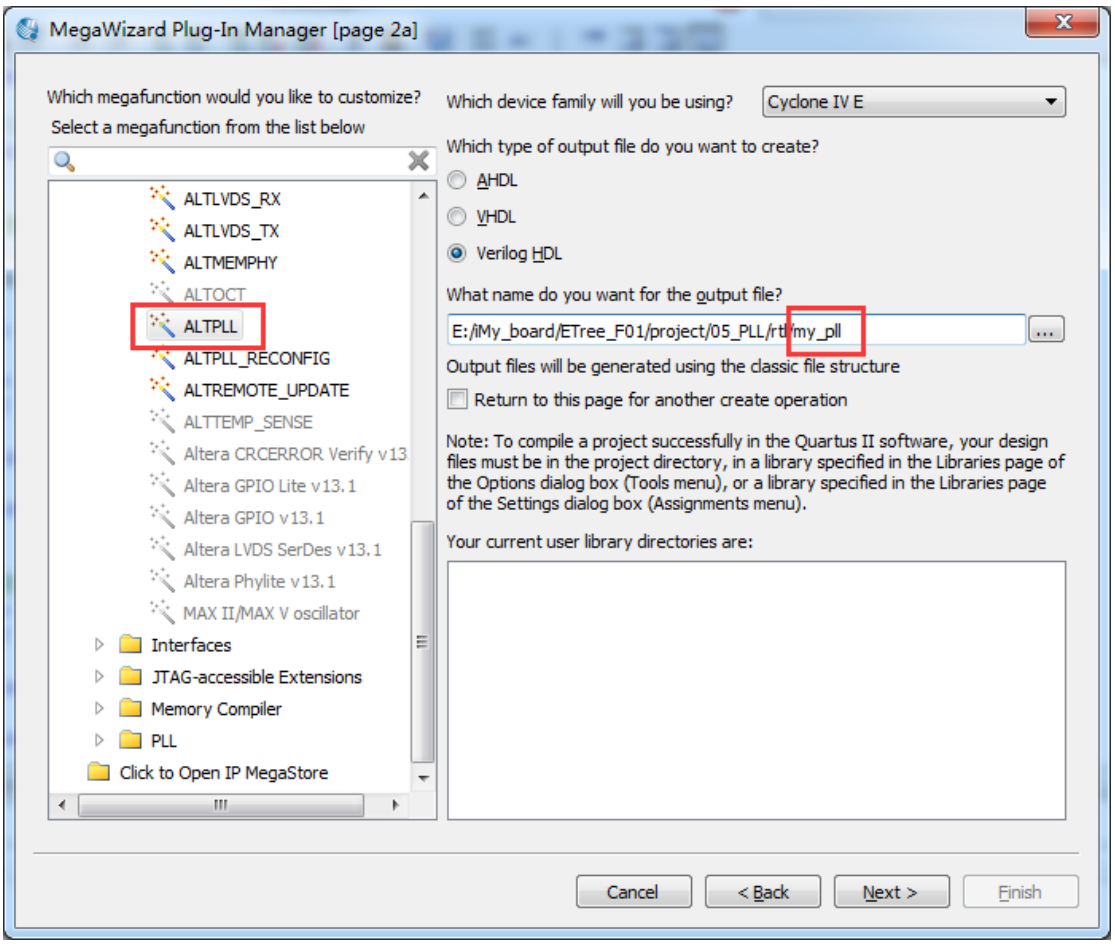
打开 Quartus II，点击 Tools | MegaWizard Plug-In Manager，弹出下面的对话框，选择创建一个新的 megafunction variation。



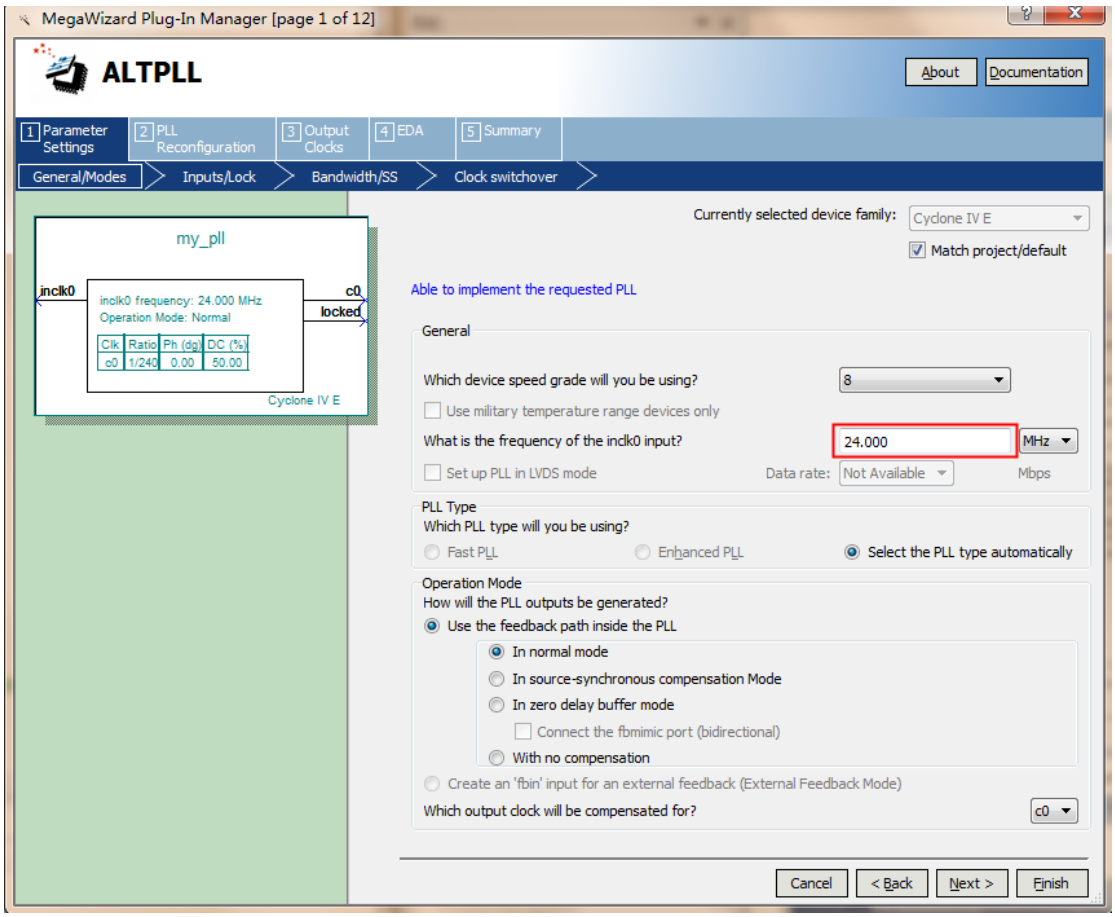
点击“Next”，弹出下面的对话框。在左侧栏中，可以看到有 9 类 IP 组件。Arithmetic 包含基本加、减、乘、除、开方等数学运算；DSP 包含各种 FIR、FFT、数字图像处理等算法；I/O 包含 DDIO、LVDS、ALTPLL 等；Interface 包含 DDR、DDR2、PCI 等各类接口；Memory Compiler 包含 FIFO、双口 RAM 等存储结构单元。



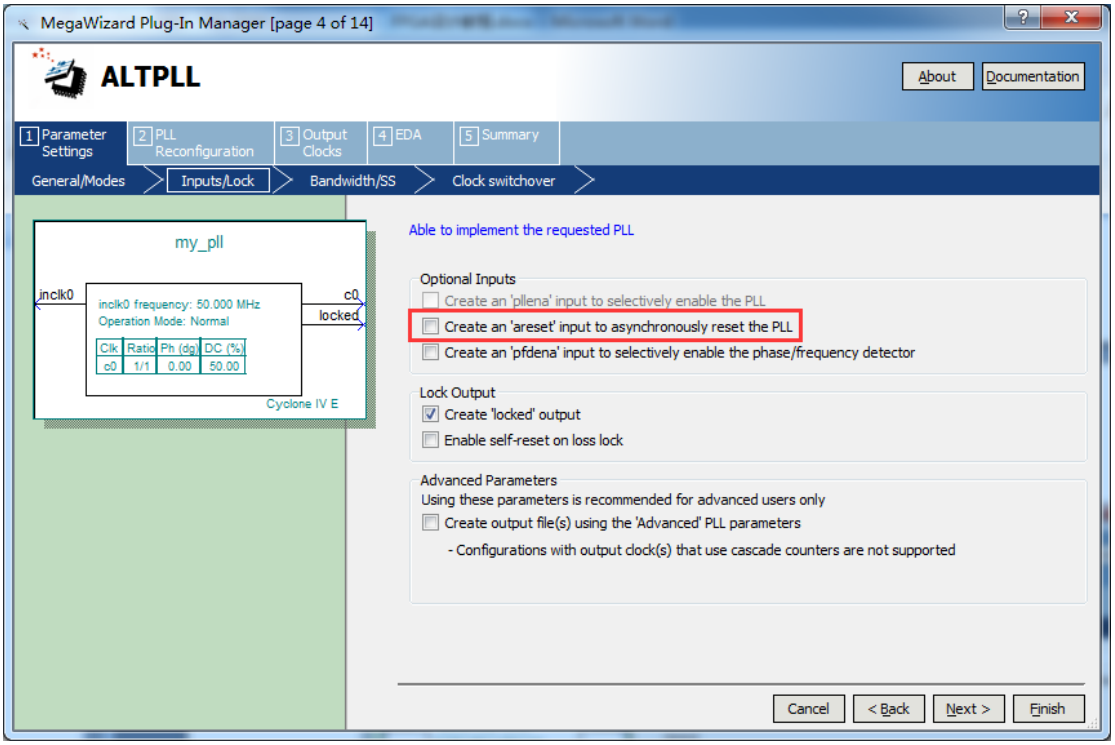
这里我们使用 I/O 中的 ALTPLL 实现 PLL 锁相环，如下图所示，命名为“my_pll”。



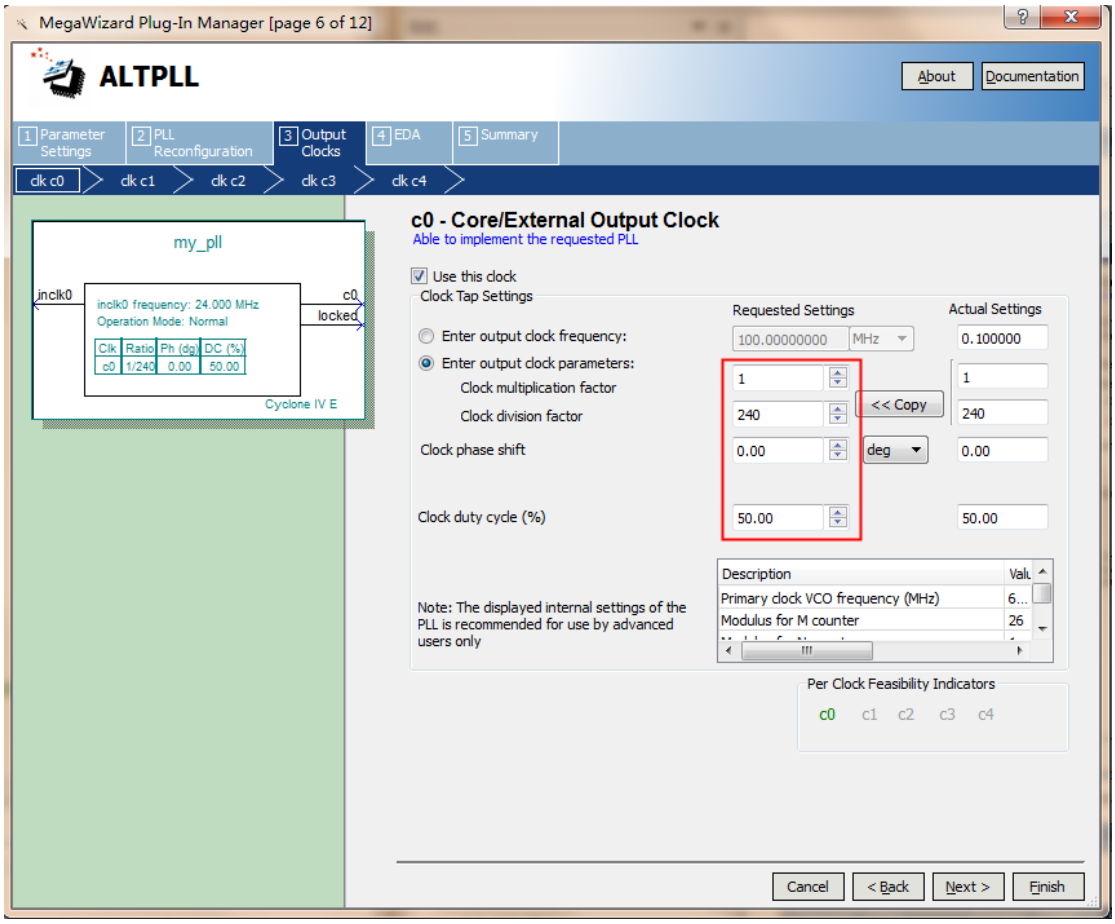
点击“Next”，弹出下面对话框，修改输入时钟为 24MHz。



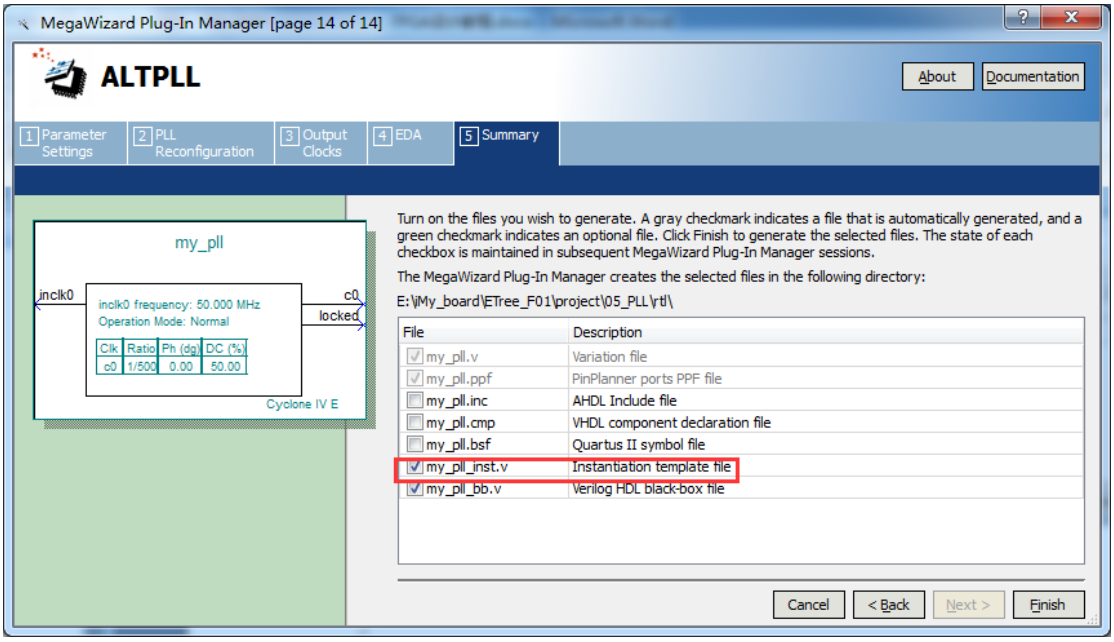
点击“Next”，取消勾选生成'areset'，如下图所示。



点击“Output Clocks”，设置时钟分频系数为 240，相位为 0 度，占空比为 50%，这样生成了需要的 100KHz 时钟，如下图所示。



点击“Summary”，勾选"my_pll_inst.v"生成例化文件，如下图所示。



点击“Finish”完成。

8.3. HDL 设计

top_fpga.v 代码设计如下：

```
//file name:      top_fpga.v
//author:         ETree
//date:           2017.9.9
//function:       top file of project
//log:            led light

module top_fpga(
    //global signal
    input clk,  //24MHz
    input rst_n,//low active

    //led
    output  led //high active
);

//----------------PLL----------------
wire locked;
wire clk_100k;

my_pll  my_pll_inst (
    .inclk0 ( clk ),
    .c0 ( clk_100k ),
    .locked ( locked )
    );

//寄存器定义
reg [31:0] timer=0;
reg led_r=0;

assign led = led_r;

always @(posedge clk_100k or negedge rst_n)
    if (~rst_n)
        timer <= 0; //计数器清零
    else if (timer == 32'd99_999) //0.5 秒计数(100K-1=99_999)
        timer <= 0; //计数器清零
    else
        timer <= timer + 1'b1; //计数器加 1

//LED 控制
always @(posedge clk or negedge rst_n)
    if (~rst_n)
        led_r <= 1'b0; //LED 灯灭
    else if (timer == 32'd49_999) //计数器计到 0.5 秒,
```

```
        led_r <= 1'b1; //LED 点亮
    else if (timer == 32'd99_999) //计数器计到 1 秒,
        led_r <= 1'b0; //LED 灯灭
    else
        led_r <= led_r;

endmodule
```

8.4. 实验操作

下载配置文件后，LED 每隔一秒闪烁一次，和“第六章 LED 闪烁”等效，PLL 正常工作。

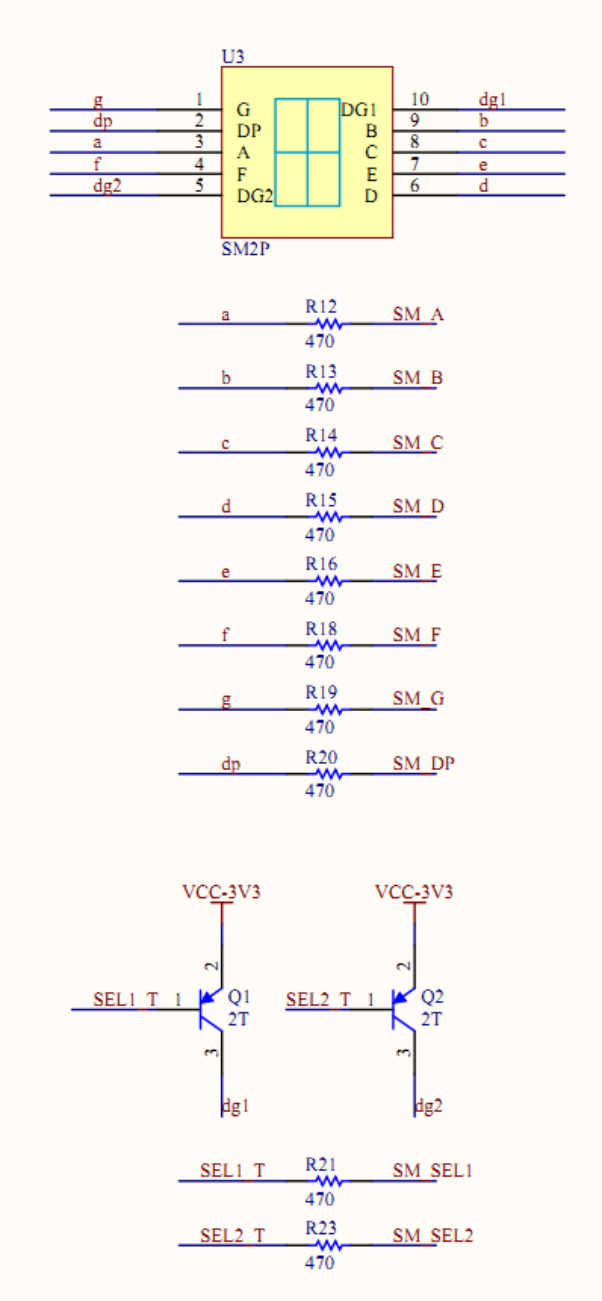
第九章 数码管秒表

本章基于 2 位数码管实现一款秒表，需要外接 VGA_SM 模块板卡。

9.1. 硬件介绍

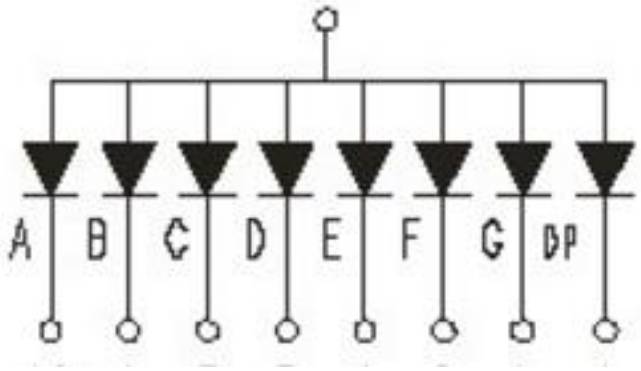
我们知道发光二极管常常用于各种电子产品中作为指示灯，除发光二极管外，常见的用于显示的器件还有数码管，比如电子时钟的显示时间就是用的数码管，万用表中的显示屏也是用的数码管。其实数码管的本质就是发光二极管的组合使用，最常见的就是七段和八段数码管，七段数码管是由 7 个长条形的发光二极管组成，八段数码管比七段数码管多了一个点(通常是右下角的一个点)。

八段数码管硬件电路如下：



八段数码管引脚图如下：数码管为公阳极，即八条 LED 的阳极公用一个端点，这样在 LED 的阴极低电平时 LED 就会发光，LED 阴极为高电平时，LED 就会熄灭，通常我们简称“低电平有效”。





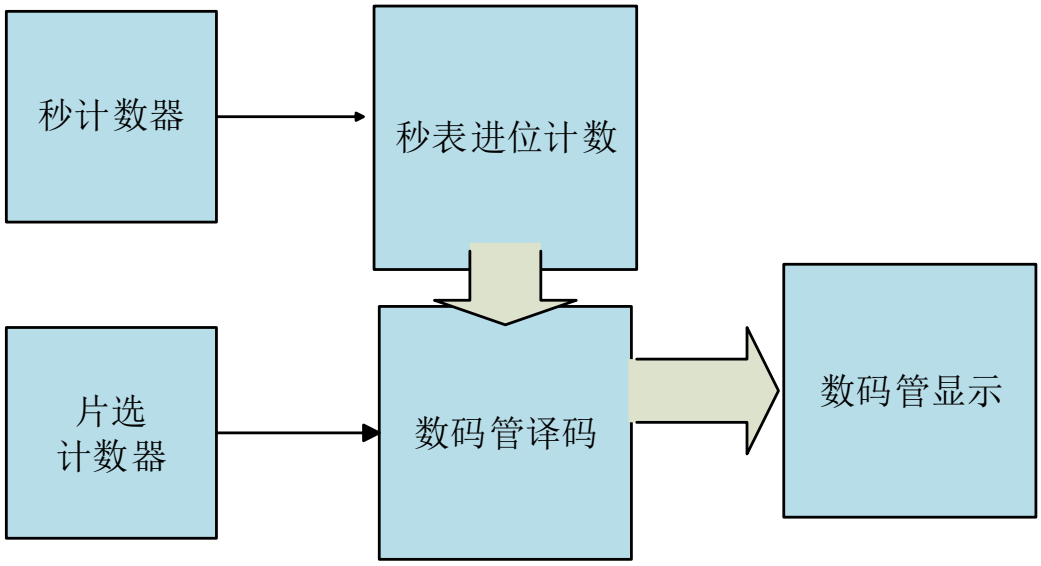
根据八段数码管的引脚图，定义 parameter 常量来对应数码显示，代码如下：

```
parameter ZERO    = 8'b0000_0011,//8'b1100_0000,
          ONE     = 8'b1001_1111,//8'b1111_1001,
          TWO     = 8'b0010_0101,//8'b1010_0100,
          THREE   = 8'b0000_1101,//8'b1011_0000,
          FOUR    = 8'b1001_1001,//8'b1001_1001,
          FIVE    = 8'b0100_1001,//8'b1001_0010,
          SIX     = 8'b0100_0001,//8'b1000_0010,
          SEVEN   = 8'b0001_1111,//8'b1111_1000,
          EIGHT   = 8'b0000_0001,//8'b1000_0000,
          NINE    = 8'b0000_1001,//8'b1001_0000;
          AA      = 8'b0001_0001,
          BB      = 8'b1100_0001,
          CC      = 8'b0110_0011,
          DD      = 8'b1000_0101,
          EE      = 8'b0110_0001,
          FF      = 8'b0111_0001;
```

其中参数编码的高位到地位分别表示：{a, b, c, d, e, f, g, dp}。

9.2. HDL 设计

- 系统框架设计



➢ 秒表进位计数

```
//----------秒表进位计数器------------
always @(posedge clk or negedge rst_n)
    if(~rst_n)
        second_0 <= 0;
    else if(timer==0)
        if(second_0<9)
            second_0 <= second_0 + 1'b1;
        else
            second_0 <= 0;
    else
        second_0 <= second_0;

always @(posedge clk or negedge rst_n)
    if(~rst_n)
        second_1 <= 0;
    else if(timer==0)
        if(second_0==9)
            if(second_1<5)
                second_1 <= second_1 + 1'b1;
            else
                second_1 <= 0;
        else
            second_1 <= second_1;
    else
        second_1 <= second_1;
```

当 second_0 计数到 9 后，产生进位信号；当 second_1 计数到 5 后，等到各位计数到 9 后，清零。

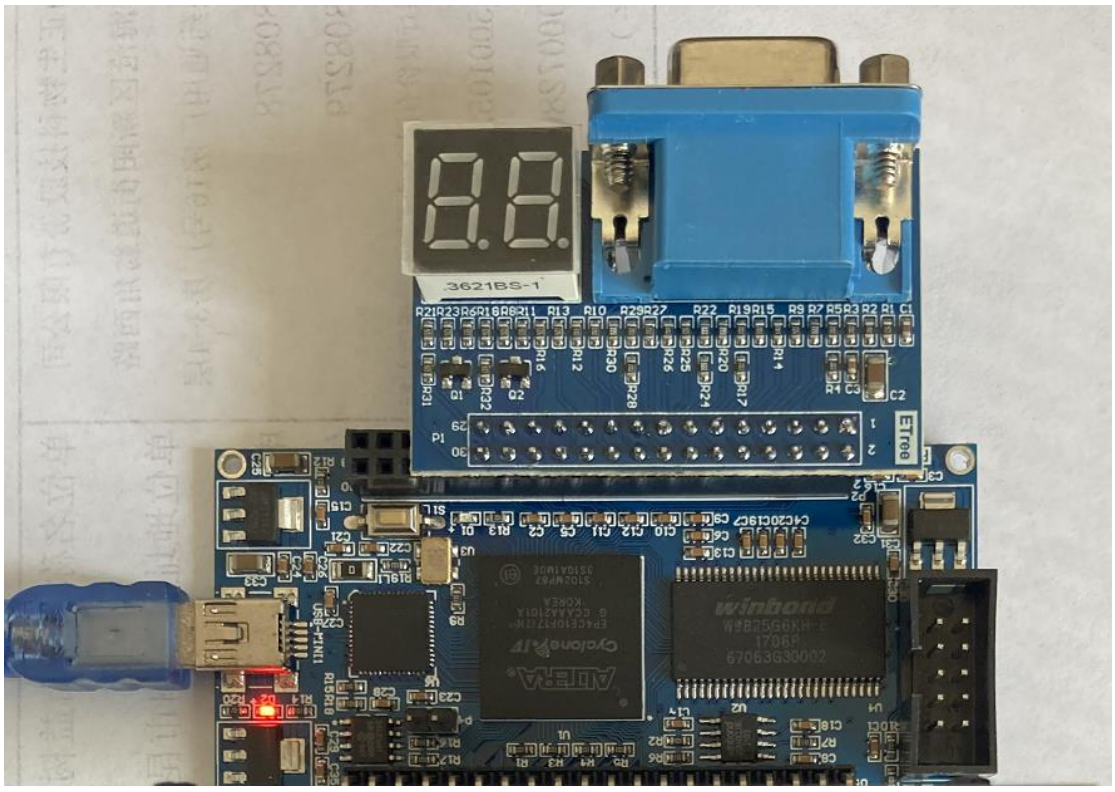
9.3. 管脚分配

选择菜单 Assignment -> Pin Planner，添加管脚约束。

Node Name	Direction	Location	I/O Bank
in clk	Input	PIN_M16	5
out led	Output	PIN_L12	5
in rst_n	Input	PIN_M12	5
out sm_dat[7]	Output	PIN_R8	3
out sm_dat[6]	Output	PIN_T7	3
out sm_dat[5]	Output	PIN_T9	4
out sm_dat[4]	Output	PIN_T8	3
out sm_dat[3]	Output	PIN_T10	4
out sm_dat[2]	Output	PIN_R9	4
out sm_dat[1]	Output	PIN_T11	4
out sm_dat[0]	Output	PIN_R10	4
out sm_sel[1]	Output	PIN_P14	4
out sm_sel[0]	Output	PIN_R14	4

9.4. 实验操作

插入 VGA_SM 模块(注意 1 脚对应 P2 的 1 脚)，插入 Mini-USB 线缆，如下图所示：



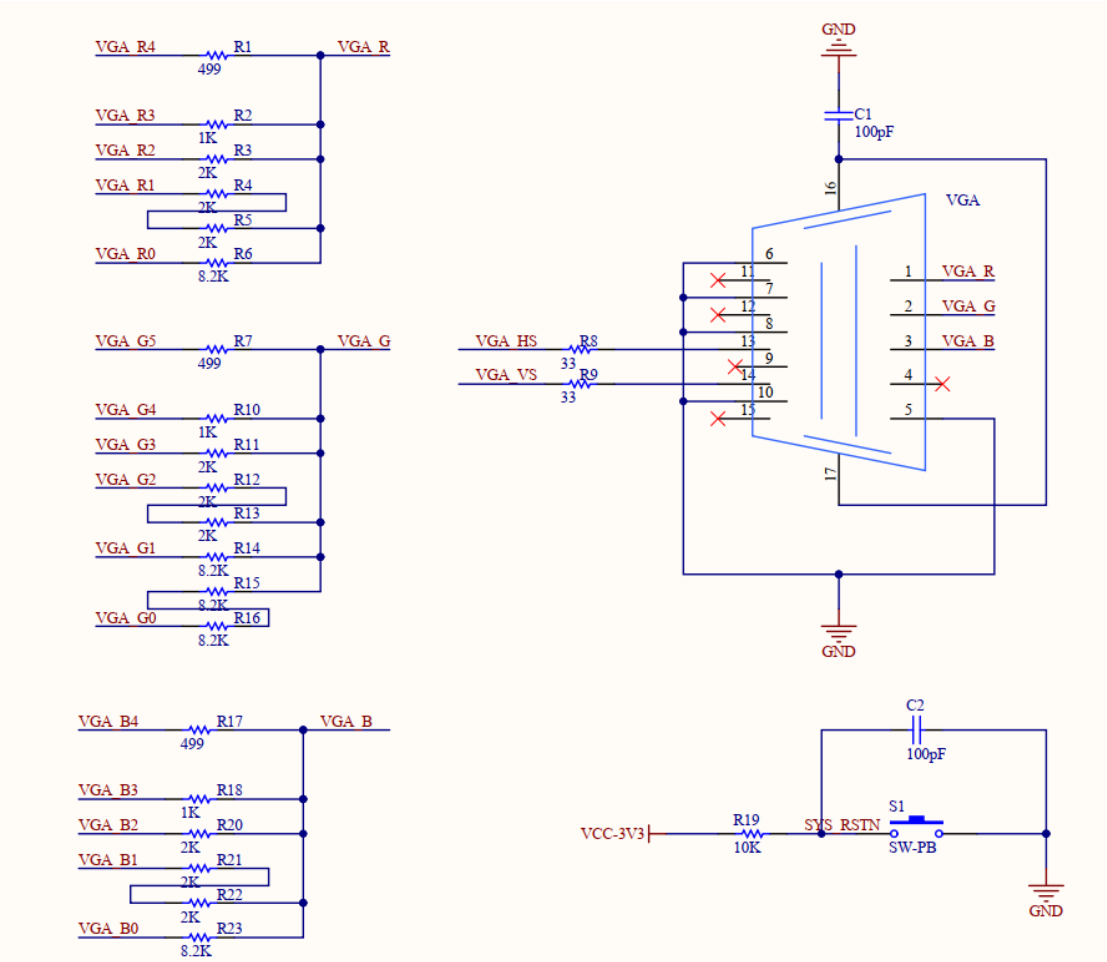
下载 FPGA 配置文件，数码管便开始秒表计数。

第十章 VGA 彩条显示

本章基于 VGA 接口，实现显示器彩条显示，需要外界 VGA_SM 模块。

10.1. 硬件介绍

FPGA 板卡采用电阻并联分压的方式实现了 VGA565(RED->5bit, GREEN->6bit, BLUE->5bit)接口，可以实现 65536 种颜色的显示，具体原理图如下所示。



考虑到底成本方案，我们采用了电阻分压网络模拟 DAC 来替换 ADV7123 视频转换芯片。下面简单描述一下设计原理。

三基色源端和终端匹配电阻都为 75 欧姆，如下图所示：

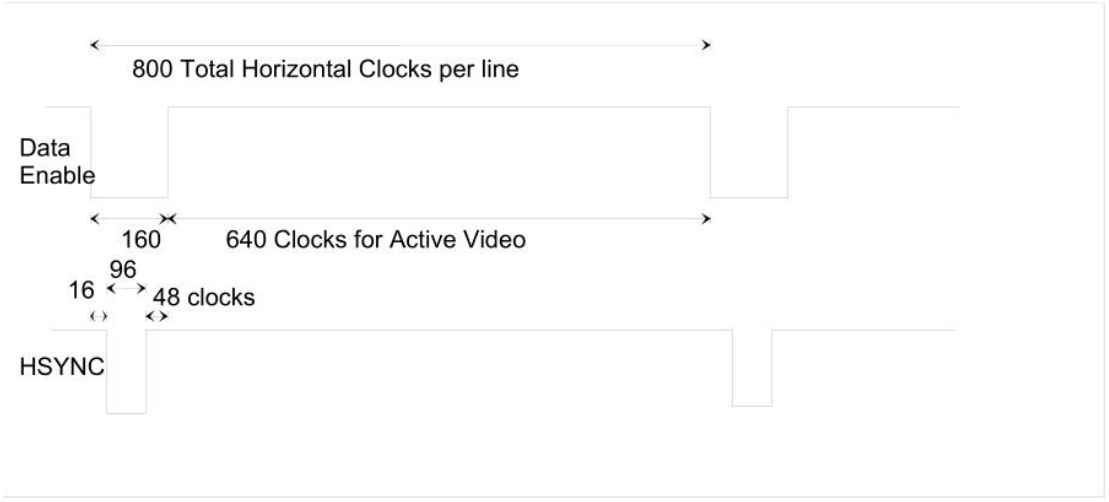


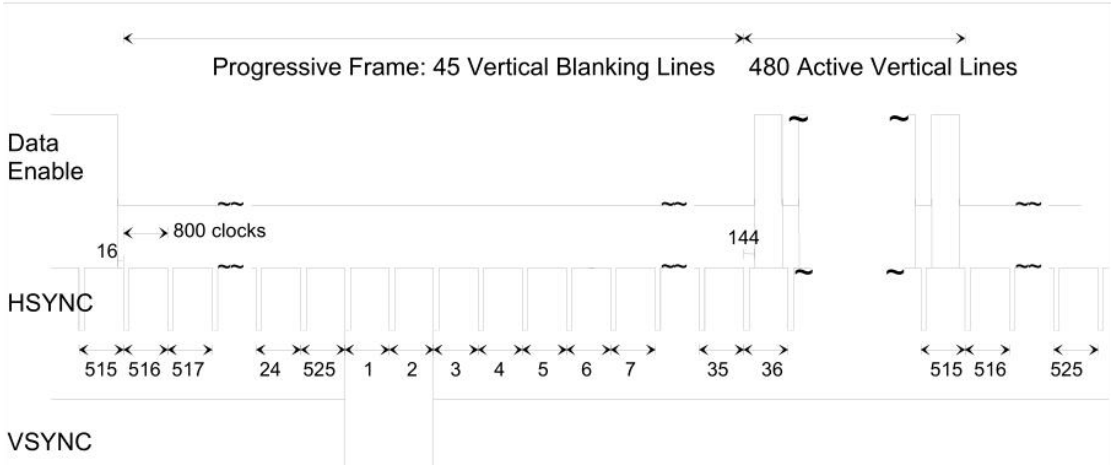
VGA 接口规定三基色分别采用模拟电压 0～0.714V 来表示，0~0.714V 可以通过不同的电阻网络分压得出。

10.2. VGA 时序

VGA(Video Graphics Array)是 IBM 在 1987 年推出的一种视频传输，具有分辨率高、显示速率快、颜色丰富等优点，在彩色显示器领域得到了广泛的应用。不支持热插拔，不支持音频传输。

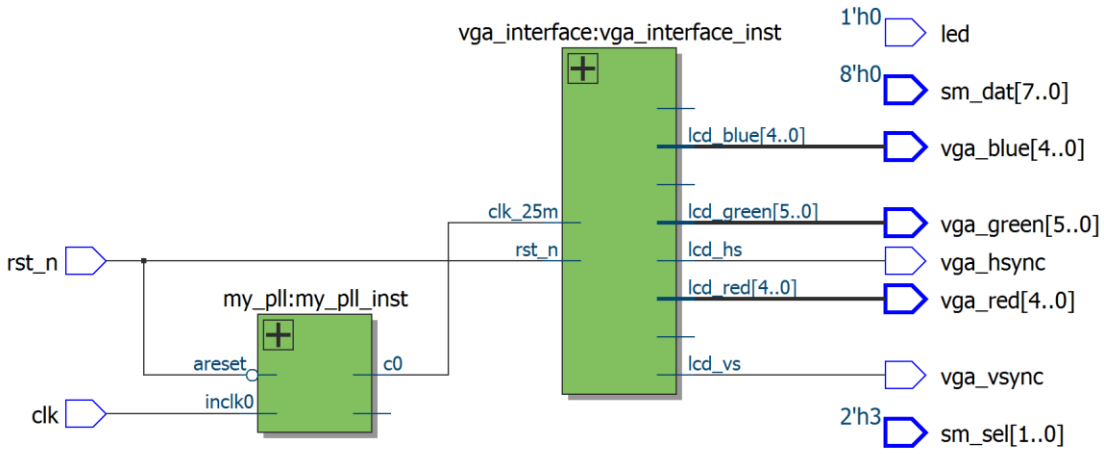
本设计使用的是 640×480@60Hz 的 VGA 时序接口，计算像素时钟 pclk = 800*525*60 = 25200000，注意这里的 HSYNC(行同步信号)和 VSYNC(场同步信号)都是低电平有效，且 HSYNC 在数据行无效区域也要提供。





10.3. HDL 设计

- 系统框架



✧ my_pll 模块：生成需要的时钟。

✧ vga_interface：生成需要的 VGA 时序接口。

➢ VGA 模块

```
//file name:    vga_show.v
//author:       ETree
//date:         2021.10.2
//function:     VGA 显示器驱动
//log:

module vga_show(
    input       vga_clk,
    input       rst_n,                  //VGA 复位，低电平有效

    output      vga_hsync,
    output      vga_vsync,
    output [4:0] vga_r,
    output [5:0] vga_g,
    output [4:0] vga_b
);

parameter RED      = 16'b11111_000000_00000;
parameter GREEN = 16'b00000_111111_00000;
parameter BLUE    = 16'b00000_000000_11111;
parameter YELLOW  = 16'b11111_111111_00000;

// 水平扫描参数的设定 640*480 VGA 60FPS_25MHz
parameter LinePeriod   = 800;           //行周期数
parameter H_SyncPulse  = 96;            //行同步脉冲（Sync a）
parameter H_BackPorch  = 48 ;           //显示后沿（Back porch b）
parameter H_ActivePix  = 640;           //显示时序段（Display interval c）
parameter H_FrontPorch = 16;            //显示前沿（Front porch d）
```

```
parameter Hde_start    = 144;
parameter Hde_end      = 784;

// 垂直扫描参数的设定 640*480 VGA
parameter FramePeriod  = 525;          //列周期数
parameter V_SyncPulse  = 2;            //列同步脉冲（Sync o）
parameter V_BackPorch  = 33 ;          //显示后沿（Back porch p）
parameter V_ActivePix  = 480;          //显示时序段（Display interval q）
parameter V_FrontPorch = 10;           //显示前沿（Front porch r）
parameter Vde_start    = 35;
parameter Vde_end      = 515;


reg [10 : 0] h_cnt;
reg [9 : 0]  v_cnt;
reg [4 : 0]  vga_r_reg;
reg [5 : 0]  vga_g_reg;
reg [4 : 0]  vga_b_reg;


reg hsync_r;
reg vsync_r;
reg vsync_de;
reg hsync_de;

//水平扫描计数
always @(posedge vga_clk)
    if(~rst_n)
        h_cnt <= 1;
    else if(h_cnt==LinePeriod)
        h_cnt <= 1;
    else
        h_cnt <= h_cnt+ 1'b1;

//水平扫描信号 hsync,hsync_de 产生
always @(posedge vga_clk)
begin
    if(~rst_n)
        hsync_r <= 1'b1;
    else if(h_cnt==1)
        hsync_r <= 1'b0;    //产生 hsync 信号
    else if(h_cnt==H_SyncPulse)
        hsync_r <= 1'b1;
```

```
    if(~rst_n)
        hsync_de <= 1'b0;
    else if(h_cnt==Hde_start)
        hsync_de <= 1'b1;   //产生 hsync_de 信号
    else if(h_cnt==Hde_end)
        hsync_de <= 1'b0;

end

//垂直扫描计数
always @(posedge vga_clk)
    if(~rst_n)
        v_cnt <= 1;
    else if(v_cnt==FramePeriod)
        v_cnt <= 1;
    else if(h_cnt==LinePeriod)
        v_cnt <= v_cnt + 1'b1;


// 垂直扫描信号 vsync, vsync_de 产生
always @(posedge vga_clk)
begin
    if(~rst_n)
        vsync_r <= 1'b1;
    else if(v_cnt==1)
        vsync_r <= 1'b0;    //产生 vsync 信号
    else if(v_cnt==V_SyncPulse) vsync_r <= 1'b1;

    if(~rst_n)
        vsync_de <= 1'b0;
    else if(v_cnt==Vde_start)
        vsync_de <= 1'b1;    //产生 vsync_de 信号
    else if(v_cnt==Vde_end)
        vsync_de <= 1'b0;
end

//生成彩条测试数据
always @(posedge vga_clk)
begin
    if(~rst_n)
        {vga_r_reg,vga_g_reg,vga_b_reg} <= 0;
    else if(h_cnt>=Hde_start && h_cnt<(Hde_start+160) && vsync_de)
        {vga_r_reg,vga_g_reg,vga_b_reg} <= RED;
    else if(h_cnt>=(Hde_start+160) && h_cnt<(Hde_start+320) && vsync_de)
```

```
        {vga_r_reg,vga_g_reg,vga_b_reg} <= GREEN;
    else if(h_cnt>=(Hde_start+320) && h_cnt<(Hde_start+480) && vsync_de)
        {vga_r_reg,vga_g_reg,vga_b_reg} <= BLUE;
    else if(h_cnt>=(Hde_start+480) && h_cnt<(Hde_start+640) && vsync_de)
        {vga_r_reg,vga_g_reg,vga_b_reg} <= YELLOW;
    else
        {vga_r_reg,vga_g_reg,vga_b_reg} <= 0;
end


assign vga_hsync = hsync_r;
assign vga_vsync = vsync_r;
assign vga_r = vga_r_reg;
assign vga_g = vga_g_reg;
assign vga_b = vga_b_reg;

endmodule
```

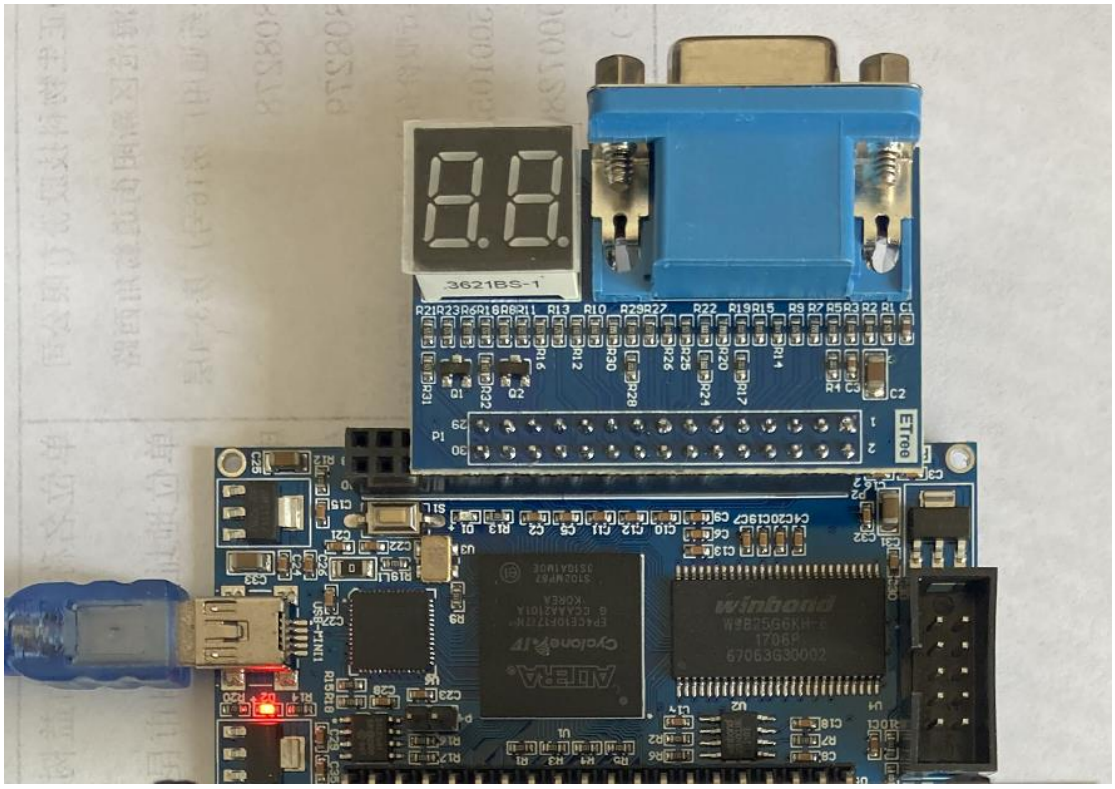
10.4. 管脚分配

选择菜单 Assignment -> Pin Planner，添加管脚约束。

Node Name	Direction	Location	I/O Bank
clk	Input	PIN_M16	5
led	Output	PIN_L12	5
rst_n	Input	PIN_M12	5
sm_dat[7]	Output	PIN_R8	3
sm_dat[6]	Output	PIN_T7	3
sm_dat[5]	Output	PIN_T9	4
sm_dat[4]	Output	PIN_T8	3
sm_dat[3]	Output	PIN_T10	4
sm_dat[2]	Output	PIN_R9	4
sm_dat[1]	Output	PIN_T11	4
sm_dat[0]	Output	PIN_R10	4
sm_sel[1]	Output	PIN_P14	4
sm_sel[0]	Output	PIN_R14	4
vga_blue[4]	Output	PIN_P9	4
vga_blue[3]	Output	PIN_N9	4
vga_blue[2]	Output	PIN_M10	4
vga_blue[1]	Output	PIN_P11	4
vga_blue[0]	Output	PIN_N11	4
vga_green[5]	Output	PIN_N8	3
vga_green[4]	Output	PIN_P8	3
vga_green[3]	Output	PIN_K9	4
vga_green[2]	Output	PIN_K10	4
vga_green[1]	Output	PIN_L9	4
vga_green[0]	Output	PIN_M9	4
vga_hsync	Output	PIN_R7	3
vga_red[4]	Output	PIN_M8	3
vga_red[3]	Output	PIN_L8	3
vga_red[2]	Output	PIN_L7	3
vga_red[1]	Output	PIN_T6	3
vga_red[0]	Output	PIN_R6	3
vga_vsync	Output	PIN_L10	4

10.5. 实验操作

插上 VGA 模块板卡(注意 1 脚对应 P2 的 1 脚)，连接 FPGA 板卡和 VGA 显示器，如下图所示：



打开电源后，下载 FPGA 配置文件，可以看到显示器上实时显示彩条 R/G/B/Y，如下图所示。

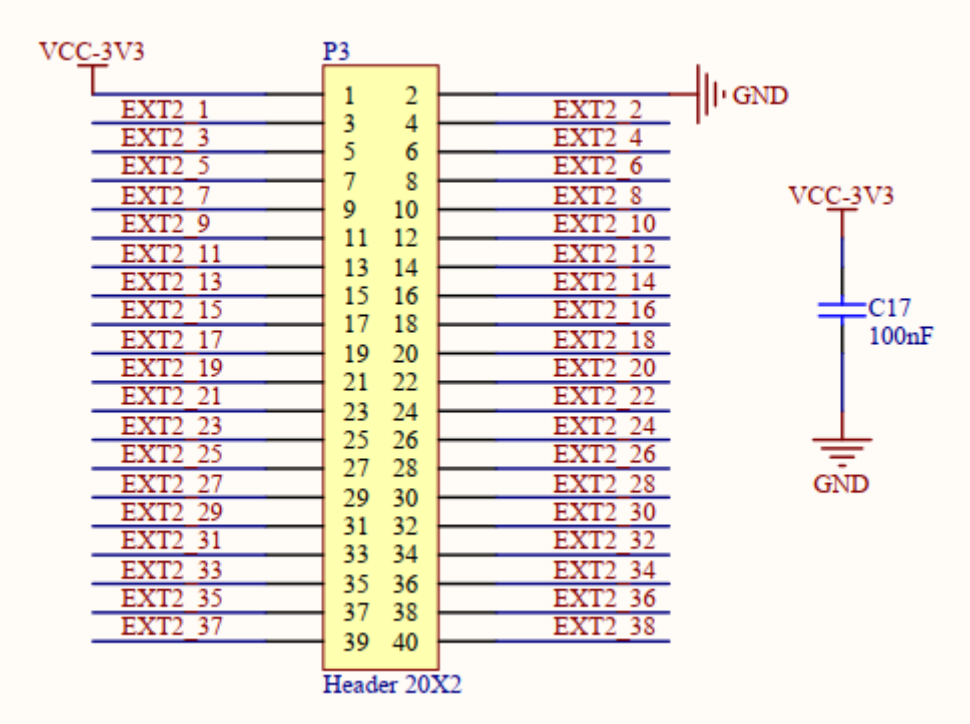


第十一章 数字频率计

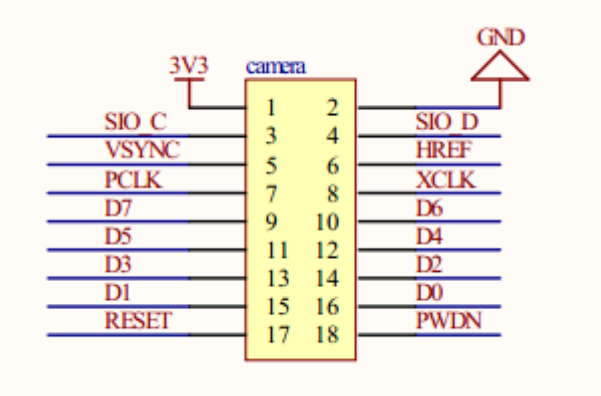
本章设计一款数字频率计，计数范围为 0～255Hz。数字频率计用来测量摄像头 OV7670 默认的输出帧率，并用嵌入式逻辑分析仪 SignalTap II 观察结果。

11.1. 硬件介绍

P3 连接摄像头模块，P3 接口电路如下所示。



OV7670 摄像头接口电路如下图所示，被测量的引脚是 CMOS_FV(VSYNC)，对应 P3 的 EXT2_3 引脚。



11.2. HDL 设计

数字频率计以板卡上的 24MHz 晶振作为时钟参考，定义一个秒计数器和频率值计数器，然后通过捕捉 1 秒时间内 CMOS_FV 帧同步信号的上升沿的个数来计算 CMOS 图像传感器的

输出帧率。

数字频率计代码设计如下。

```
//file name        : frame_counter.v
//data             : 2017.10.1
//author              : ETree
//function         : frame counter
//log                 :

module frame_counter(
    input clk, //24MHz
    input rst_n,

    input cmos_vsync,

    //ledsm
    output reg [7:0] frame_out,
);

//--------frame count--------
reg [25:0] cnt_1sec;
reg [7:0] cnt_freq;
reg [1:0] cmos_vsync_r;
reg [7:0] frame_out;

always @(posedge clk)
    cmos_vsync_r <= {cmos_vsync_r[0],cmos_vsync};

wire cmos_vsync_pos = (cmos_vsync_r==2'b01);

always @(posedge clk)
    if(~rst_n)
        cnt_1sec <= 0;
    else if(cnt_1sec==23_999_999)
        cnt_1sec <= 0;
    else
        cnt_1sec <= cnt_1sec + 1'b1;


always @(posedge clk)
    if(~rst_n)
    begin
        cnt_freq <= 0;
    end
    else if(cnt_1sec<23_999_999)
```

```
    begin
        if(cmos_vsync_pos)
            cnt_freq <= cnt_freq + 1'b1;
        else
            cnt_freq <= cnt_freq;
    end
    else
        cnt_freq <= 0;

always @(posedge clk)
    if(~rst_n)
        frame_out <= 0;
    else if(cnt_1sec==23_999_998)
        frame_out <= cnt_freq;
    else
        frame_out <= frame_out;

endmodule
```

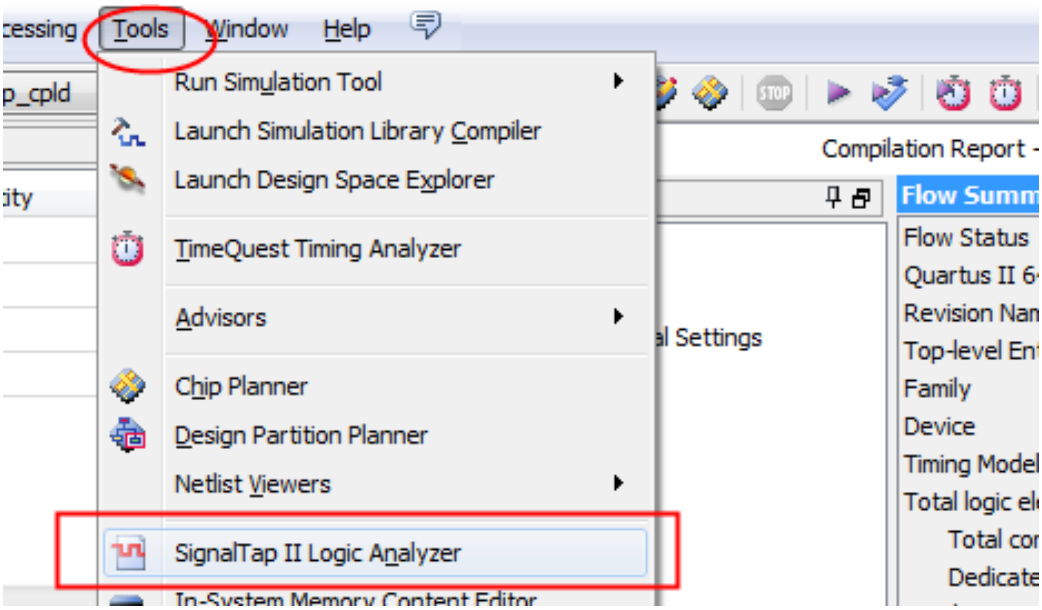
11.3. 管脚约束

选择菜单 Assignment -> Pin Planner，添加管脚约束。

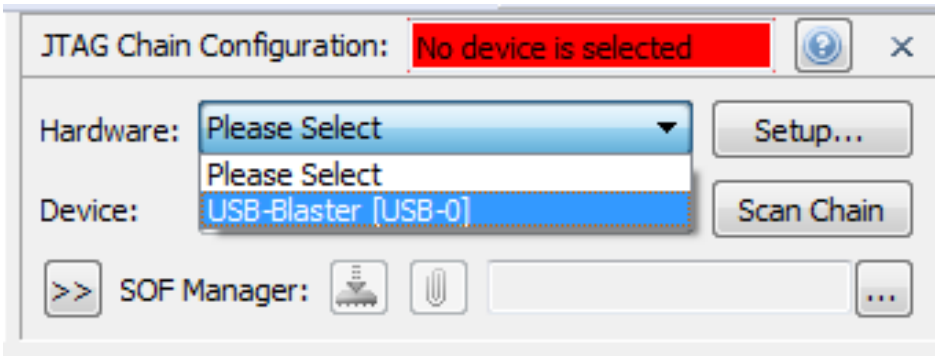
cam_data[7]	Input	PIN_A13	7
cam_data[6]	Input	PIN_B12	7
cam_data[5]	Input	PIN_A12	7
cam_data[4]	Input	PIN_B11	7
cam_data[3]	Input	PIN_A11	7
cam_data[2]	Input	PIN_B10	7
cam_data[1]	Input	PIN_A10	7
cam_data[0]	Input	PIN_B9	7
cam_hsync	Input	PIN_B13	7
cam_pclk	Input	PIN_F11	7
cam_pdown	Output	PIN_C9	7
cam_reset	Output	PIN_A9	7
cam_scl	Input	PIN_A15	7
cam_sda	Input	PIN_B14	7
cam_vsync	Input	PIN_A14	7
cam_xclk	Output	PIN_D12	7
clk	Input	PIN_M16	5
led	Output	PIN_L12	5
rst_n	Input	PIN_M12	5

11.4. SignalTap 使用

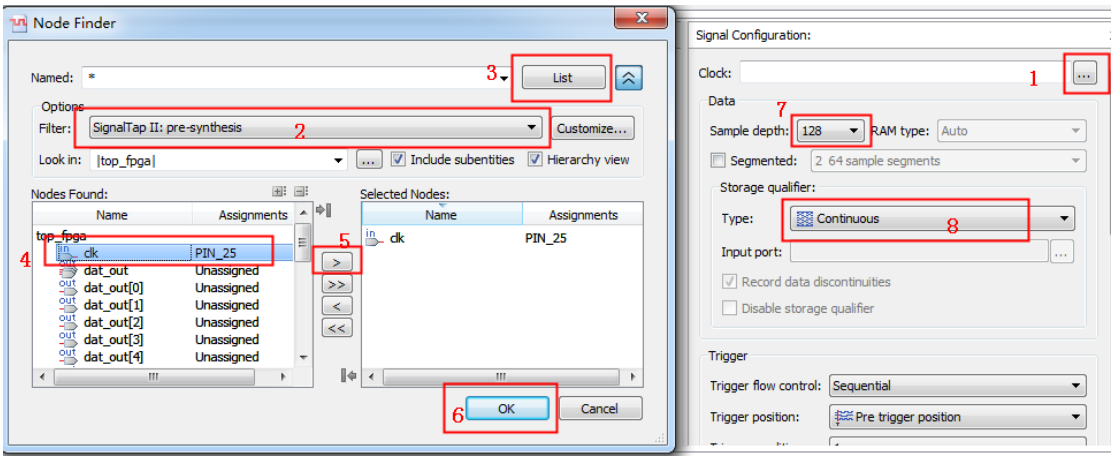
- 打开 SignalTap II Logic Analyzer



➢ 选择 JTAG Chain Configuration



➢ 设置采样时钟和采样深度



第一步：选择采样时钟；

第二步：选择过滤类型，这里选 pre-synthesis；

第三步：将所有符合的信号列出来；

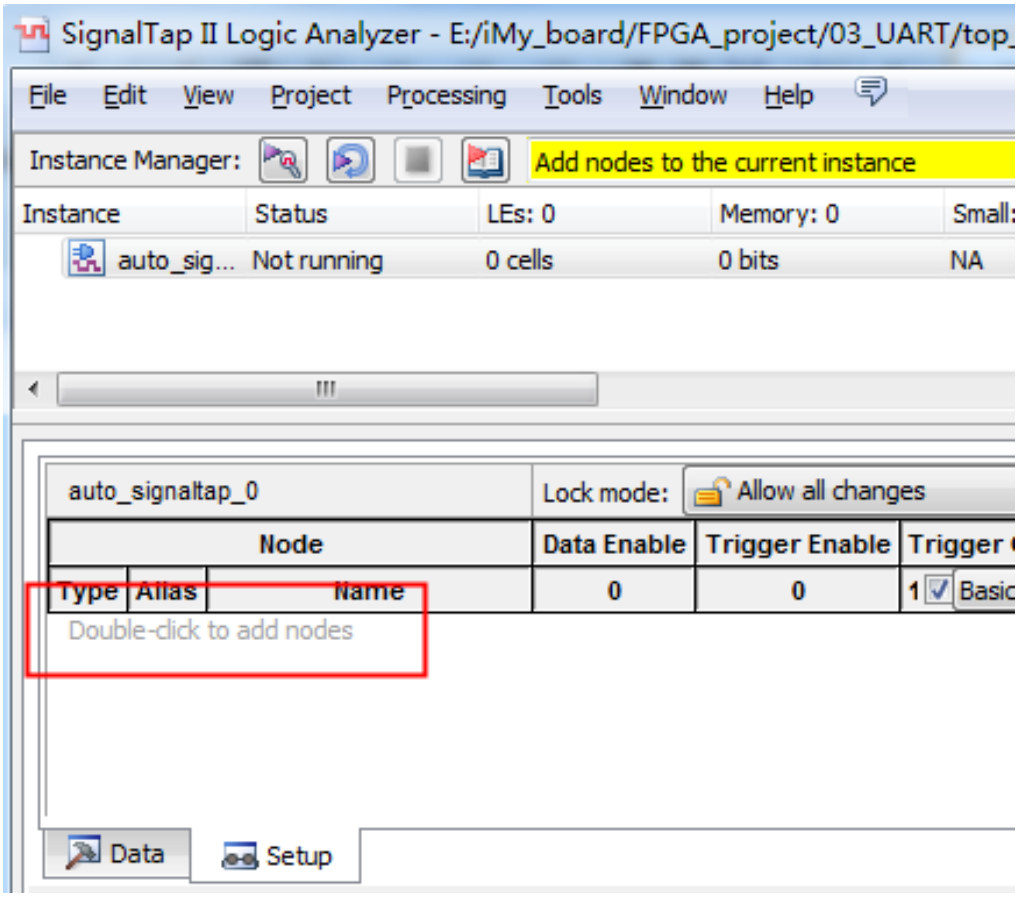
第四步：选择信号，这里选择 clk；

第五步：添加信号；

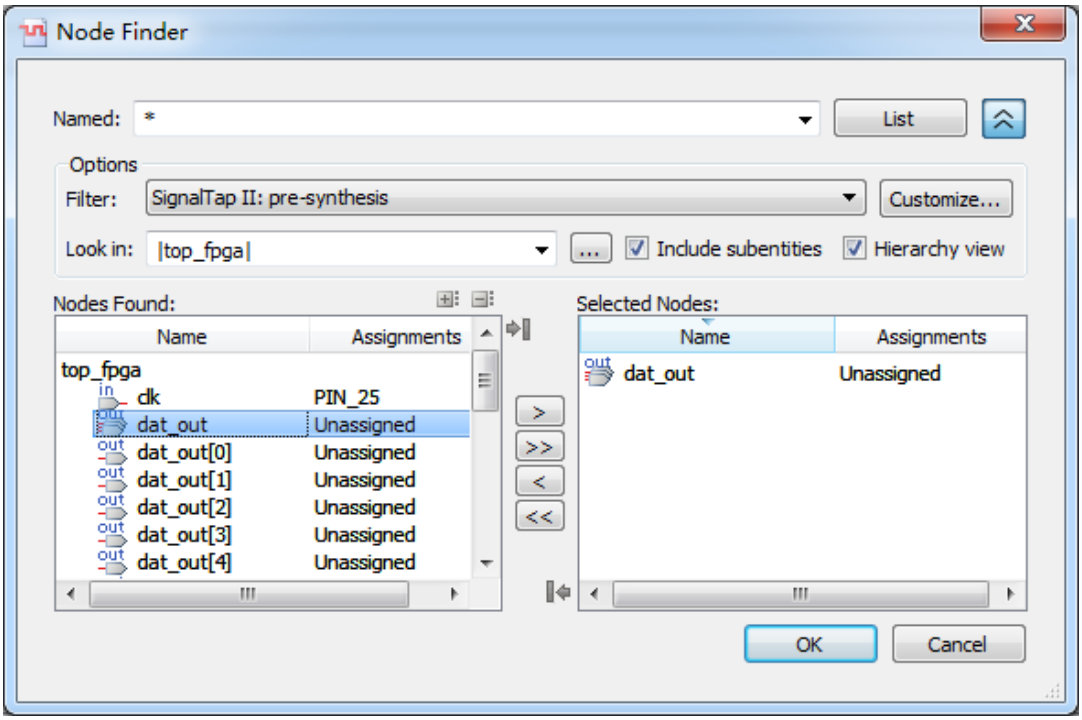
第六步：选择采样时钟完成；
第七步：选择采样深度，这里选择 128；
第八步：选择触发方式，这里选择连续触发方式(Continuous)。

- 添加被观察信号

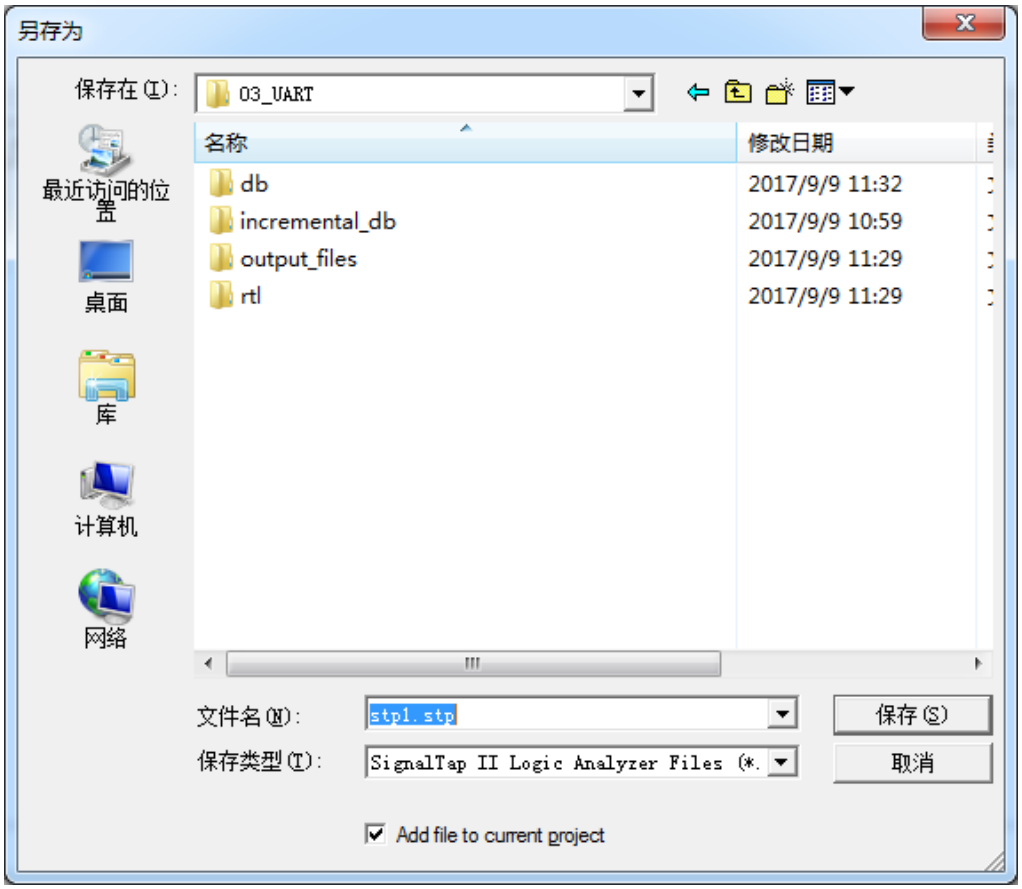
在 Setup 空白区域双击，弹出信号选择窗口；



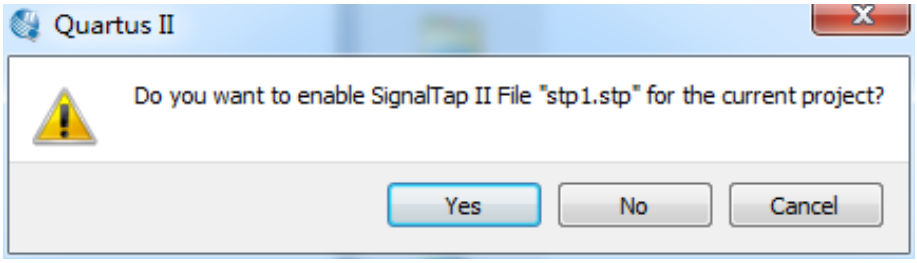
选择需要观察的信号，这里选择 dat_out。



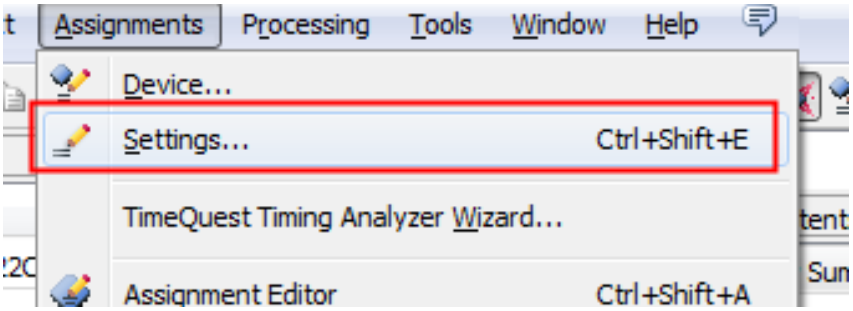
➢ 保存 SinalTap II 文件

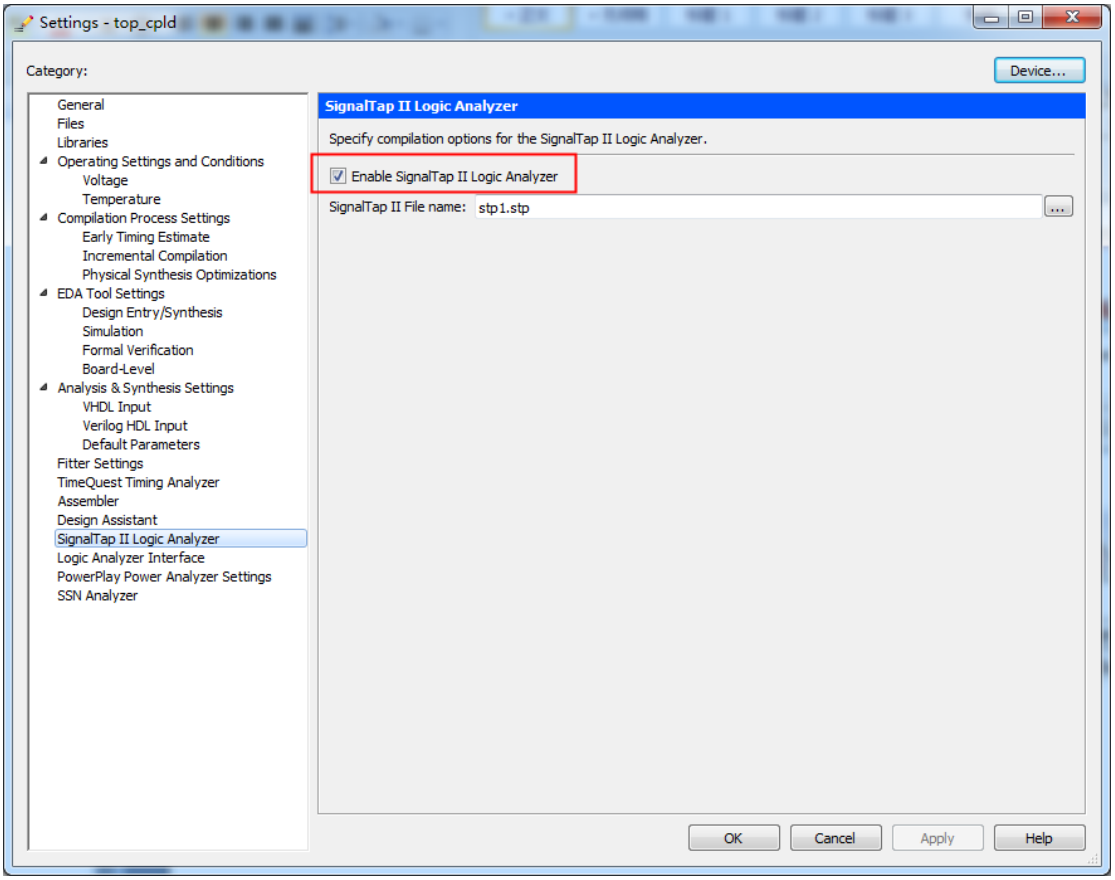


保存好 SignalTap II 文件后，Quartus II 会自动弹出“是否打开 SignalTap II 功能”对话框，这里点击‘Yes’。

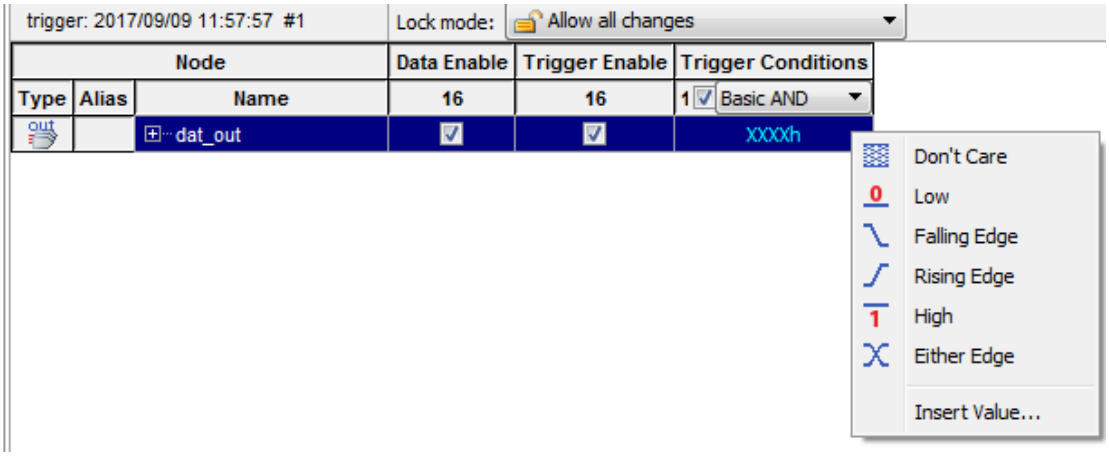


如果要关闭 SignalTap II 功能，可在 Settings 中关闭。





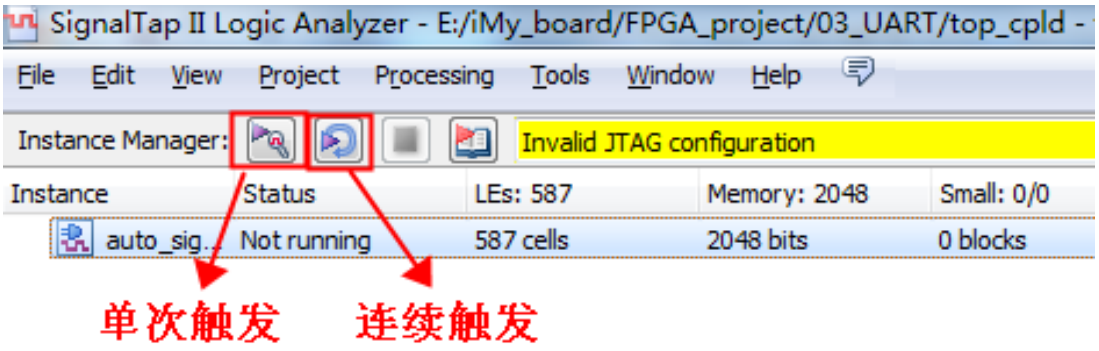
➢ 选择信号的触发条件



➢ 编译工程

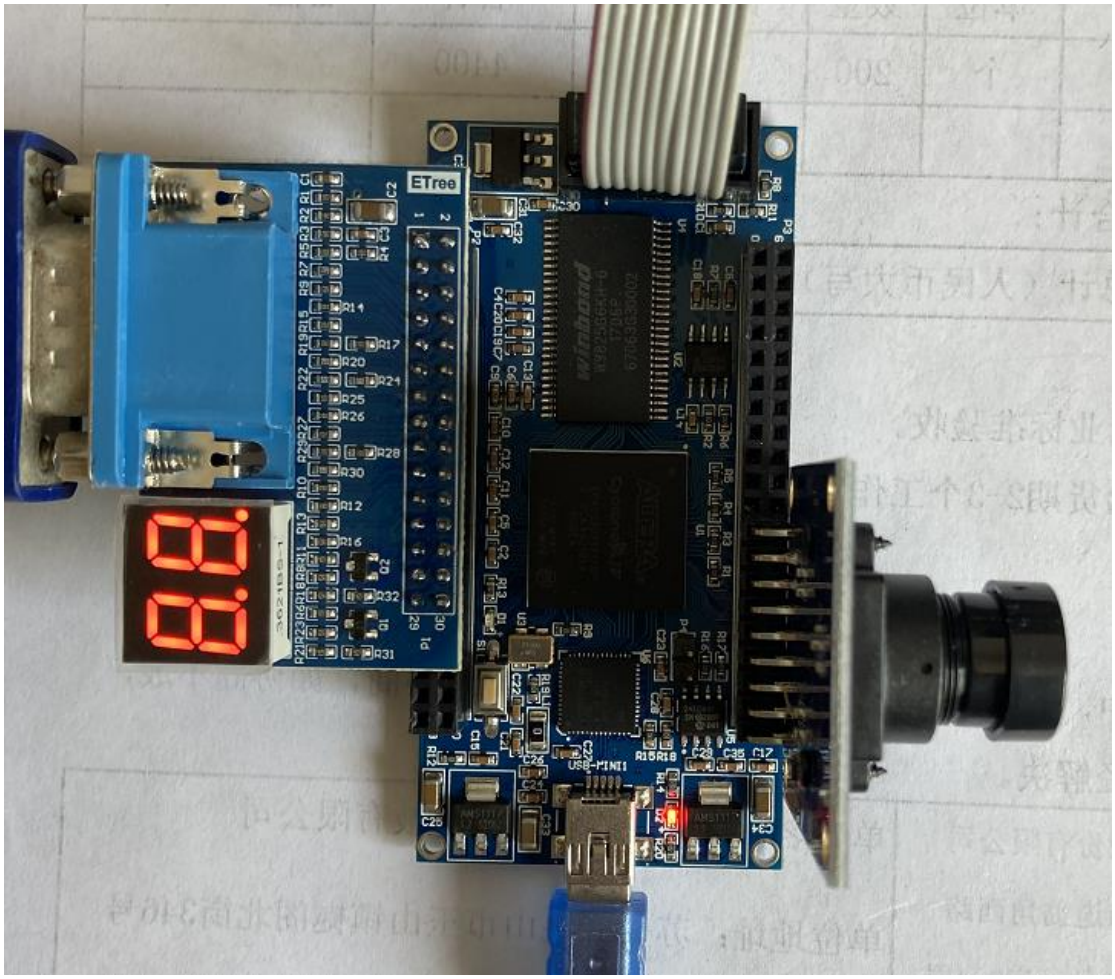
添加完 SignalTap II 文件或修改了 SignalTap II 文件的内容，都需要重新编译工程。

➢ 运行 SignalTap II

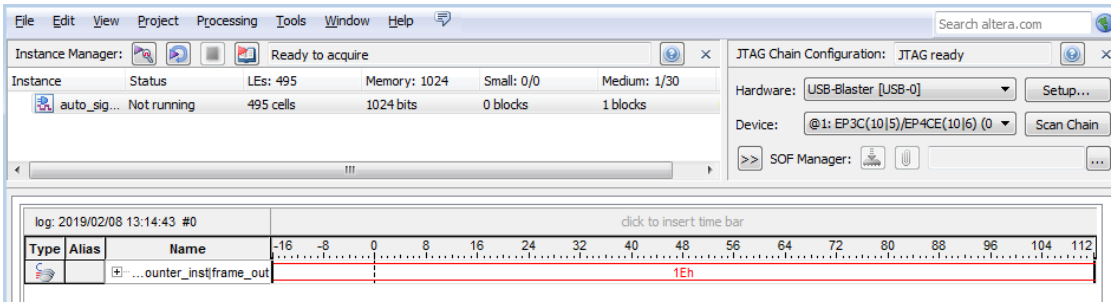


11.5. 实验操作

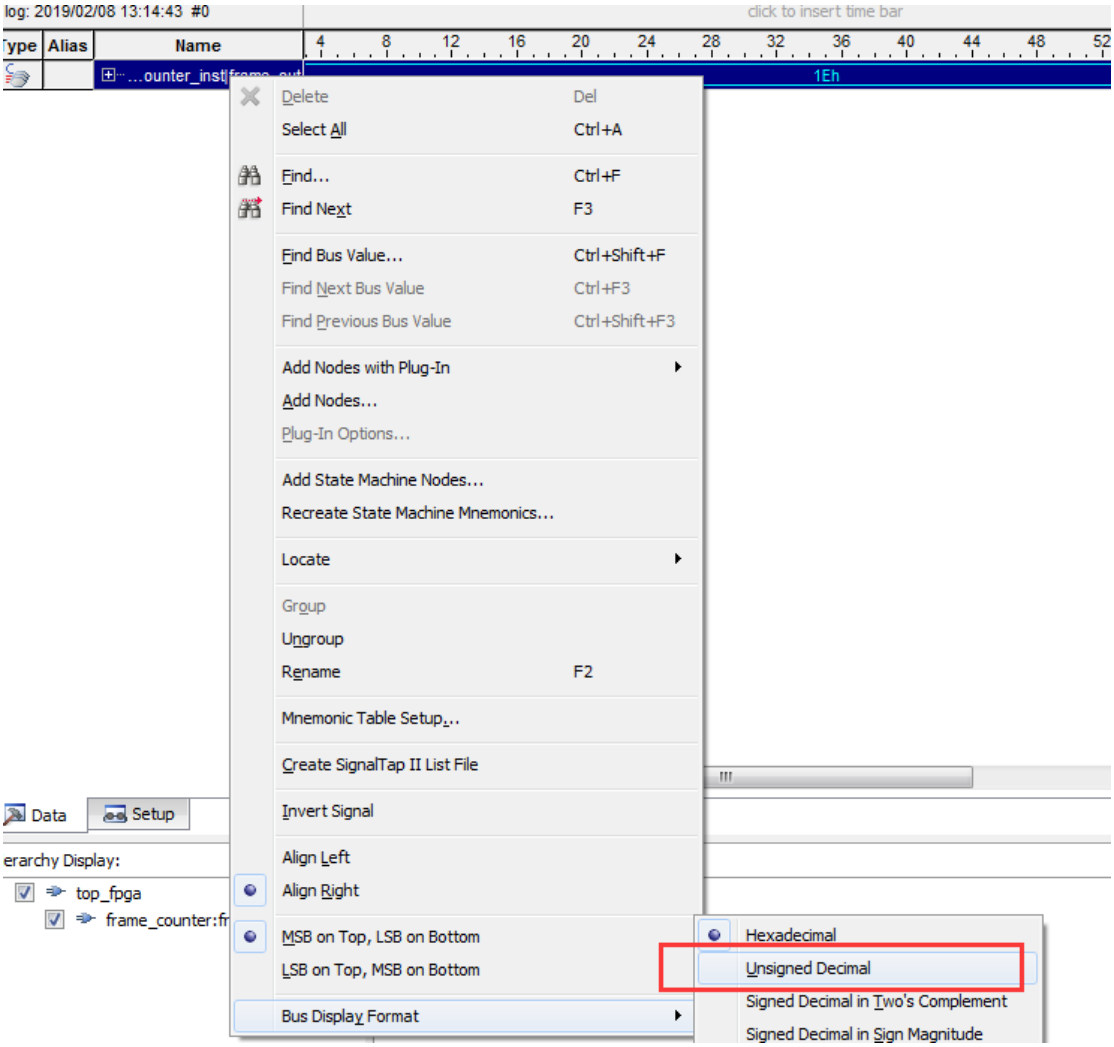
插上 JTAG 下载器、MINI USB 电源线、OV7670 摄像头模块(注意 1 脚对应 P3 的 1 脚)和 VGA 模块板卡(注意 1 脚对应 P2 的 1 脚)，如下图所示。



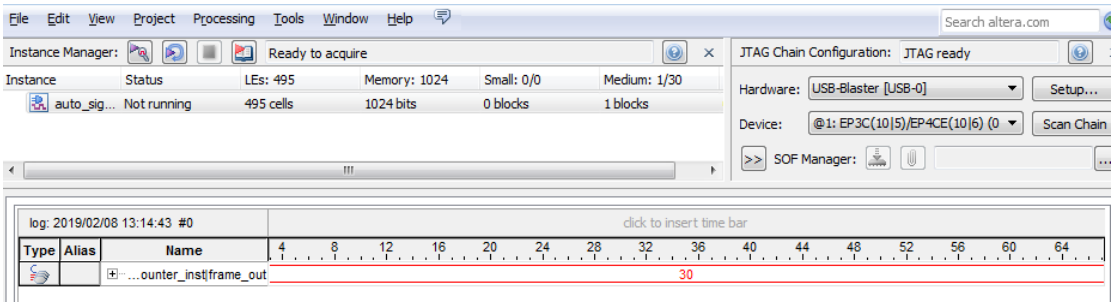
下载编译好的 top_fpga.sof 文件，打开 SignalTap II 界面，点击‘单次触发’按钮。



上图得到计算的频率为‘1Eh’，为十六进制显示，我们为了方便观察结果，选中 frame_out 信号右击选择无符号十进制显示，如下图所示。



再一次点击‘单次触发’按钮，得到计算频率值为 30(根据 OV7670 芯片手册知实际输出帧率也为 30 帧/秒)，如下图所示。

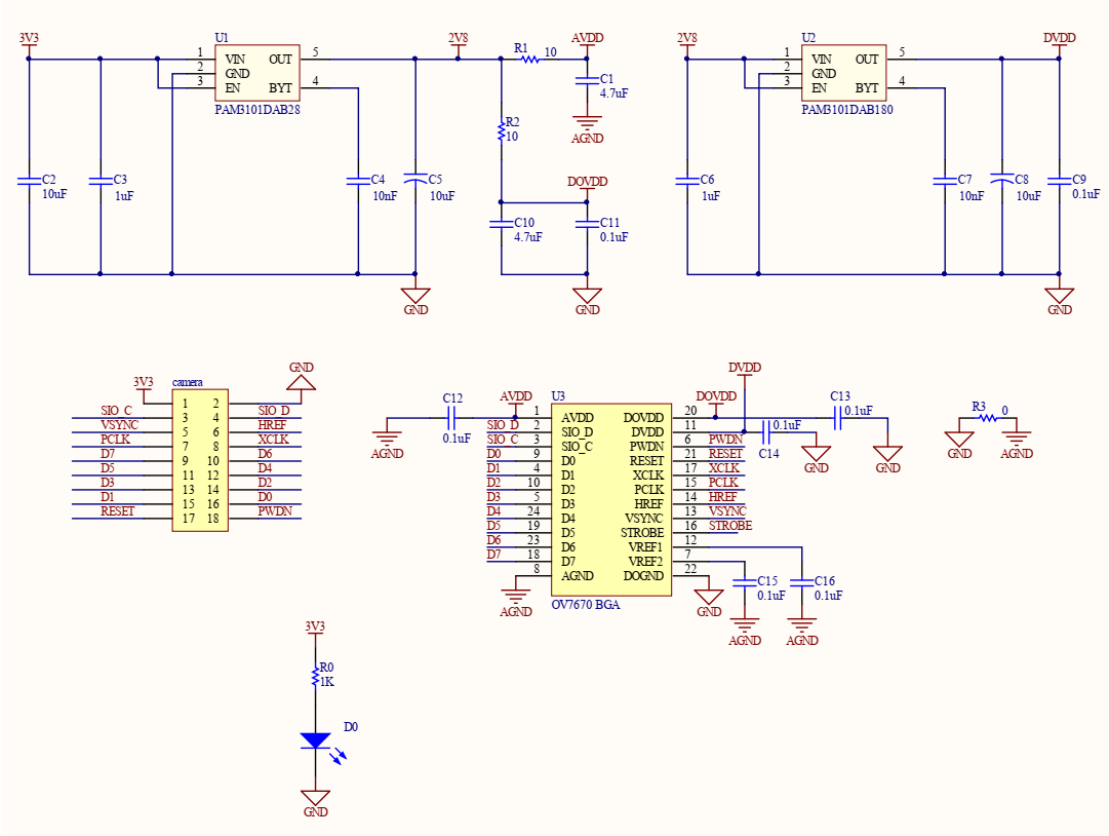


第十二章 OV7670 VGA 显示器显示

本章基于 OV7670 摄像头和 VGA 接口实现摄像头数据的 VGA 实时显示功能。

12.1. 硬件介绍

本设计采用 OV7670 模块和 VGA_SM 模块，OV7670 硬件原理图如下：



OV7670 的输出接口为 DVP 并行信号，图像分辨率为 30 万像素，信价比很高。

12.2. OV7670 摄像头

OV7670 是豪威(OmniVision)公司生产的一颗 1/6 寸的 CMOS VGA 图像传感器。该传感器体积小、工作电压低，提供单片 VGA 摄像头和影像处理器的所有功能。通过 SCCB 总线控制，可以输出整帧、子采样、取窗口等方式的各种分辨率 8 位影像数据。该产品 VGA 图像最高达到 30 帧/秒。用户可以完全控制图像质量、数据格式和传输方式。所有图像处理功能过程包括伽玛曲线、白平衡、度、色度等都可以通过 SCCB 接口编程。 OmmiVision 图像传感器应用独有的传感器技术，通过减少或消除光学或电子缺陷如固定图案噪声、托尾、浮散等，提高图像质量，得到清晰的稳定的彩色图像。

OV7670 的特点有：

- 高灵敏度、低电压适合嵌入式应用
- 标准的 SCCB 接口，兼容 IIC 接口
- 支持 RAW8、 RGB(GBR4:2:2， RGB565/RGB555/RGB444)， YUV(4:2:2)和 YCbCr(422)

输出格式

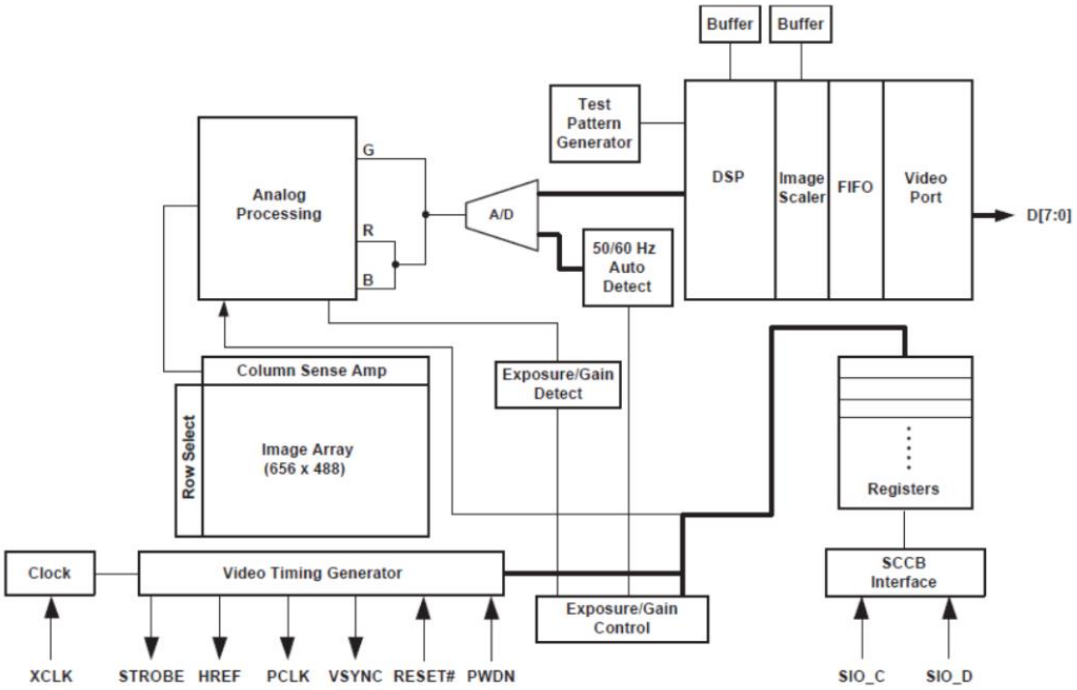
◆ 支持 VGA、 CIF，和从 CIF 到 40*30 的各种尺寸输出

◆ 支持自动曝光控制、自动增益控制、自动白平衡、自动消除灯光条纹、自动黑电平校准等自动控制功能。同时支持色饱和度、色相、伽马、锐度等设置

◆ 支持闪光灯

◆ 支持图像缩放

OV7670 的功能框图如下图所示。



OV7670 的寄存器通过 SCCB 时序访问并设置， SCCB 时序和 IIC 时序十分类似，这里不做详细介绍，用户可以查询相关文档。

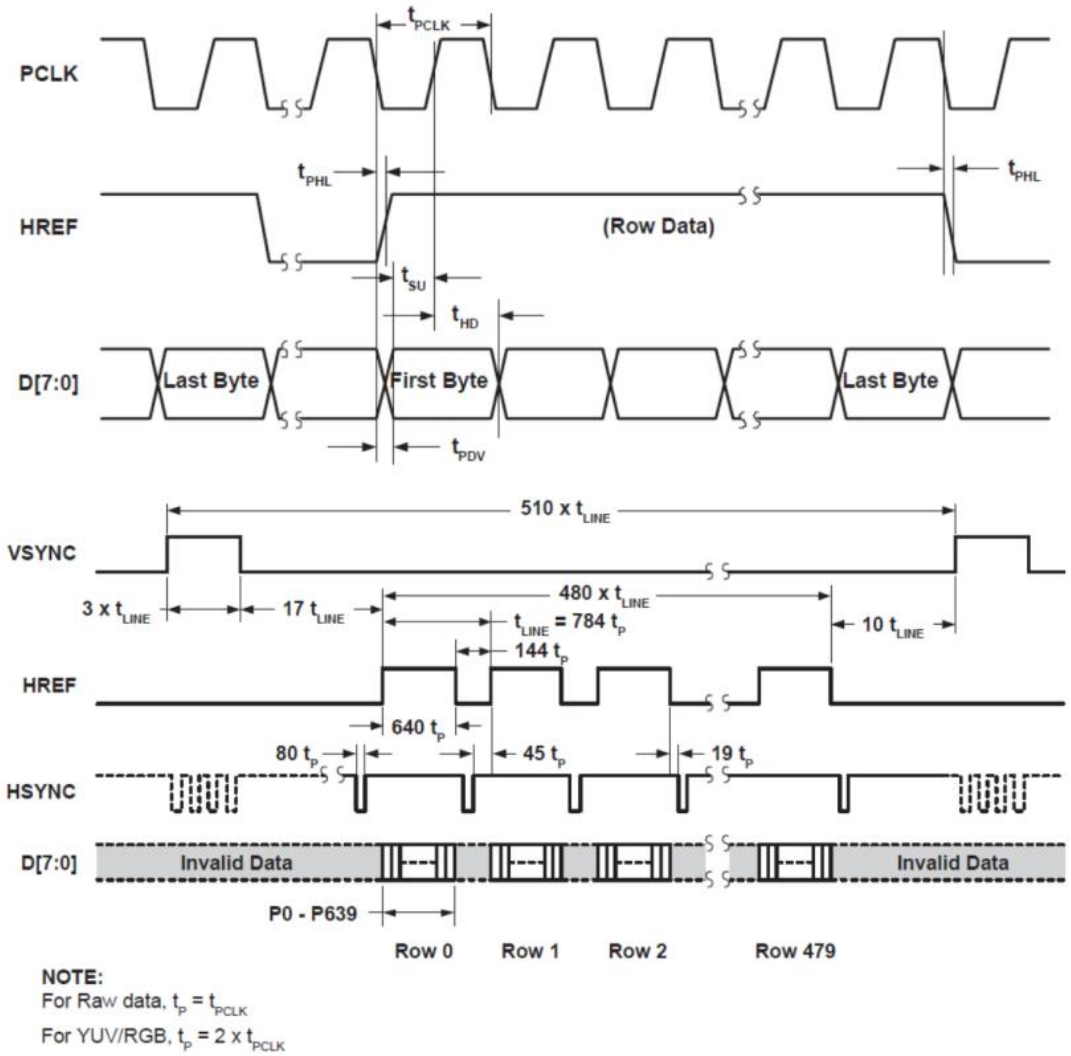
下面我们来介绍 OV7670 图像输出格式。

◆ PCLK：像素时钟

◆ VSYNC：帧同步信号

◆ HREF/HSYNC：行同步信号

◆ D[7:0]：图像数据输出

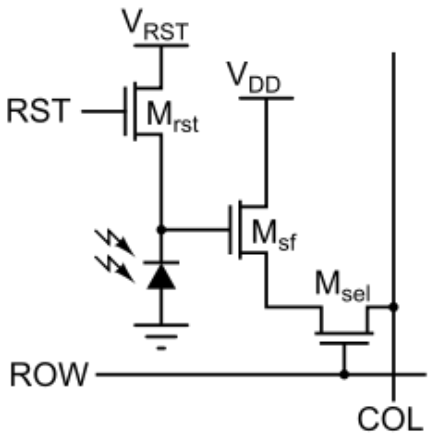


从上图我们可以看出，当 HREF 为高时，每个 pclk 时钟输出一个字节数据。当 VSYNC 为高时表示新的一帧开始。

关于 OV7670 详细的寄存器操作可以参考官方提供的芯片手册。

12.3. CMOS 工作原理

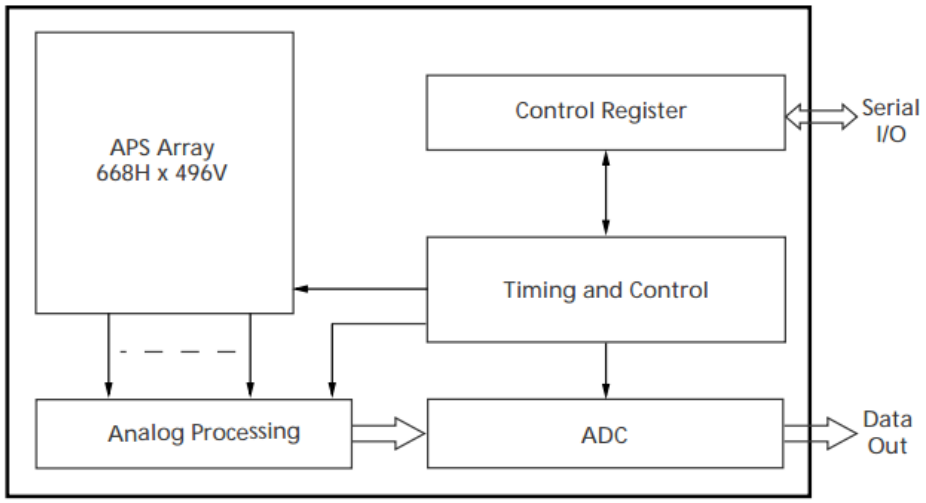
APS(Active Pixel Sensor)是最新的 CMOS 图像传感器的像素结构设计，下图为 3T-APS 的结构电路：



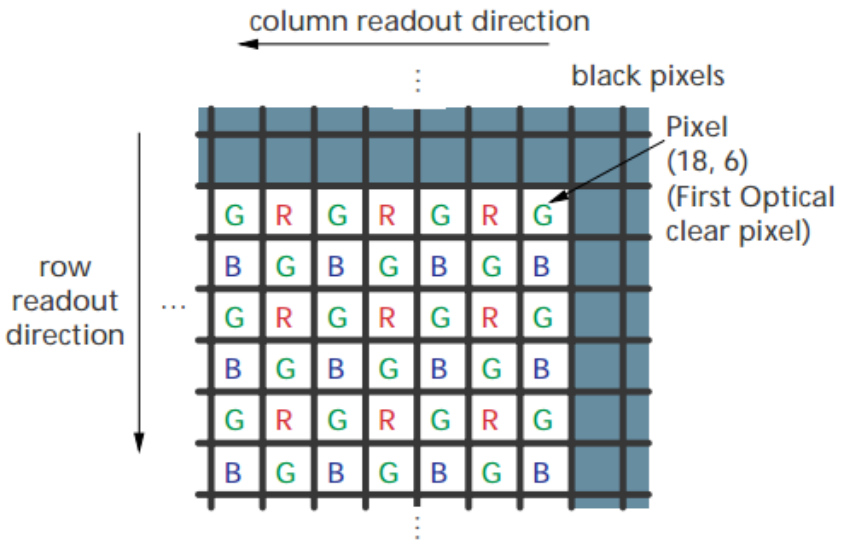
APS 的工作过程：

- 复位晶体管 Mrst 开通
- PD 复位到 Vrst-Vth(Mrs 的阈值电压)
- 关闭 Mrst，PD 进入电气浮动状态
- 光线入射，PD 结电容聚集光生载流子
- 在一个积累时间后，选通晶体管 Msf
- 打开晶体管 Msel，像素中的输出信号在垂直输出线上被读出
- 关闭 Msel，Mrst 再次接通，重复上述过程。

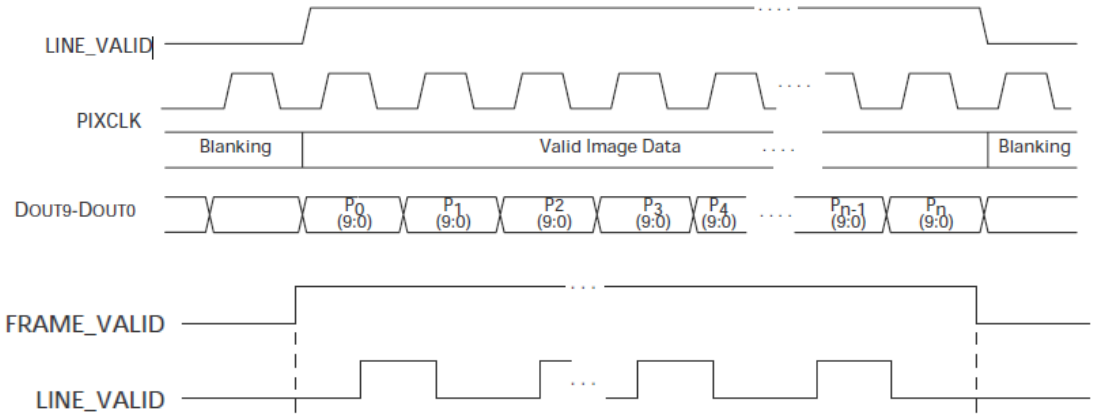
MT9V011 为 30 万像素，最高 30 帧，感光阵列面积为 1/3”，内部结构如下图所示：



APS Array 和 Analog Processing 为模拟电路部分，其中感光阵列排序为 GRBG；



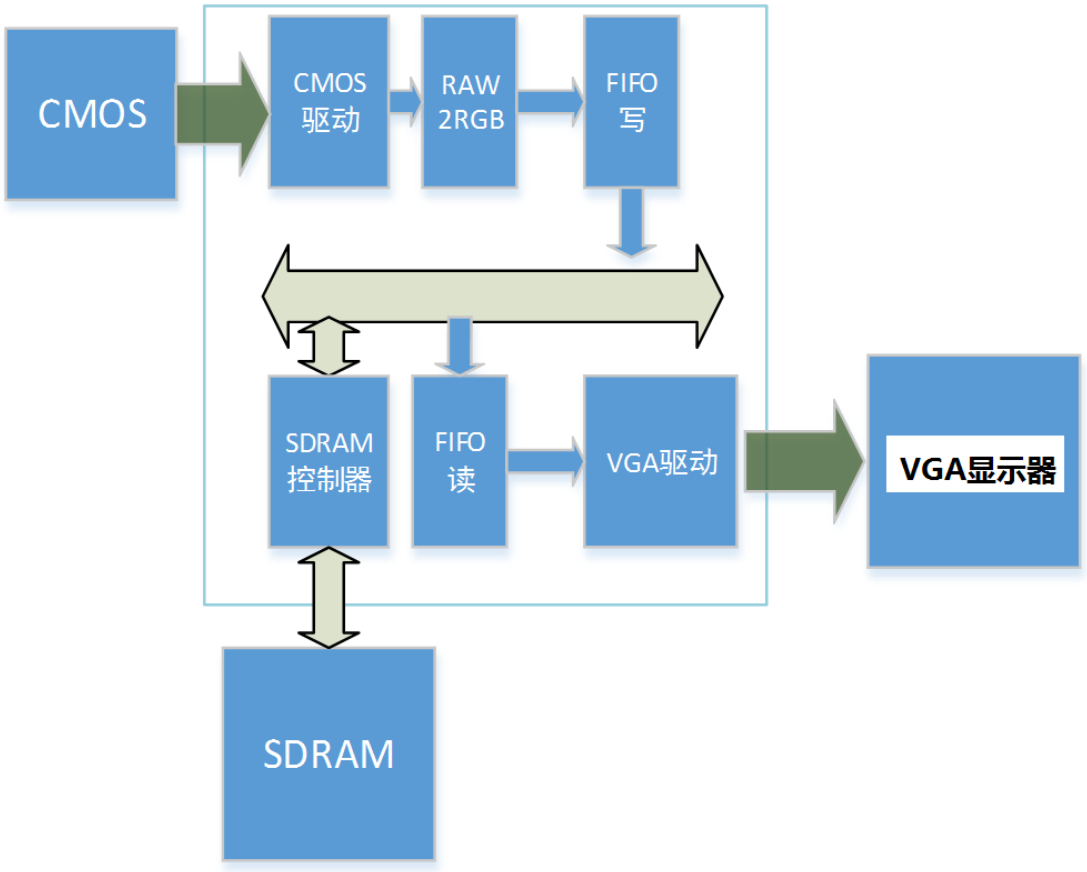
Control Register 和 Timing and Control 为数字逻辑部分，其中 Serial IO 为 I2C 接口，Data Out 为并行 DVP 视频信号接口。具体 DVP 波形如下图所示，LINE_VALID(h_ref)为行有效信号，PIXCLK 为像素时钟信号，FRAME_VALID 为帧同步信号，Dout 为像素数据信号。



12.4. HDL 设计

➢ 系统框架

FPGA 将采集的 OV7670 数据存放在 SDRAM 中，然后 VGA 驱动模块再将 SDRAM 中的视频数据发送给 VGA 显示器。



- CMOS 驱动：负责 CMOS 图像采集与裁剪；
- RAW2RGB：负责将原始的 RAW 数据转换成 RGB656 数据，这部功能由摄像头芯片内部完成，FPGA 直接接受的便是 RGB565 数据；
- FIFO 写负责将 RAW2RGB 转换的的 RGB 数据发送给 SDRAM 控制器；
- FIFO 读负责将 SDRAM 中的视频帧数据读出给 VGA 驱动模块；
- VGA 模块负责将视频信号发送给 VGA 显示器显示。

➢ VGA 驱动

```
module video_gen(
    input clk,//25.175MHz
    input rst_n,

    input  [15:0] vga_datain,

    output reg vsyn,
    output reg hsyn,
    output reg [15:0] video_data_out,       //VGA565
    output reg video_valid
);

parameter IMG_HDISP = 16'd640;
parameter IMG_VDISP = 16'd480;
```

```
reg [11:0] hcnt;
reg [11:0] vcnt;

always @(posedge clk)
begin
    if(~rst_n)
        hcnt <= 0;
    else if(hcnt>=12'd799)
        hcnt <= 0;
    else
        hcnt <= hcnt + 1'b1;
end


always @(posedge clk)
begin
    if(~rst_n)
        vcnt <= 0;
    else if(hcnt>=12'd799)
        if(vcnt>=12'd524)
            vcnt <= 0;
        else
            vcnt <= vcnt + 1'b1;
    else
        vcnt <= vcnt;
end

always @(posedge clk)
begin
    hsyn <= (hcnt>=12'd13 && hcnt<12'd109); //96
end

always @(posedge clk)
begin
    vsyn <= (vcnt>=12'd9 && vcnt<12'd11);
end

always @(posedge clk)
begin
    video_valid <= (hcnt>=12'd160 && hcnt<=12'd799) && (vcnt>=12'd45 &&
vcnt<=12'd524);
end
```

```
always @(*)
    if(video_valid)
        video_data_out <= vga_datain;
    else
        video_data_out <= 0;

endmodule
```

- SDRAM 驱动

本章的 SDRAM 驱动修改于“友晶”开源的 SDRAM 控制器，实现 SDRAM 读写的控制。有兴趣的读者可以研究一下，这里不作说明。

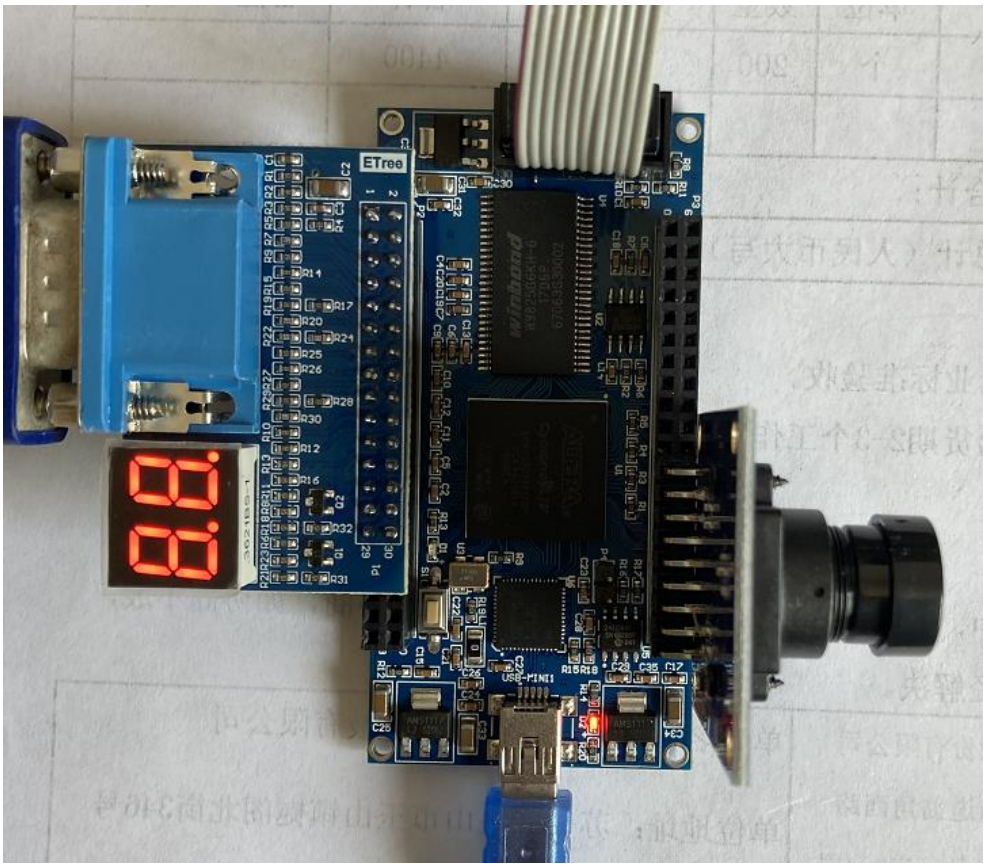
12.5. 管脚约束

选择菜单 Assignment -> Pin Planner，添加管脚约束。

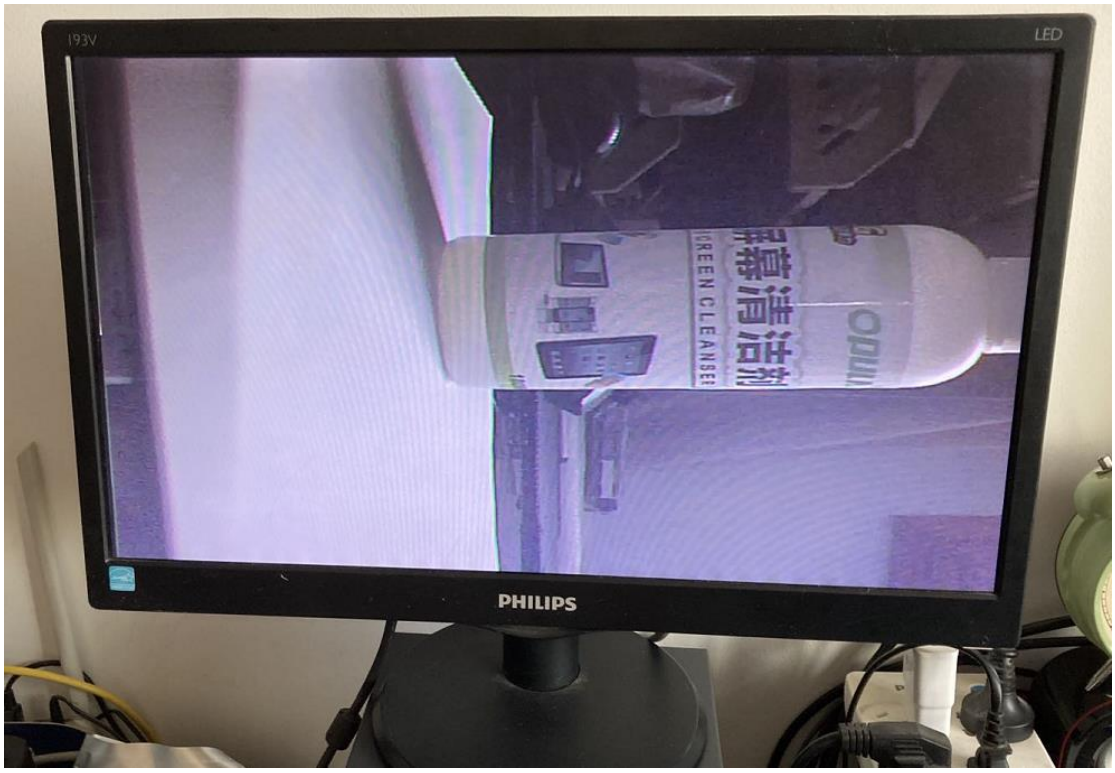
Node Name	Direction	Location	I/O Bank
cam_data[7]	Input	PIN_A13	7
cam_data[6]	Input	PIN_B12	7
cam_data[5]	Input	PIN_A12	7
cam_data[4]	Input	PIN_B11	7
cam_data[3]	Input	PIN_A11	7
cam_data[2]	Input	PIN_B10	7
cam_data[1]	Input	PIN_A10	7
cam_data[0]	Input	PIN_B9	7
cam_hsync	Input	PIN_B13	7
cam_pclk	Input	PIN_F11	7
cam_pdown	Output	PIN_C9	7
cam_reset	Output	PIN_A9	7
cam_scl	Output	PIN_A15	7
cam_sda	Bidir	PIN_B14	7
cam_vsync	Input	PIN_A14	7
cam_xclk	Output	PIN_D12	7
clk	Input	PIN_M16	5
led	Output	PIN_L12	5
rst_n	Input	PIN_M12	5
sdram_addr[12]	Output	PIN_N2	2
sdram_addr[11]	Output	PIN_N1	2
sdram_addr[10]	Output	PIN_P3	3
sdram_addr[9]	Output	PIN_L2	2
sdram_addr[8]	Output	PIN_L1	2
sdram_addr[7]	Output	PIN_K2	2
sdram_addr[6]	Output	PIN_K1	2
sdram_addr[5]	Output	PIN_J2	2
sdram_addr[4]	Output	PIN_J1	2
sdram_addr[3]	Output	PIN_B3	8
sdram_addr[2]	Output	PIN_A3	8
sdram_addr[1]	Output	PIN_B4	8
sdram_addr[0]	Output	PIN_A4	8
sdram_ba[1]	Output	PIN_C3	8
sdram_ba[0]	Output	PIN_N6	3
sdram_cas_n	Output	PIN_N5	3
sdram_cke	Output	PIN_P1	2
sdram_clk	Output	PIN_P2	2
sdram_cs_n	Output	PIN_D5	8
sdram_dq[15]	Bidir	PIN_R5	3
sdram_dq[14]	Bidir	PIN_M6	3
sdram_dq[13]	Bidir	PIN_P6	3

12.6. 实验操作

插上 JTAG 下载器、MINI USB 电源线、OV7670 摄像头模块(注意 1 脚对应 P3 的 1 脚)和 VGA 模块板卡(注意 1 脚对应 P2 的 1 脚)，如下图所示。



下载编译好的 top_fpga.sof 文件，效果如下图所示：

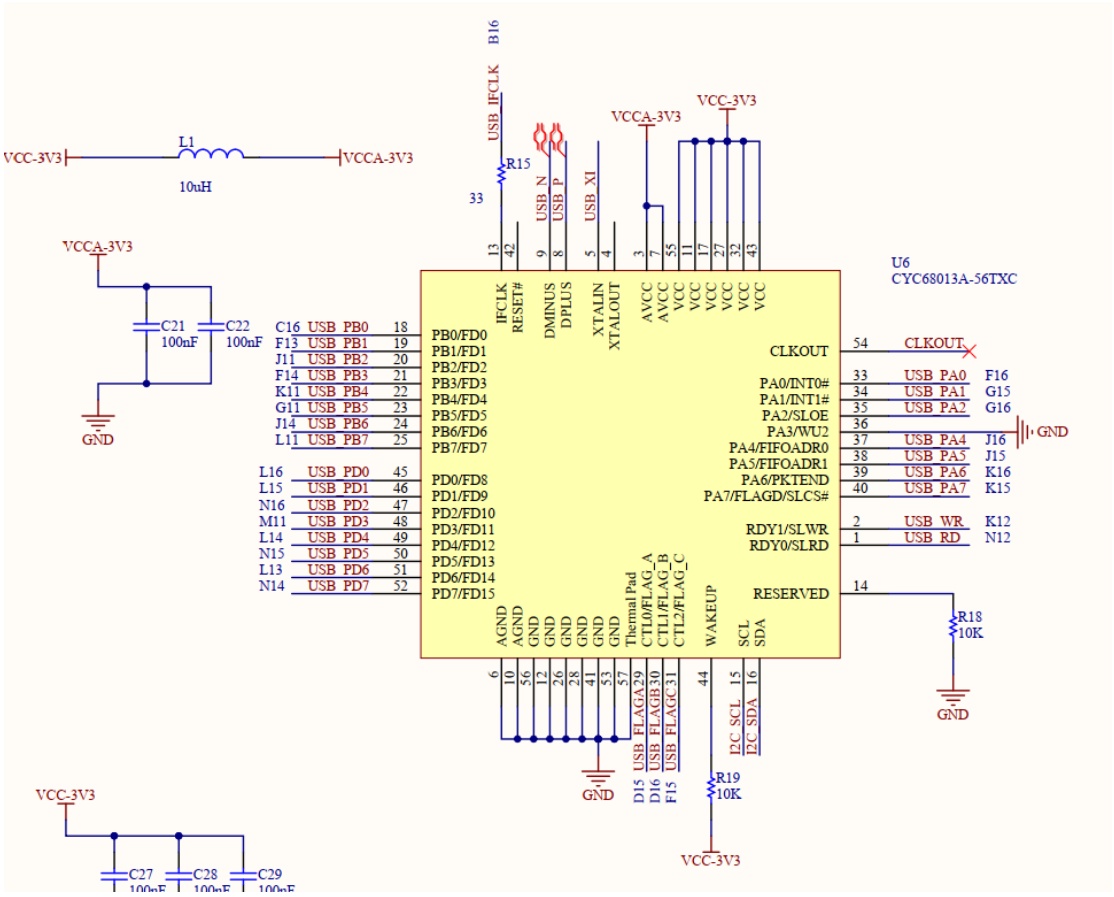


第十三章 USB Slave FIFO 写

本章基于 FPGA 和 USB2.0 芯片(68013a)实现 FPGA 发送数据包给 PC。

13.1. 硬件介绍

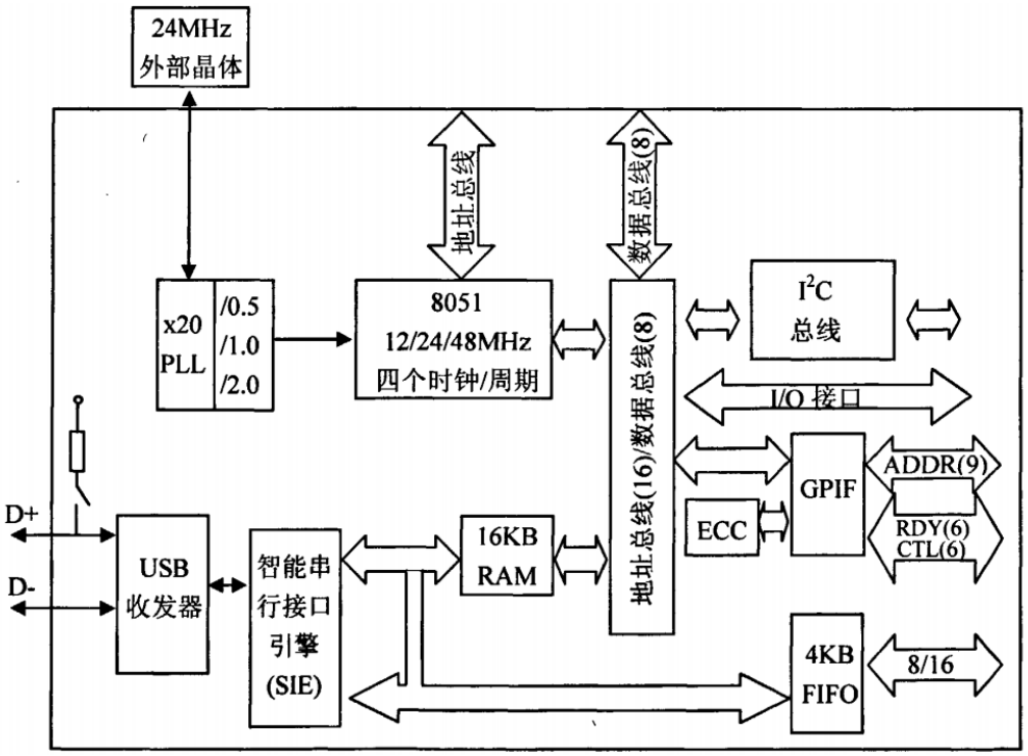
开发板板载 USB2.0 芯片，具体原理图如下图所示：



13.2. CY7C68013A 芯片简介

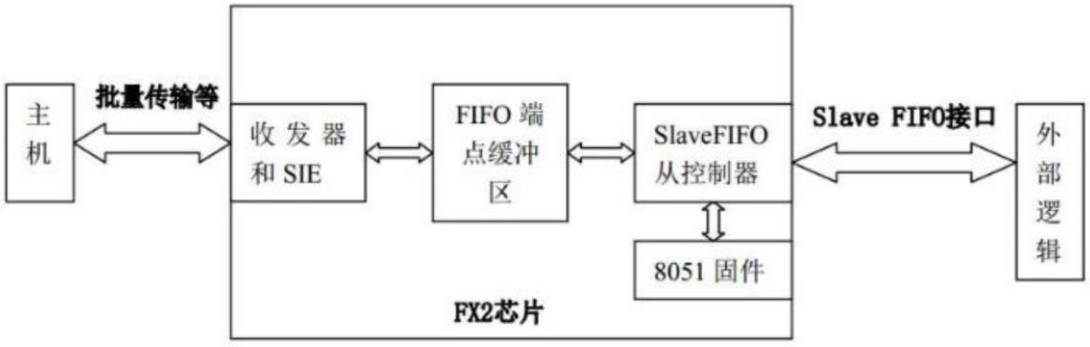
CY7C68013A 是 Cypress 公司推出的高速 USB2.0 芯片，集成 USB2.0 协议的微处理器，支持高达 480Mb/s 的传输速率，可使用控制传输、 中断传输、 块传输和同步传输等传输方式。

CY7C68013A 结构独特，其中智能串行接口引擎(SIE)负责完成串行数据的编解码、差错控制、位填充等 USB 协议相关的功能，内嵌的 MCU(增强型 8051)只负责相关寄存器配置和控制。下图为 CY7C68013A 内部结构图。



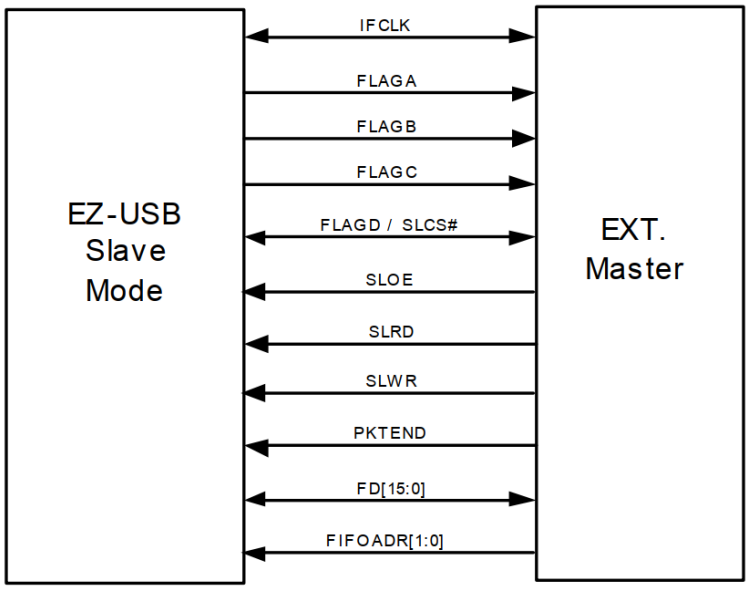
13.3. USB Slave FIFO 传输

当有一个与芯片(FPGA)相连主控只需要利用 FX2(CY7C68013A)做为一个 USB2.0 接口而实现与主机的高速通讯，而它本身又能够提供满足 Slave FIFO 要求的传输时序时，可采用 SlaveFIFO 传输方式。如下图所示。



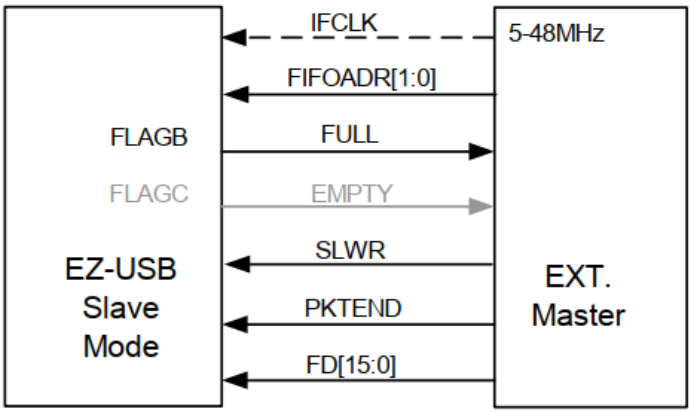
这种方式下，内嵌的 8051 固件只负责配置 Slave FIFO 相关的寄存器以及控制 FX2(CY7C68013A)何时工作在 Slave FIFO 模式下。

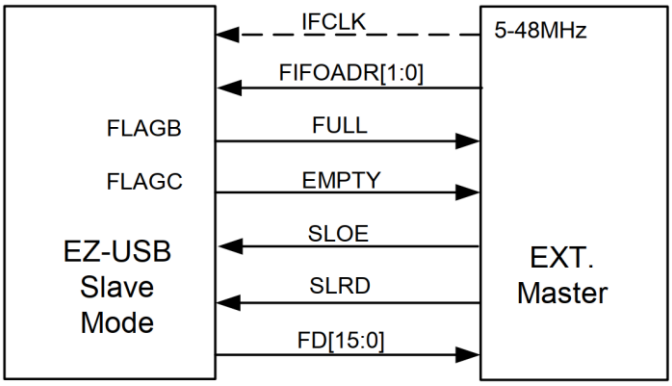
在 Slave FIFO 模式下,主控逻辑(FPGA)与 FX2(CY7C68013A)的连接如下图所示：



- IFCLK： FX2 输出的时钟，可做为通讯的同步时钟
- FLAGA/FLAGB/FLAGC/FLAGD： FX2 输出的 FIFO 状态信息，如满，空等
- SLCS： FIFO 的片选信号，外部逻辑控制，当 SLCS 输出高时，不可进行数据传输
- SLOE： FIFO 输出使能，外部逻辑控制，当 SLOE 无效时，数据线不输出有效数据
- SLRD： FIFO 读信号，外部逻辑控制，同步读时， FIFO 指针在 SLRD 有效时的每个 IFCLK 的上升沿递增，异步读时， FIFO 读指针在 SLRD 的每个有效—无效的跳变沿时递增
- SLWR： FIFO 写信号，外部逻辑控制，同步写时，在 SLWR 有效时的每个 IFCLK 的上升沿时数据被写入， FIFO 指针递增，异步写时，在 SLWR 的每个有效—无效的跳变沿时数据被写入， FIFO 写指针递增
- PKTEND：包结束信号，外部逻辑控制，在正常情况下，外部逻辑向 FX2 的 FIFO 中写数，当写入 FIFO 端点的字节数等于 FX2 固件设定的包大小时，数据将自动被打成一包进行传输，但有时外部逻辑可能需要传输一个字节数小于 FX2 固件设定的包大小的包，这时它只需在写入一定数目的字节后，声明此信号，此时 FX2 硬件不管外部逻辑写入了多少字节，都自动将之打成一包进行传输
- FD[15:0]：数据线；
- FIFOADR[1:0]：选择四个 FIFO 端点的地址线，外部逻辑控制

这里我们用到同步 FIFO 写和读(IFCLK 由 FPGA 提供)下图所示。





13.4. USB 固件开发

USB2.0 的固件程序开发在 KEIL2 下完成，在工程中添加 EZUSB 的 USBJmp.OBJ 和 EZUSB.LIB 两个目标和库文件，工程下有三个重要源文件：

- fw.c， FirmWare 的缩写， USB 协议方面的通信都是在这里完成的，包括上电枚举、重枚举、唤醒及调用用户程序等。 这个文件一般不用修改。
- bulkloop.c，用户代码在这里完成， Cypress 已经为我们搭好了框架。文件中包括 void TD_Init(void)、 void TD_Poll(void)、 BOOL DR_VendorCmnd(void)、 void ISR_Ep0in(void) interrupt 0～void ISR_Ep8inout(void) interrupt 0 函数。
- dscr.51， USB 描述符文件，包括了设备描述符、接口描述符、端点描述符、字符串等。用户可以根据自己的需求适当修改。

固件代码开放了两个节点，EP6 节点用于 PC 发送数据给 USB 芯片，即 OUT，EP2 节点用于 PC 接受 USB 给的数据，即 IN。在 dscr.51 中定义，具体代码如下：

```
;; Endpoint Descriptor
     db   DSCR_ENDPNT_LEN      ;; Descriptor length
     db   DSCR_ENDPNT          ;; Descriptor type
     db   82H             ;; Endpoint number, and direction
     db   ET_BULK           ;; Endpoint type
     db   40H             ;; Maximun packet size (LSB)
     db   00H             ;; Max packect size (MSB)
     db   00H             ;; Polling interval

;; Endpoint Descriptor
     db   DSCR_ENDPNT_LEN      ;; Descriptor length
     db   DSCR_ENDPNT          ;; Descriptor type
     db   06H             ;; Endpoint number, and direction
     db   ET_BULK           ;; Endpoint type
     db   40H             ;; Maximun packet size (LSB)
     db   00H             ;; Max packect size (MSB)
     db   00H             ;; Polling interval
```

在 TD_Init(void)中设置 Slave FIFO 的时钟为外部提供，即 FPGA 提供，具体代码如下所示：

```
//Slave 使用外部 48MHz 的时钟
IFCONFIG = 0x03; //External clock, 48 MHz, Slave FIFO interface
```

在 TD_Init(void)中设置 Slave FIFO 的数据接口为 8 位，即 WORDWIDE=0，具体代码如下所示：

```
SYNCDELAY;                  //
EP2FIFOCFG = 0x08;           // AUTOIN=1, ZEROLENIN=0, WORDWIDE=0
SYNCDELAY;                  //
EP6FIFOCFG = 0x10;           // AUTOOUT=1, WORDWIDE=0
```

在 BOOL DR_VendorCmnd(void)中定义两个命令 VR_NAKALL_ON 和 VR_NAKALL_OFF，用于执行上位机发送下来的命令，具体代码如下所示：

```
#define VR_NAKALL_ON    0xA8
#define VR_NAKALL_OFF   0xA9

//User request
BOOL DR_VendorCmnd(void)
{
  switch (SETUPDAT[1])
  {
     case VR_NAKALL_ON:
        EP0BCH = 0;
        EP0BCL = 0;  // Arm endpoint with # bytes to transfer
        EP0CS |= bmHSNAK; // Acknowledge handshake phase of device request
        break;
     case VR_NAKALL_OFF:
        EP0BCH = 0;
        EP0BCL = 0;  // Arm endpoint with # bytes to transfer
        EP0CS |= bmHSNAK; // Acknowledge handshake phase of device request

        SYNCDELAY;
        FIFORESET = 0x80;// activate NAK-ALL to avoid race conditions
        SYNCDELAY;
        FIFORESET = 0x02;// reset, FIFO 2
        SYNCDELAY;
        FIFORESET = 0x06;// reset, FIFO 6
        SYNCDELAY;
        FIFORESET = 0x00;// deactivate NAK-AL
        SYNCDELAY;
        break;
     default:
        return(TRUE);
  }

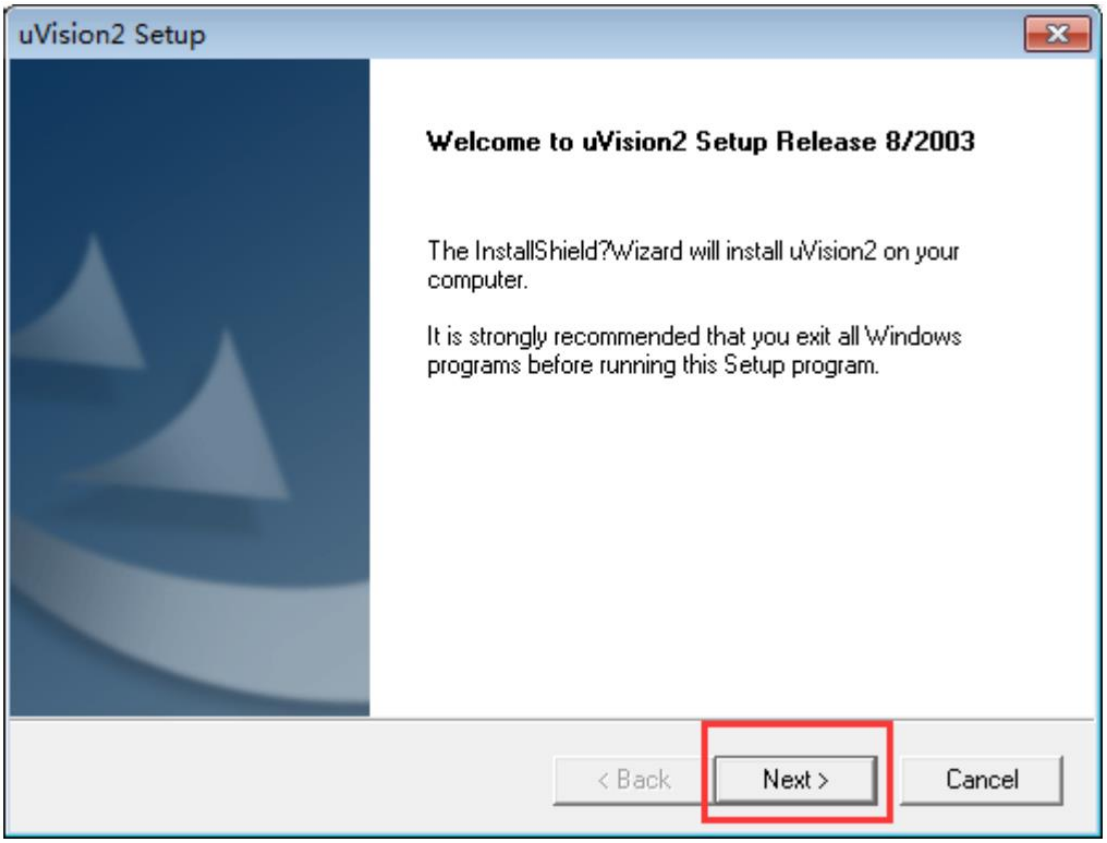
  return(FALSE);
}
```

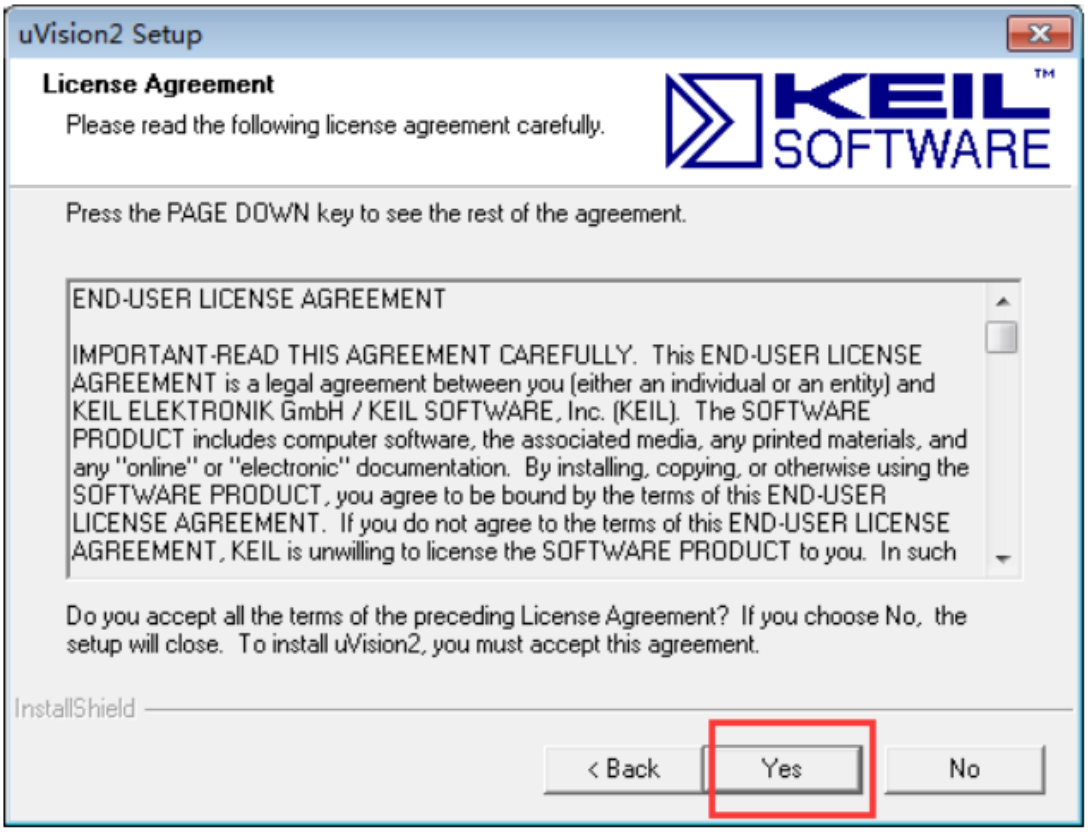
关于更多的固件代码读者可以参看具体工程源码和官方的技术文档，这里不再介绍。

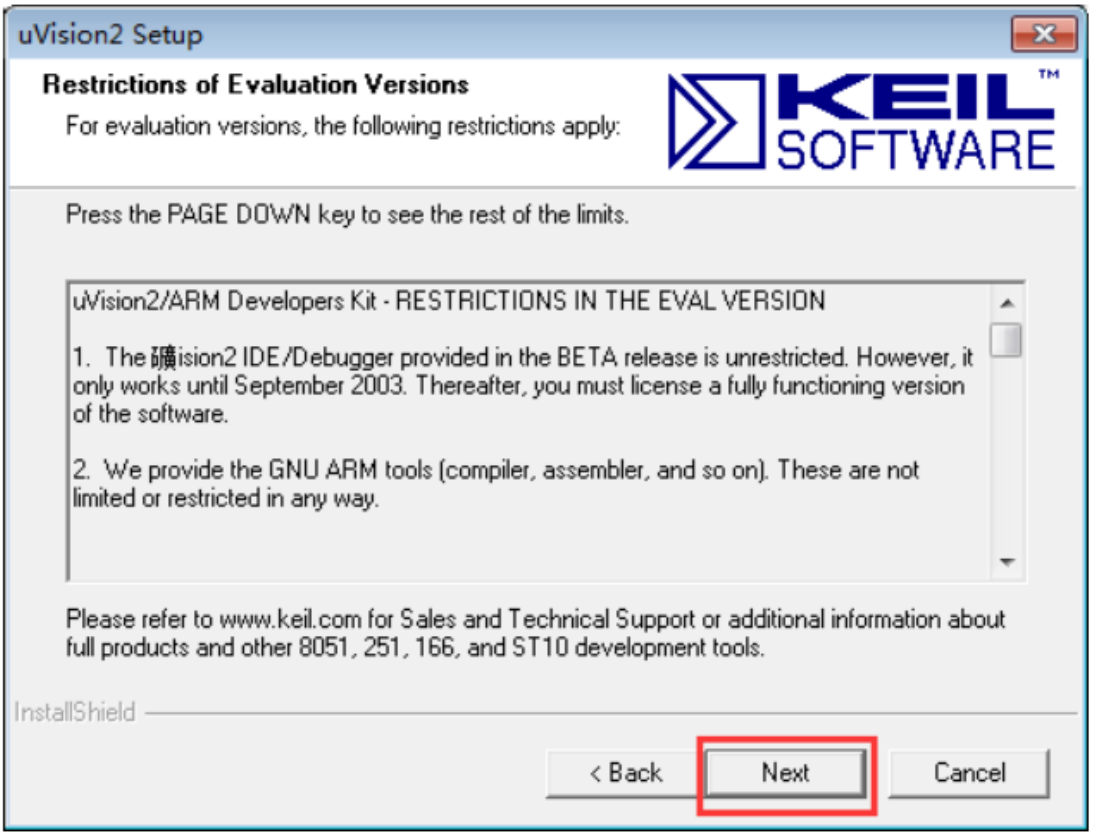
13.5. Keil 搭建工程

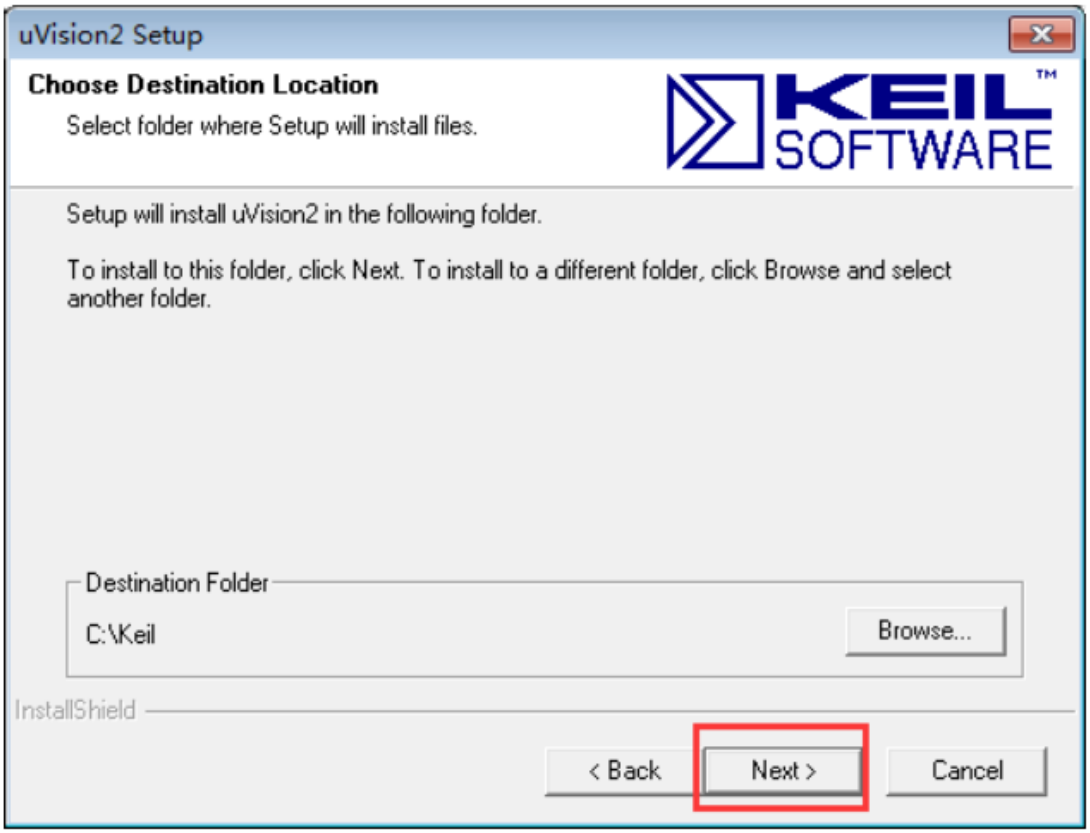
- 安装软件

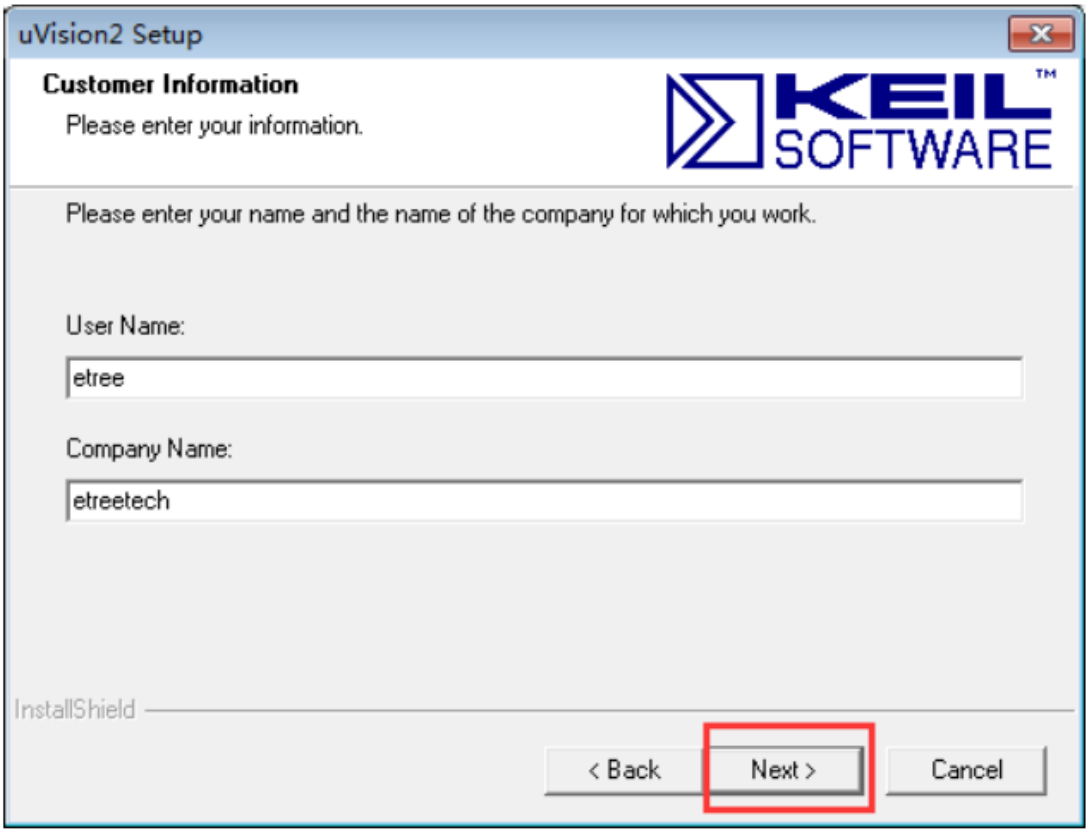
点击“USB2.0\uV2_4k\setup”目录下的“Setup.exe”文件，弹出下面安装界面，

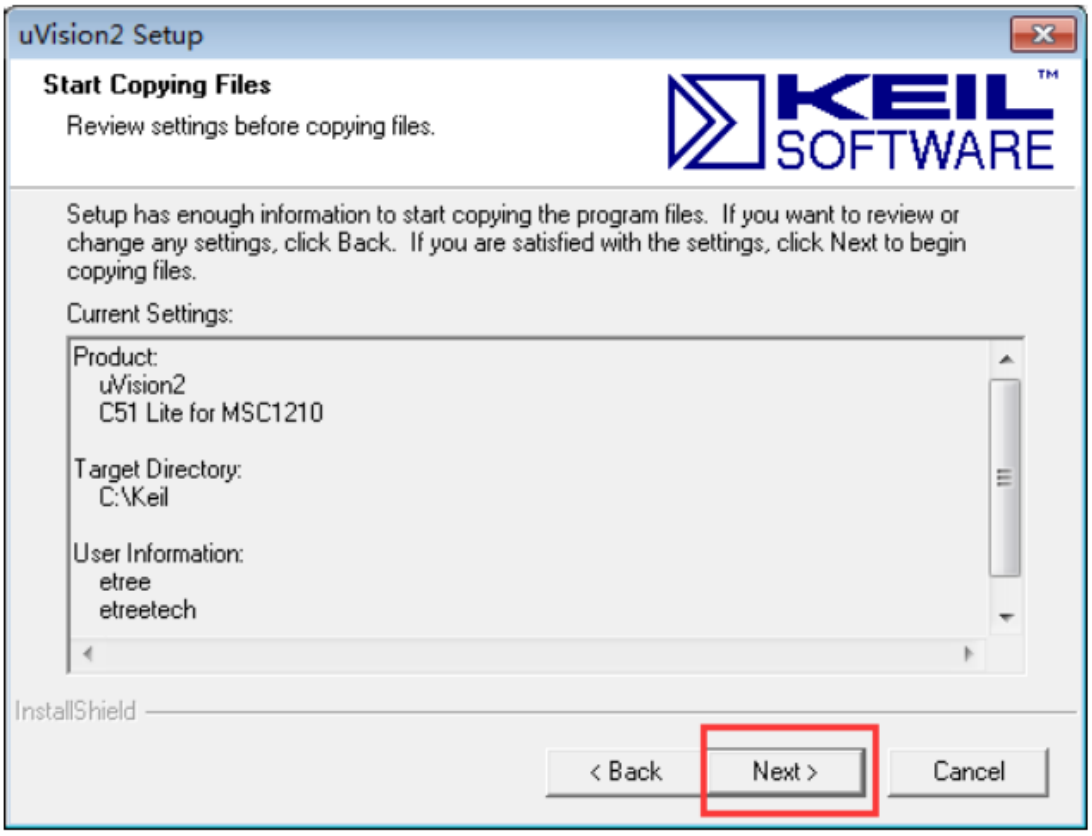


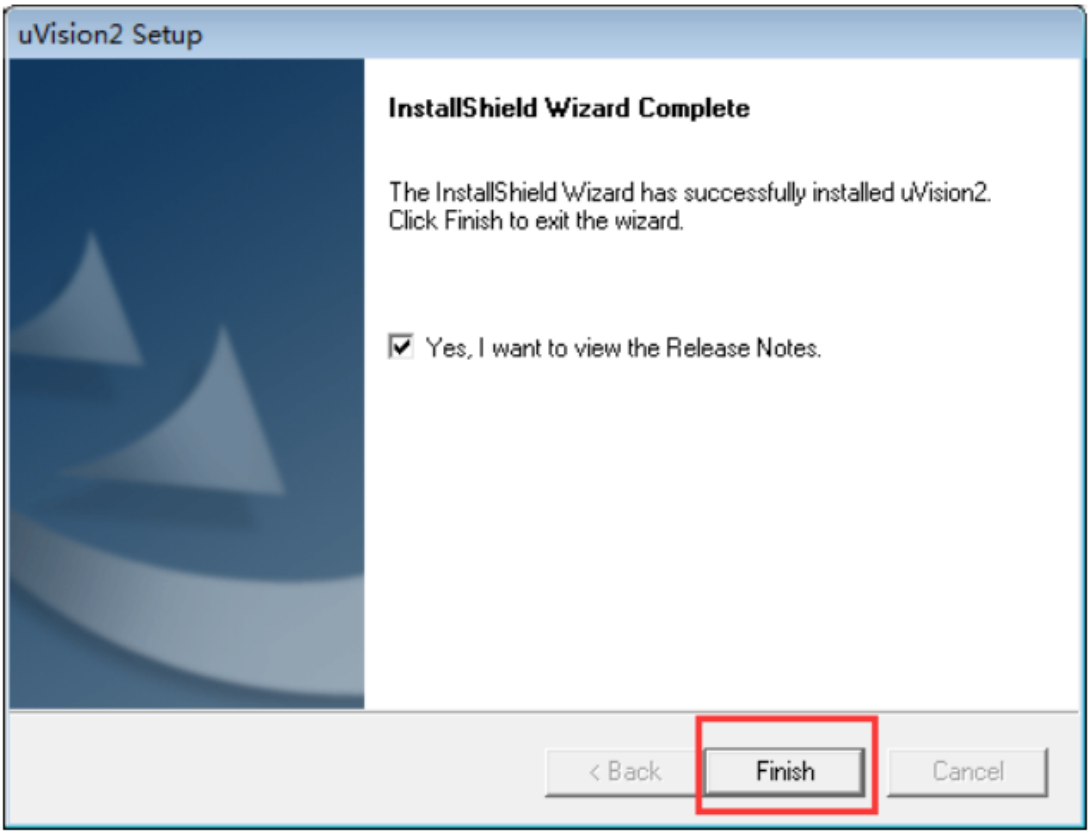

uVision2 Setup
License Agreement
Please read the following license agreement carefully.
KEIL SOFTWARE
Press the PAGE DOWN key to see the rest of the agreement.
END-USER LICENSE AGREEMENT
IMPORTANT-READ THIS AGREEMENT CAREFULLY. This END-USER LICENSE AGREEMENT is a legal agreement between you (either an individual or an entity) and KEIL ELEKTRONIK GmbH / KEIL SOFTWARE, Inc. (KEIL). The SOFTWARE PRODUCT includes computer software, the associated media, any printed materials, and any "online" or "electronic" documentation. By installing, copying, or otherwise using the SOFTWARE PRODUCT, you agree to be bound by the terms of this END-USER LICENSE AGREEMENT. If you do not agree to the terms of this END-USER LICENSE AGREEMENT, KEIL is unwilling to license the SOFTWARE PRODUCT to you. In such
Do you accept all the terms of the preceding License Agreement? If you choose No, the setup will close. To install uVision2, you must accept this agreement.
InstallShield
< Back
Yes
No


uVision2 Setup
Restrictions of Evaluation Versions
For evaluation versions, the following restrictions apply:
KEIL SOFTWARE
Press the PAGE DOWN key to see the rest of the limits.
uVision2/ARM Developers Kit - RESTRICTIONS IN THE EVAL VERSION
1. The 礦ision2 IDE/Debugger provided in the BETA release is unrestricted. However, it only works until September 2003. Thereafter, you must license a fully functioning version of the software.
2. We provide the GNU ARM tools (compiler, assembler, and so on). These are not limited or restricted in any way.
Please refer to www.keil.com for Sales and Technical Support or additional information about full products and other 8051, 251, 166, and ST10 development tools.
InstallShield
< Back
Next
Cancel


uVision2 Setup
Choose Destination Location
Select folder where Setup will install files.
KEIL SOFTWARE
Setup will install uVision2 in the following folder.
To install to this folder, click Next. To install to a different folder, click Browse and select another folder.
Destination Folder
C:\Keil
Browse...
InstallShield
< Back
Next >
Cancel


uVision2 Setup
Customer Information
Please enter your information.
KEIL SOFTWARE
Please enter your name and the name of the company for which you work.
User Name:
etree
Company Name:
etreetech
InstallShield
< Back
Next >
Cancel


uVision2 Setup
Start Copying Files
Review settings before copying files.
KEIL SOFTWARE
Setup has enough information to start copying the program files. If you want to review or change any settings, click Back. If you are satisfied with the settings, click Next to begin copying files.
Current Settings:
Product:
uVision2
C51 Lite for MSC1210
Target Directory:
C:\Keil
User Information:
etree
etreetech
InstallShield
< Back
Next >
Cancel


uVision2 Setup
InstallShield Wizard Complete
The InstallShield Wizard has successfully installed uVision2. Click Finish to exit the wizard.
Yes, I want to view the Release Notes.
< Back
Finish
Cancel

13.6. HDL 设计

根据 USB2.0 芯片的 Slave FIFO 接口定义，当 USB2.0 芯片的 Slave FIFO 为非满的状态时，写入测试数据，每过一个有效时钟循环增加 1，如下所示：

```
//file name:    usb_ctrl.v
//author:      ETree
//date:        2018.1.9
//function:    USB Slave FIFO Writer
//log:

module usb_ctrl(
    input rst_n,
    input ext_pclk,

    output  [7:0]      usb_dq,
    output            usb_pclk,
    output            usb_cs,
    output            usb_wr,
    output            usb_rd,
    output            usb_oe,
    output            usb_a0,
    output            usb_a1,
    output            usb_pkt,
    output            usb_pa0,
    input           usb_pd2,
    input           usb_flaga,
    input           usb_flagb,
    input           usb_flagc
);

reg ext_wr;
reg [7:0] ext_data;

parameter ST_IN_IDLE  = 1'b0;
parameter ST_IN_WRITE = 1'b1;

reg mystate;

always @(posedge ext_pclk or negedge rst_n)
    if(~rst_n)
    begin
        mystate <= ST_IN_IDLE;
        ext_wr <= 1'b0;
    end
```

```
    else
    case(mystate)
    ST_IN_IDLE:
    begin
        if(usb_flagb)
            mystate <= ST_IN_WRITE;
        else
            mystate <= ST_IN_IDLE;
        ext_wr <= 1'b0;
    end
    ST_IN_WRITE:
    begin
        if(usb_flagb)
        begin
            mystate <= ST_IN_WRITE;
            ext_wr <= 1'b1;
        end
        else
        begin
            mystate <= ST_IN_IDLE;
            ext_wr <= 1'b0;
        end
    end
    default:;
    endcase

always @(posedge ext_pclk)
    if(~rst_n)
    begin
        ext_data <= 0;
    end
    else if(ext_wr)
        ext_data <= ext_data + 1'b1;
    else
        ext_data <= ext_data;


assign usb_cs   = 1'b0;
assign usb_rd   = 1'b1;
assign usb_wr   = ~ext_wr;
assign usb_oe   = 1'b1;

assign usb_a0   = 1'b0;
assign usb_a1   = 1'b1;
```

```
assign usb_pkt  = 1'b1;

assign usb_pclk = ~ext_pclk;
assign usb_dq   = ext_data;
assign usb_pa0  = 1;

endmodule
```

13.7. 管教约束

选择菜单 Assignment -> Pin Planner，添加管脚约束。

in cam_data[7]	Input	PIN_A13	7
in cam_data[6]	Input	PIN_B12	7
in cam_data[5]	Input	PIN_A12	7
in cam_data[4]	Input	PIN_B11	7
in cam_data[3]	Input	PIN_A11	7
in cam_data[2]	Input	PIN_B10	7
in cam_data[1]	Input	PIN_A10	7
in cam_data[0]	Input	PIN_B9	7
in cam_hsync	Input	PIN_B13	7
in cam_pclk	Input	PIN_F11	7
out cam_pdown	Output	PIN_C9	7
out cam_reset	Output	PIN_A9	7
out cam_scl	Output	PIN_A15	7
io cam_sda	Bidir	PIN_B14	7
in cam_vsync	Input	PIN_A14	7
out cam_xclk	Output	PIN_D12	7
in clk	Input	PIN_M16	5
out led	Output	PIN_L12	5
in rst_n	Input	PIN_M12	5
out sdram_addr[12]	Output	PIN_N2	2
out sdram_addr[11]	Output	PIN_N1	2
out sdram_addr[10]	Output	PIN_P3	3
out sdram_addr[9]	Output	PIN_L2	2
out sdram_addr[8]	Output	PIN_L1	2
out sdram_addr[7]	Output	PIN_K2	2
out sdram_addr[6]	Output	PIN_K1	2
out sdram_addr[5]	Output	PIN_J2	2
out sdram_addr[4]	Output	PIN_J1	2
out sdram_addr[3]	Output	PIN_B3	8
out sdram_addr[2]	Output	PIN_A3	8
out sdram_addr[1]	Output	PIN_B4	8
out sdram_addr[0]	Output	PIN_A4	8
out sdram_ba[1]	Output	PIN_C3	8
out sdram_ba[0]	Output	PIN_N6	3
out sdram_cas_n	Output	PIN_N5	3
out sdram_cke	Output	PIN_P1	2
out sdram_clk	Output	PIN_P2	2

sdram_dq[7]	Bidir	PIN_M7	3
sdram_dq[6]	Bidir	PIN_L3	2
sdram_dq[5]	Bidir	PIN_K8	3
sdram_dq[4]	Bidir	PIN_D6	8
sdram_dq[3]	Bidir	PIN_K6	2
sdram_dq[2]	Bidir	PIN_D3	8
sdram_dq[1]	Bidir	PIN_K5	2
sdram_dq[0]	Bidir	PIN_J6	2
sdram_dqm[1]	Output	PIN_R1	2
sdram_dqm[0]	Output	PIN_L6	2
sdram_ras_n	Output	PIN_N3	3
sdram_we_n	Output	PIN_L4	2
usb_dq[7]	Output	PIN_L11	4
usb_dq[6]	Output	PIN_J14	5
usb_dq[5]	Output	PIN_G11	6
usb_dq[4]	Output	PIN_K11	5
usb_dq[3]	Output	PIN_F14	6
usb_dq[2]	Output	PIN_J11	5
usb_dq[1]	Output	PIN_F13	6
usb_dq[0]	Output	PIN_C16	6
usb_flaga	Input	PIN_D15	6
usb_flagb	Input	PIN_D16	6
usb_flagc	Input	PIN_F15	6
usb_pa0	Output	PIN_F16	6
usb_pa1	Output	PIN_G15	6
usb_pa2	Output	PIN_G16	6
usb_pa4	Output	PIN_J16	5
usb_pa5	Output	PIN_J15	5
usb_pa6	Output	PIN_K16	5
usb_pa7	Output	PIN_K15	5
usb_pclk	Output	PIN_B16	6
usb_pd0	Input	PIN_L16	5
usb_pd1	Input	PIN_L15	5
usb_pd2	Input	PIN_N16	5
usb_pd3	Input	PIN_M11	4
usb_pd4	Input	PIN_L14	5
usb_pd5	Input	PIN_N15	5
usb_pd6	Input	PIN_L13	5
usb_pd7	Input	PIN_N14	5
usb_rd	Output	PIN_N12	4
usb_wr	Output	PIN_K12	5

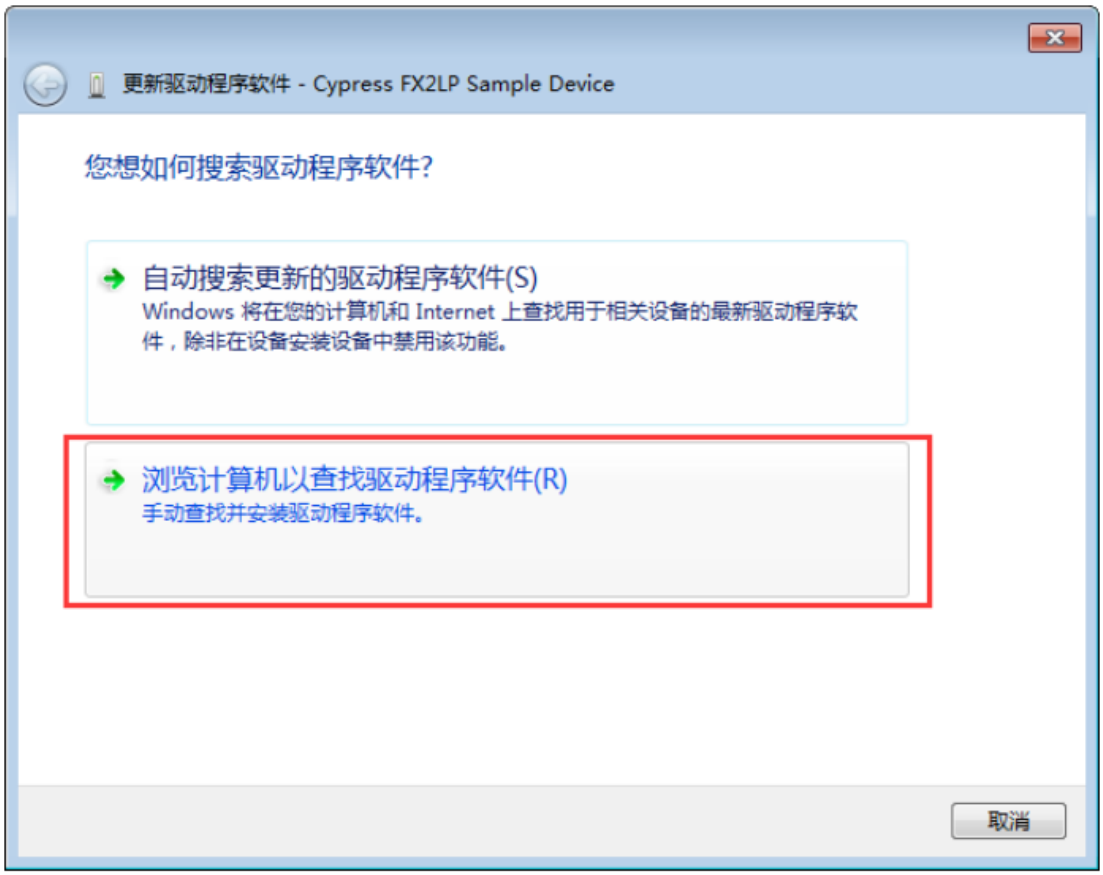
13.8. 实验操作

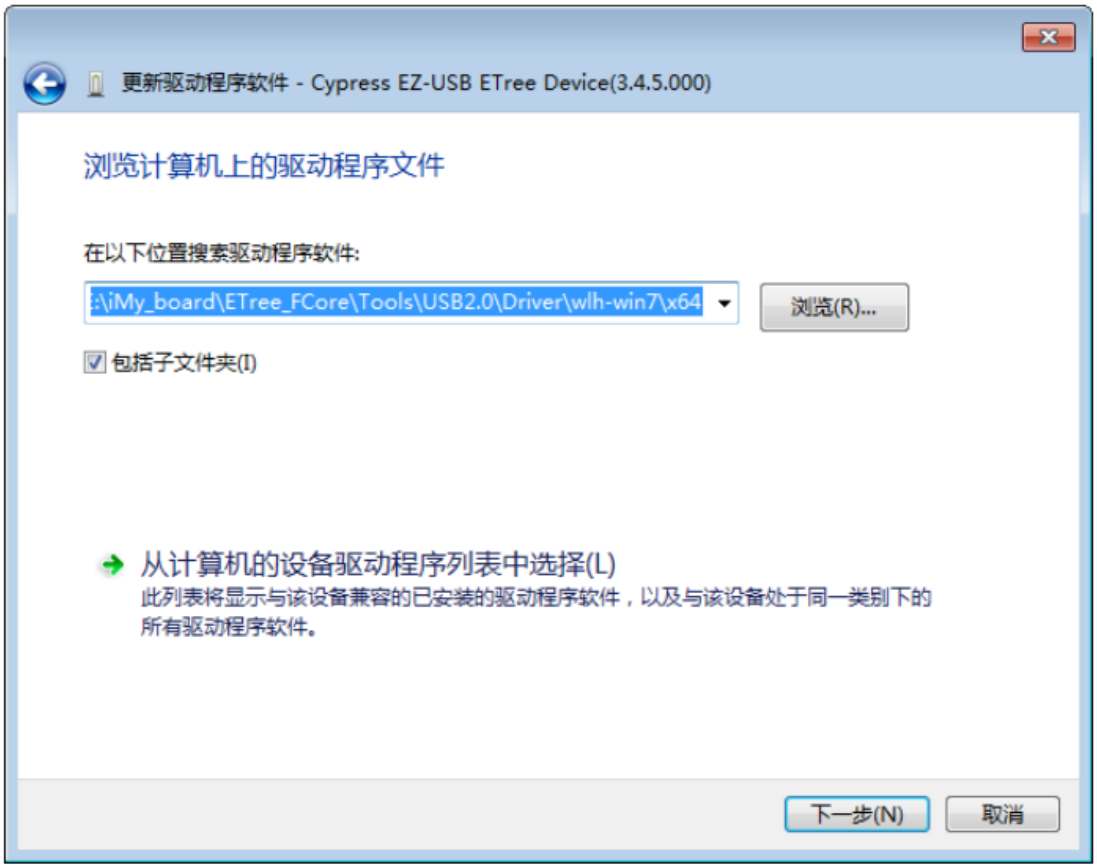
- 安装 USB2.0 驱动

用 MINI-USB 线将 FCORE2 核心板卡和电脑相连，在设备管理器中会看到“EZ-USB”设备，这里有黄色的感叹号表示没有正确安装驱动。



右击“EZ-USB”手动更新驱动程序，如下图所示(以 WIN7 64 位系统为例)，驱动安装目录在“USB2.0\driver\bin\win7\x64”下，如果是 WIN7 32 位系统选择 x86 目录。驱动还支持 WINXP/WIN8/WIN8.1/WIN10 等系统。





点击“始终安装此类驱动程序软件”。

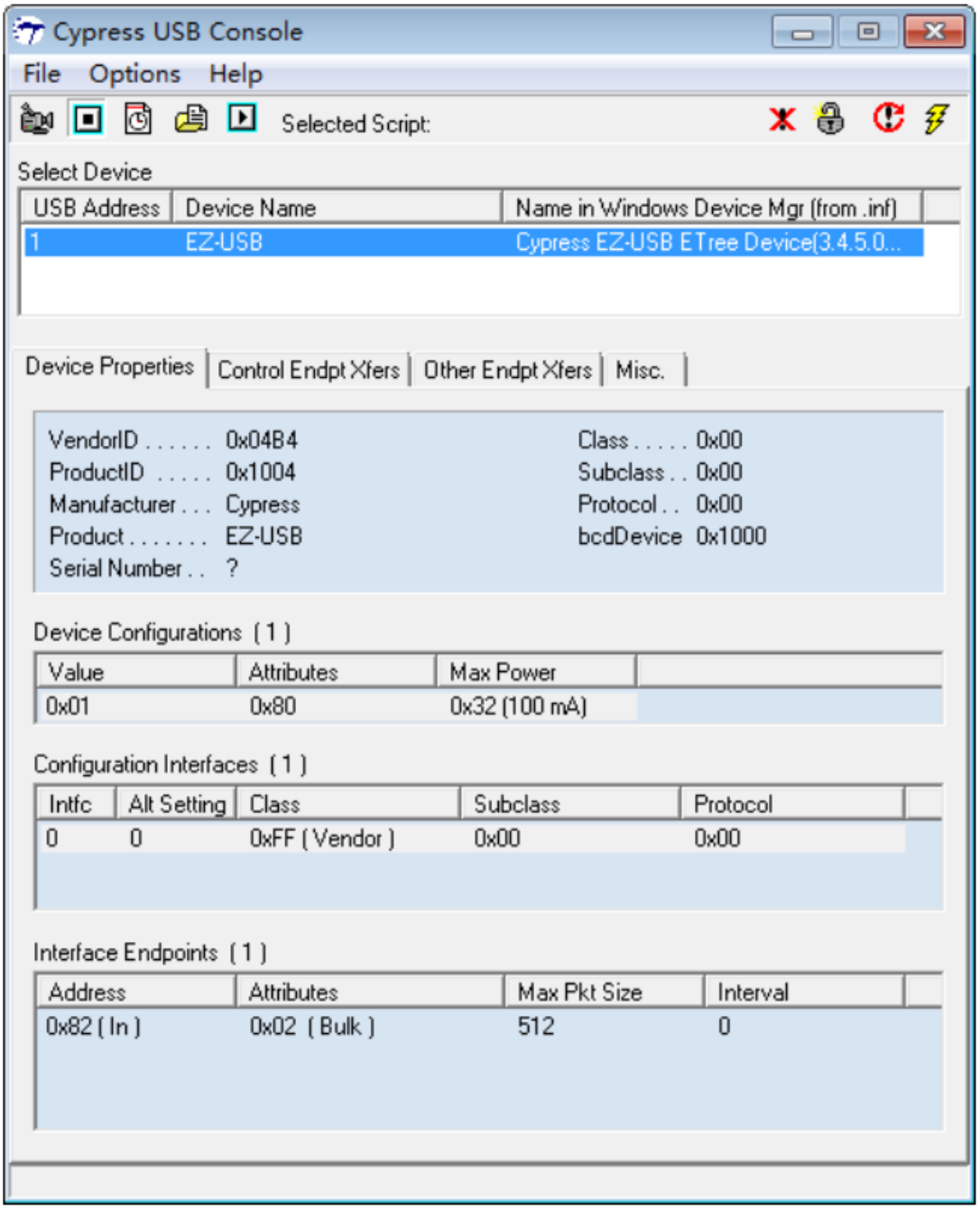


安装好驱动后，在设备管理器中可以看到装好的 USB 设备，如下图所示。

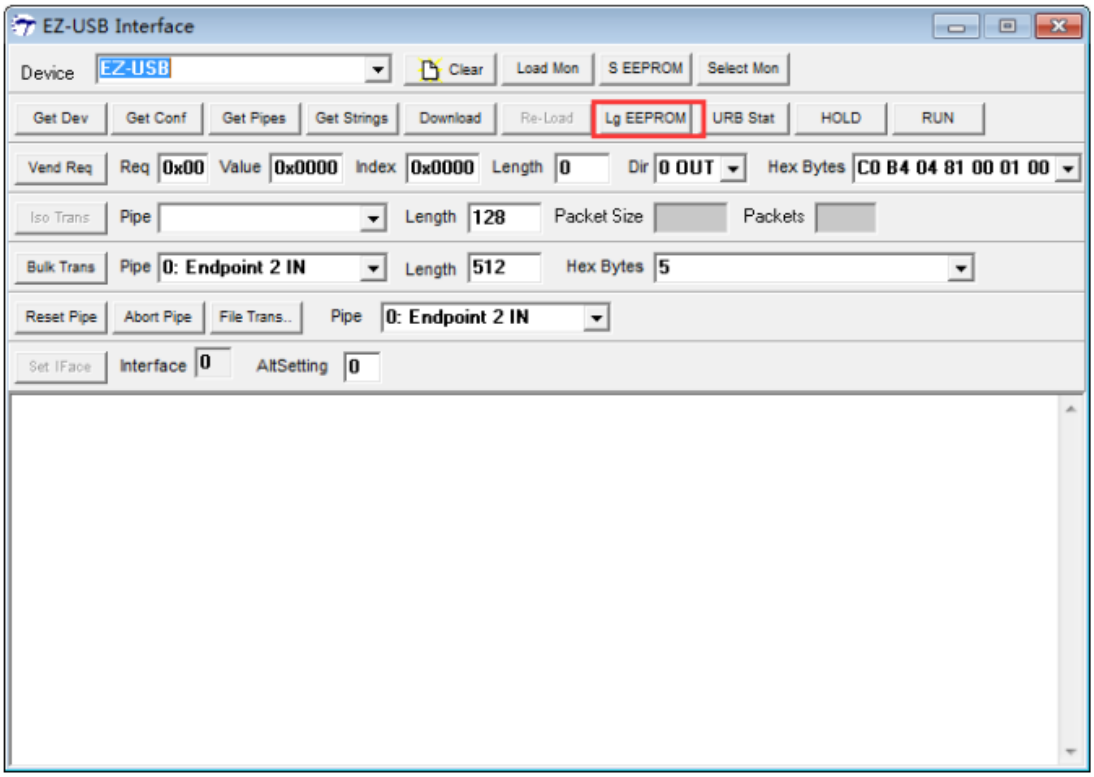


➢ 下载 USB 固件代码

下载固件代码前需要把 USB2.0 模块板卡上的跳线帽拔掉，不然不能识别。打开“USB2.0”目录下的“CyConsole.exe”，界面如下图所示。



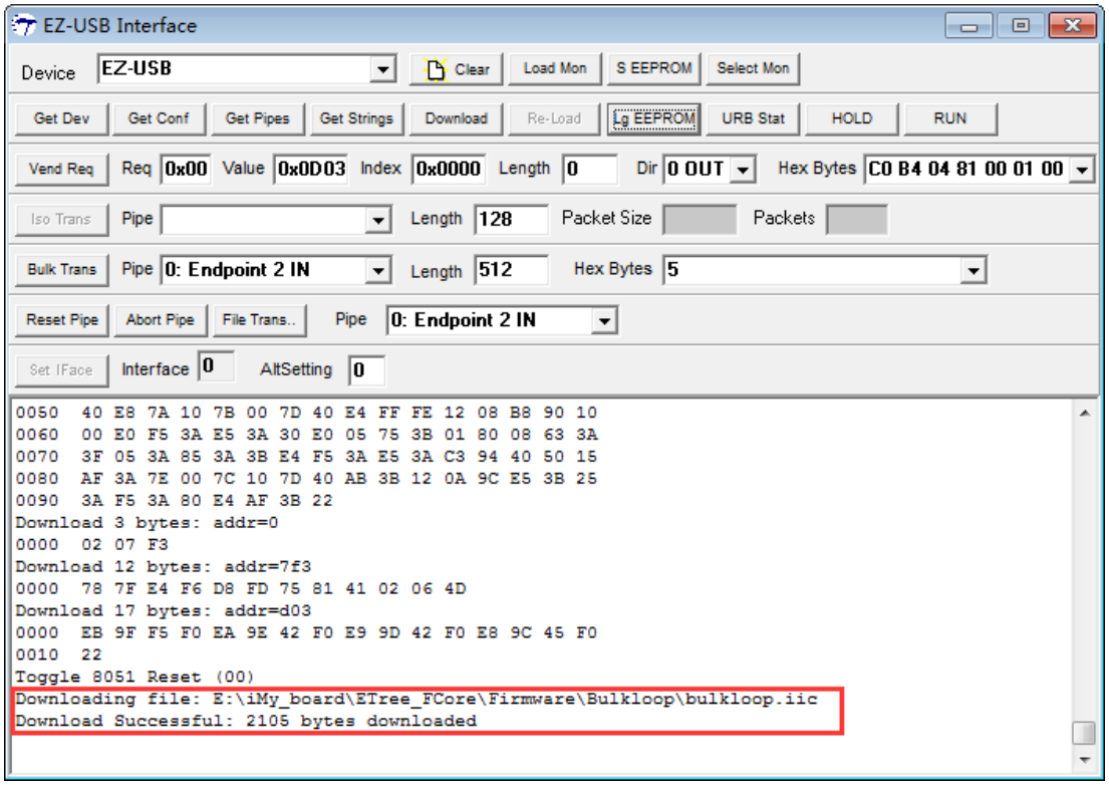
点击“Option-->EZ-USB Interface”弹出下图界面。



点击“Lg EEPROM”,选择需要下载的固件 iic 文件，固件下载文件在“USB2.0\Firmware\Bulkloop”目录下。



下载完成后会提示下面的提示，表示烧写成功。

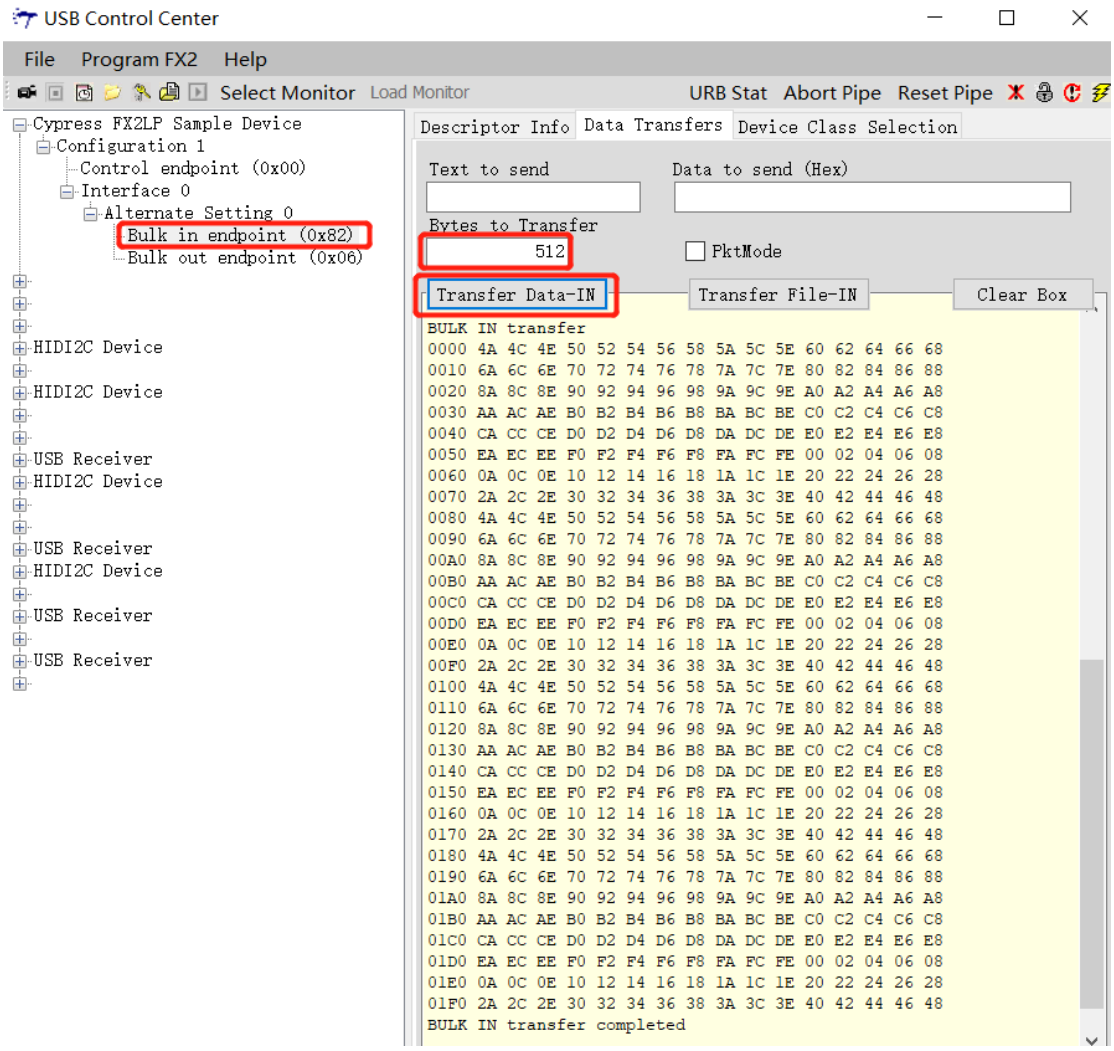


- 下载 FPGA 配置代码

  下载 output_file.jic 文件到 SPI Flash 中，断电后重启。

- 上位机 USB 接受演示

重新插拔 MINI-USB2.0 线缆(断电重启)，打开“USB2.0\USB Control Center\”目录下的“CyControl.exe”，选中 Bulk in endpoint(0x82)，设置接受长度为 512，点击“Transfer Data-IN”按钮，可以看到 FPGA 发送上来的测试渐进数据，如下图所示：

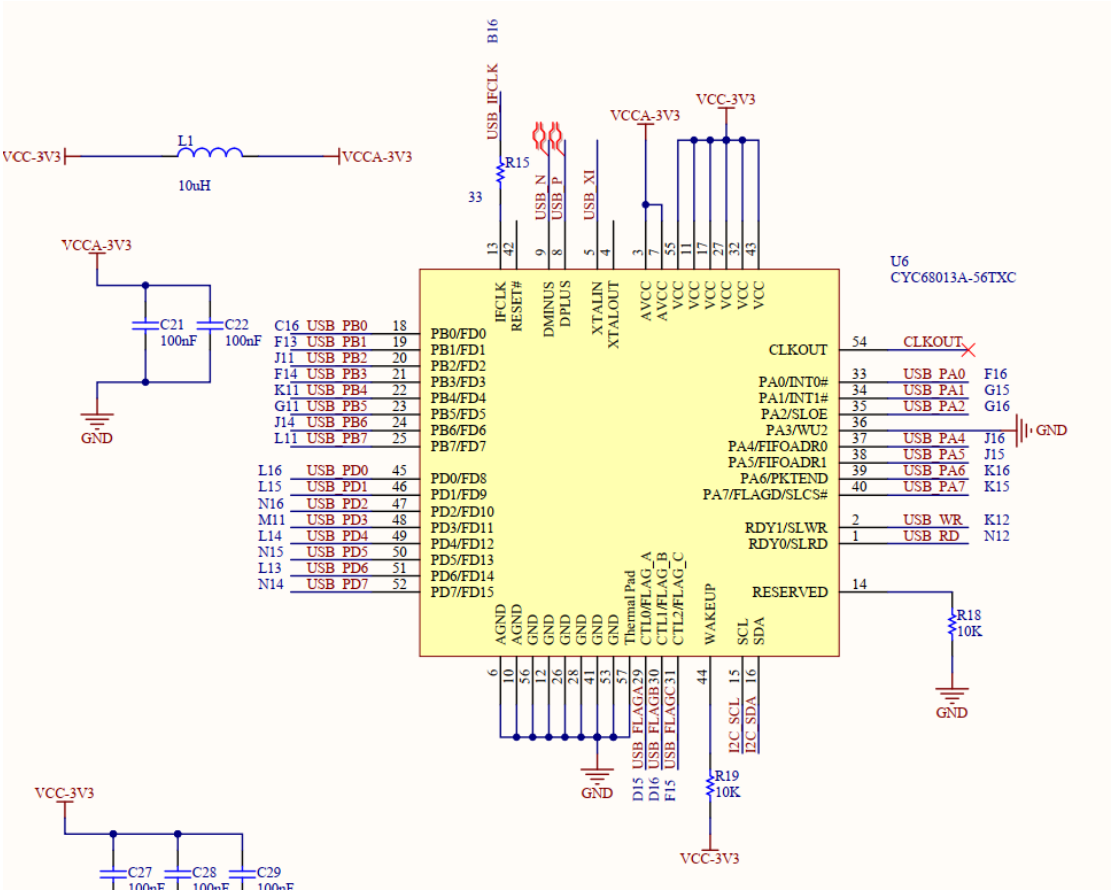

USB Control Center
File
Program FX2
Help
Select Monitor
Load Monitor
URB Stat
Abort Pipe
Reset Pipe
Cypress FX2LP Sample Device
Configuration 1
Control endpoint (0x00)
Interface 0
Alternate Setting 0
Bulk in endpoint (0x82)
Bulk out endpoint (0x06)
HIDI2C Device
HIDI2C Device
USB Receiver
HIDI2C Device
USB Receiver
HIDI2C Device
USB Receiver
USB Receiver
Descriptor Info
Data Transfers
Device Class Selection
Text to send
Data to send (Hex)
Bytes to Transfer
512
PktMode
Transfer Data-IN
Transfer File-IN
Clear Box
BULK IN transfer
0000 4A 4C 4E 50 52 54 56 58 5A 5C 5E 60 62 64 66 68
0010 6A 6C 6E 70 72 74 76 78 7A 7C 7E 80 82 84 86 88
0020 8A 8C 8E 90 92 94 96 98 9A 9C 9E A0 A2 A4 A6 A8
0030 AA AC AE B0 B2 B4 B6 B8 BA BC BE C0 C2 C4 C6 C8
0040 CA CC CE D0 D2 D4 D6 D8 DA DC DE E0 E2 E4 E6 E8
0050 EA EC EE F0 F2 F4 F6 F8 FA FC FE 00 02 04 06 08
0060 0A 0C 0E 10 12 14 16 18 1A 1C 1E 20 22 24 26 28
0070 2A 2C 2E 30 32 34 36 38 3A 3C 3E 40 42 44 46 48
0080 4A 4C 4E 50 52 54 56 58 5A 5C 5E 60 62 64 66 68
0090 6A 6C 6E 70 72 74 76 78 7A 7C 7E 80 82 84 86 88
00A0 8A 8C 8E 90 92 94 96 98 9A 9C 9E A0 A2 A4 A6 A8
00B0 AA AC AE B0 B2 B4 B6 B8 BA BC BE C0 C2 C4 C6 C8
00C0 CA CC CE D0 D2 D4 D6 D8 DA DC DE E0 E2 E4 E6 E8
00D0 EA EC EE F0 F2 F4 F6 F8 FA FC FE 00 02 04 06 08
00E0 0A 0C 0E 10 12 14 16 18 1A 1C 1E 20 22 24 26 28
00F0 2A 2C 2E 30 32 34 36 38 3A 3C 3E 40 42 44 46 48
0100 4A 4C 4E 50 52 54 56 58 5A 5C 5E 60 62 64 66 68
0110 6A 6C 6E 70 72 74 76 78 7A 7C 7E 80 82 84 86 88
0120 8A 8C 8E 90 92 94 96 98 9A 9C 9E A0 A2 A4 A6 A8
0130 AA AC AE B0 B2 B4 B6 B8 BA BC BE C0 C2 C4 C6 C8
0140 CA CC CE D0 D2 D4 D6 D8 DA DC DE E0 E2 E4 E6 E8
0150 EA EC EE F0 F2 F4 F6 F8 FA FC FE 00 02 04 06 08
0160 0A 0C 0E 10 12 14 16 18 1A 1C 1E 20 22 24 26 28
0170 2A 2C 2E 30 32 34 36 38 3A 3C 3E 40 42 44 46 48
0180 4A 4C 4E 50 52 54 56 58 5A 5C 5E 60 62 64 66 68
0190 6A 6C 6E 70 72 74 76 78 7A 7C 7E 80 82 84 86 88
01A0 8A 8C 8E 90 92 94 96 98 9A 9C 9E A0 A2 A4 A6 A8
01B0 AA AC AE B0 B2 B4 B6 B8 BA BC BE C0 C2 C4 C6 C8
01C0 CA CC CE D0 D2 D4 D6 D8 DA DC DE E0 E2 E4 E6 E8
01D0 EA EC EE F0 F2 F4 F6 F8 FA FC FE 00 02 04 06 08
01E0 0A 0C 0E 10 12 14 16 18 1A 1C 1E 20 22 24 26 28
01F0 2A 2C 2E 30 32 34 36 38 3A 3C 3E 40 42 44 46 48
BULK IN transfer completed

第十四章 USB Slave FIFO 读

上一章基于 FPGA 和 USB2.0 芯片(68013a)实现 FPGA 发送数据包给 PC，这一章在上一章基础上实现 PC 发送数据包给 FPGA。

14.1. 硬件介绍

开发板板载 USB2.0 芯片，具体原理图如下图所示：



14.2. USB 固件开发

同上一章

14.3. HDL 设计

根据 USB2.0 芯片的 Slave FIFO 接口定义，当 USB2.0 芯片的 Slave FIFO 为非空的状态时，读出测试数据，如下所示：

```
//file name:    usb_ctrl.v
//author:       ETree
//date:         2018.1.9
//function:     USB Slave FIFO READ
```

```
//log:

module usb_ctrl(
    input rst_n,
    input ext_pclk,

    input  [7:0]       usb_dq,
    output            usb_pclk,
    output            usb_cs,
    output            usb_wr,
    output            usb_rd,
    output            usb_oe,
    output            usb_a0,
    output            usb_a1,
    output            usb_pkt,
    output            usb_pa0,
    input           usb_pd2,
    input           usb_flaga,
    input           usb_flagb,
    input           usb_flagc
);

reg ext_rd;
(* noprune *)reg ext_data_valid;
(* noprune *)reg [7:0] ext_data;

parameter [1:0] ST_OUT_IDLE   = 2'b00;
parameter [1:0] ST_OUT_READ   = 2'b01;
parameter [1:0] ST_OUT_WAIT   = 2'b10;

reg [1:0] mystate;

always @(posedge ext_pclk)
    if(~rst_n)
    begin
        mystate <= ST_OUT_IDLE;
        ext_rd <= 1'b0;
    end
    else
    case(mystate)
    ST_OUT_IDLE:
    begin
        if(usb_flagc==1'b1)
            mystate <= ST_OUT_READ;
```

```
        else
            mystate <= ST_OUT_IDLE;
        ext_rd <= 1'b0;
    end
    ST_OUT_READ:
    begin
        if(usb_flagc==1'b0)
        begin
            mystate <= ST_OUT_WAIT;
            ext_rd <= 1'b0;
        end
        else
        begin
            mystate <= ST_OUT_READ;
            ext_rd <= 1'b1;
        end
    end
    default:
    begin
        mystate <= ST_OUT_IDLE;
        ext_rd <= 1'b0;
    end
    endcase


always @(posedge ext_pclk)
    if(~rst_n)
    begin
        ext_data_valid <= 0;
    end
    else if(ext_rd && usb_flagc)
        ext_data_valid <= 1'b1;
    else
        ext_data_valid <= 1'b0;

always @(posedge ext_pclk)
    if(~rst_n)
    begin
        ext_data <= 0;
    end
    else if(ext_rd)
        ext_data <= usb_dq;
    else
        ext_data <= ext_data;
```

```
assign usb_cs   = 1'b0;
assign usb_rd   = ~ext_rd;
assign usb_wr   = 1'b1;
assign usb_oe   = 1'b0;

assign usb_a0   = 1'b0;
assign usb_a1   = 1'b0;

assign usb_pkt  = 1'b1;

assign usb_pclk = ext_pclk;
assign usb_pa0  = 1;

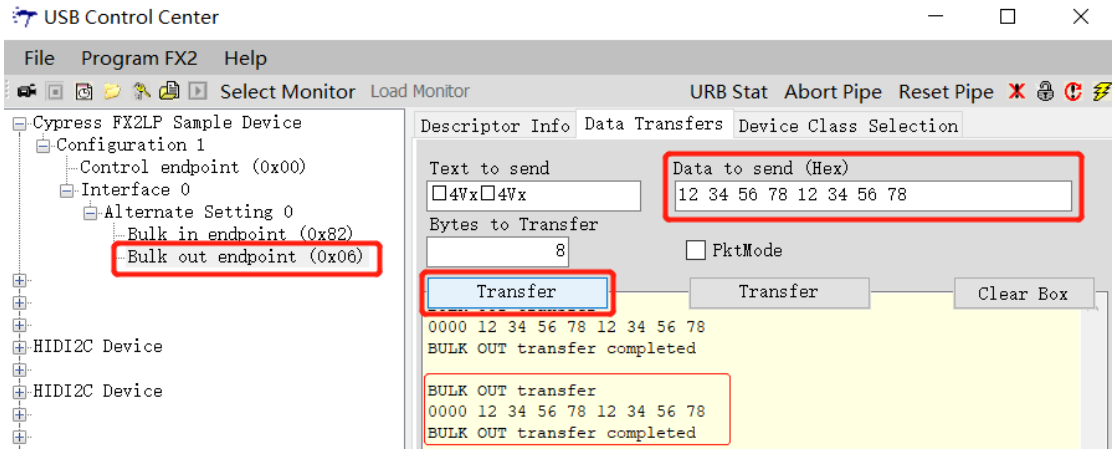
endmodule
```

14.4. 管教约束

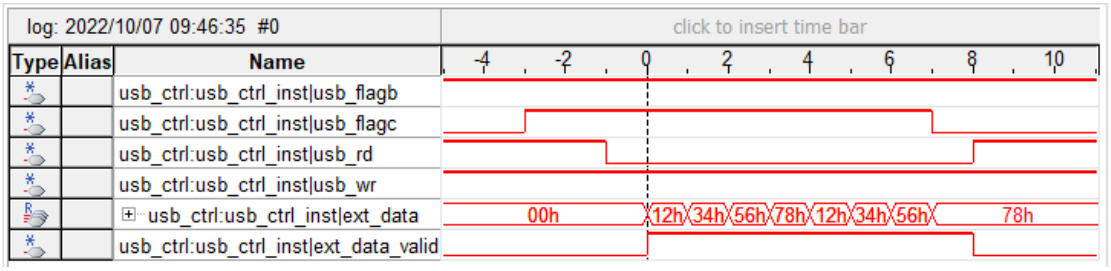
同上一章

14.5. 实验操作

下载 FPGA 配置代码和 USB 固件代码，打开 SignalTap-II，设置 ext_data_valid 上升沿触发，等待上位机发送数据。打开“USB2.0\USB Control Center\”目录下的“CyControl.exe”，选中 Bulk out endpoint(0x06)，输入 8 个字节的数据(0x12,0x34,0x56,0x78, 0x12,0x34,0x56,0x78)，点击右边的“Transfer”按钮，数据就发送下去，如下图所示：



在 SignalTap-II 处可以捕获到 PC 发送过来的 8 个字节数据，如下图所示，和 PC 发送的数据一致。

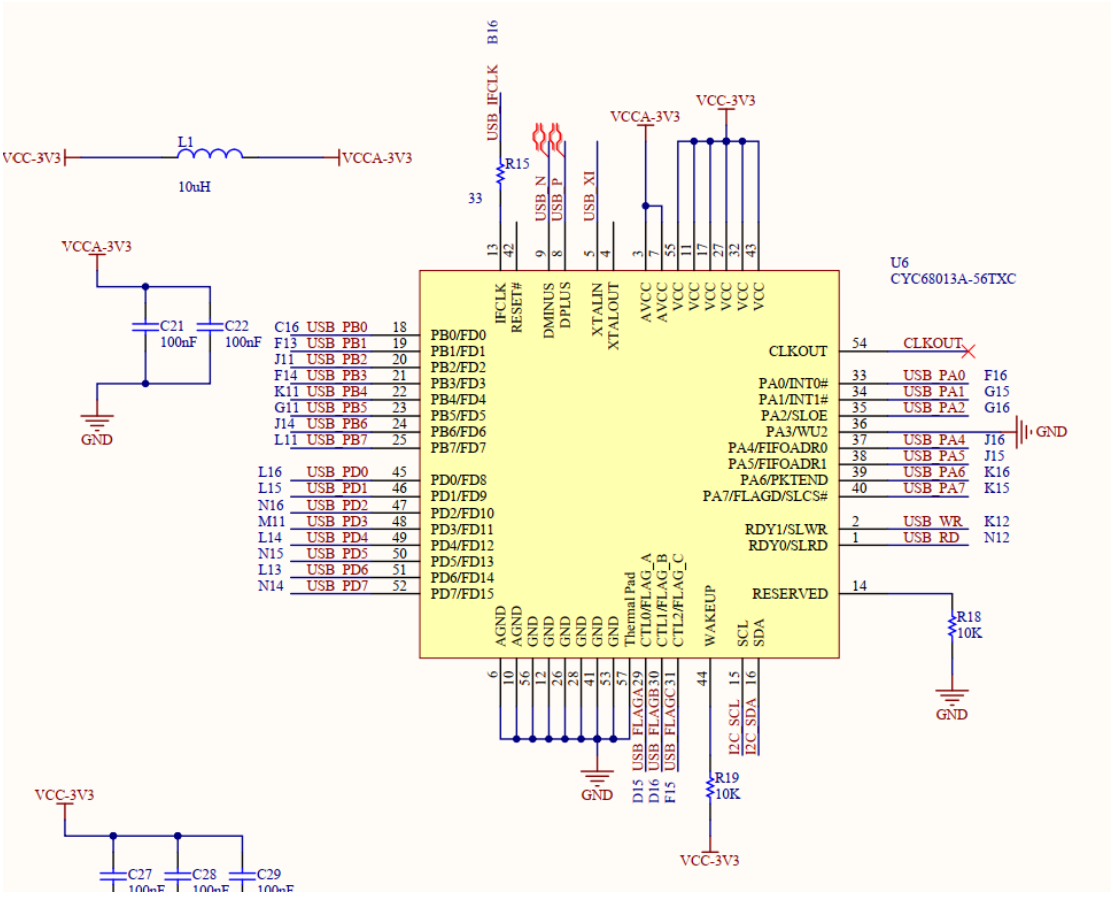

log: 2022/10/07 09:46:35 #0
click to insert time bar
Type
Alias
Name
-4
-2
0
2
4
6
8
10
usb_ctrl:usb_ctrl_inst|usb_flagb
usb_ctrl:usb_ctrl_inst|usb_flagc
usb_ctrl:usb_ctrl_inst|usb_rd
usb_ctrl:usb_ctrl_inst|usb_wr
usb_ctrl:usb_ctrl_inst|ext_data
usb_ctrl:usb_ctrl_inst|ext_data_valid
00h
12h
34h
56h
78h
12h
34h
56h
78h

第十五章 USB 上位机数据显示

本章 FPGA 生成视频信号(640*480@30 帧)发送给 USB2.0 芯片(68013a)，USB2.0 芯片(68013a)将 FPGA 发送过来的视频信号打包发送给 PC，上位机软件通过 USB 驱动实现视频采集与播放功能。

15.1. 硬件介绍

开发板板载 USB2.0 芯片，具体原理图如下图所示：

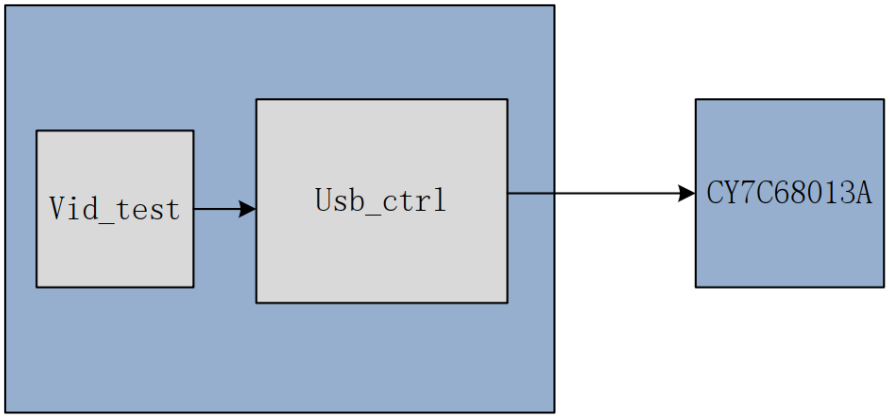


15.2. USB 固件开发

同上一章

15.3. HDL 设计

- 系统框架



- vid_test 模块负责生成测试视频数据(640*480@30 帧)；
- usb_ctrl 模块负责生产 USB2.0 芯片引脚配置和数据转发给 USB2.0 芯片的 slave FIFO 接口。

➢ vid_test 设计

产生测试视频数据，视频格式为 640*480@30 帧，视频内容为循环渐进数据。

➢ usb_ctrl 设计

usb_ctrl 模块配置对应的 IO 状态，同时 vid_test 的视频数据发送到 USB 的 Slave FIFO 端口，具体代码如下：

```
//file name:    usb_ctrl.v
//author:      ETree
//date:        2018.1.9
//function:    USB Slave FIFO controllor
//log:

module usb_ctrl(
    input rst_n,
    input cam_vsync,
    input cam_hsync,
    input [7:0] cam_data,
    input cam_pclk,

    output  [7:0]       usb_dq,
    output             usb_pclk,
    output             usb_cs,
    output             usb_wr,
    output             usb_rd,
    output             usb_oe,
    output             usb_a0,
    output             usb_a1,
    output             usb_pkt,
    output             usb_pa0,
    input              usb_pa1
```

```
);

reg cam_vsync_r0;
reg cam_vsync_r1;

always @(posedge cam_pclk)
begin
    cam_vsync_r0 <= cam_vsync;
    cam_vsync_r1 <= cam_vsync_r0;
end

wire cam_vsync_pos;
assign cam_vsync_pos = (cam_vsync_r0==1'b1 && cam_vsync_r1==1'b0);

reg [7:0] frame_cnt;

always @(posedge cam_pclk)
    if(usb_pa1)
        if(cam_vsync_pos)
            frame_cnt <= frame_cnt + 1'b1;
        else;
    else
        frame_cnt <= 0;

assign usb_cs  = 1'b0;
assign usb_rd  = 1'b1;
assign usb_oe  = 1'b1;

assign usb_a0 = 1'b0;
assign usb_a1 = 1'b0;

assign usb_pkt = 1'b1;

assign usb_pclk    = ~cam_pclk;
assign usb_wr      = ~(cam_hsync && (frame_cnt>=8));
assign usb_dq      = cam_data;
assign usb_pa0     = 1;

endmodule
```

15.4. 管脚约束

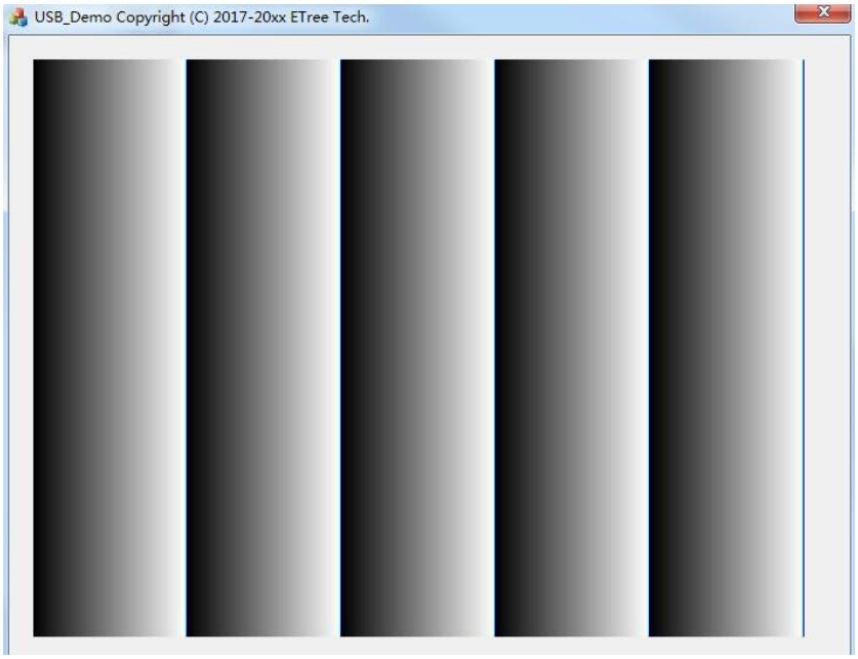
选择菜单 Assignment -> Pin Planner，添加管脚约束。

cam_data[7]	Input	PIN_A13	7
cam_data[6]	Input	PIN_B12	7
cam_data[5]	Input	PIN_A12	7
cam_data[4]	Input	PIN_B11	7
cam_data[3]	Input	PIN_A11	7
cam_data[2]	Input	PIN_B10	7
cam_data[1]	Input	PIN_A10	7
cam_data[0]	Input	PIN_B9	7
cam_hsync	Input	PIN_B13	7
cam_pclk	Input	PIN_F11	7
cam_pdown	Output	PIN_C9	7
cam_reset	Output	PIN_A9	7
cam_scl	Output	PIN_A15	7
cam_sda	Bidir	PIN_B14	7
cam_vsync	Input	PIN_A14	7
cam_xclk	Output	PIN_D12	7
clk	Input	PIN_M16	5
led	Output	PIN_L12	5
rst_n	Input	PIN_M12	5
sdram_addr[12]	Output	PIN_N2	2
sdram_addr[11]	Output	PIN_N1	2
sdram_addr[10]	Output	PIN_P3	3
sdram_addr[9]	Output	PIN_L2	2
sdram_addr[8]	Output	PIN_L1	2
sdram_addr[7]	Output	PIN_K2	2
sdram_addr[6]	Output	PIN_K1	2
sdram_addr[5]	Output	PIN_J2	2
sdram_addr[4]	Output	PIN_J1	2
sdram_addr[3]	Output	PIN_B3	8
sdram_addr[2]	Output	PIN_A3	8
sdram_addr[1]	Output	PIN_B4	8
sdram_addr[0]	Output	PIN_A4	8
sdram_ba[1]	Output	PIN_C3	8
sdram_ba[0]	Output	PIN_N6	3
sdram_cas_n	Output	PIN_N5	3
sdram_cke	Output	PIN_P1	2
sdram_clk	Output	PIN_P2	2

sdram_dq[7]	Bidir	PIN_M7	3
sdram_dq[6]	Bidir	PIN_L3	2
sdram_dq[5]	Bidir	PIN_K8	3
sdram_dq[4]	Bidir	PIN_D6	8
sdram_dq[3]	Bidir	PIN_K6	2
sdram_dq[2]	Bidir	PIN_D3	8
sdram_dq[1]	Bidir	PIN_K5	2
sdram_dq[0]	Bidir	PIN_J6	2
sdram_dqm[1]	Output	PIN_R1	2
sdram_dqm[0]	Output	PIN_L6	2
sdram_ras_n	Output	PIN_N3	3
sdram_we_n	Output	PIN_L4	2
usb_dq[7]	Output	PIN_L11	4
usb_dq[6]	Output	PIN_J14	5
usb_dq[5]	Output	PIN_G11	6
usb_dq[4]	Output	PIN_K11	5
usb_dq[3]	Output	PIN_F14	6
usb_dq[2]	Output	PIN_J11	5
usb_dq[1]	Output	PIN_F13	6
usb_dq[0]	Output	PIN_C16	6
usb_flaga	Input	PIN_D15	6
usb_flagb	Input	PIN_D16	6
usb_flagc	Input	PIN_F15	6
usb_pa0	Output	PIN_F16	6
usb_pa1	Output	PIN_G15	6
usb_pa2	Output	PIN_G16	6
usb_pa4	Output	PIN_J16	5
usb_pa5	Output	PIN_J15	5
usb_pa6	Output	PIN_K16	5
usb_pa7	Output	PIN_K15	5
usb_pclk	Output	PIN_B16	6
usb_pd0	Input	PIN_L16	5
usb_pd1	Input	PIN_L15	5
usb_pd2	Input	PIN_N16	5
usb_pd3	Input	PIN_M11	4
usb_pd4	Input	PIN_L14	5
usb_pd5	Input	PIN_N15	5
usb_pd6	Input	PIN_L13	5
usb_pd7	Input	PIN_N14	5
usb_rd	Output	PIN_N12	4
usb_wr	Output	PIN_K12	5

15.5. 实验操作

下载 FPGA 配置代码和 USB 固件代码，重新插拔 MINI-USB2.0 线缆(断电重启)， 打开“USB2.0”目录下的“USB_DEMO”软件，点击“Play”按钮即可，效果如下图所示


USB_Demo Copyright (C) 2017-20xx ETree Tech.

第十六章 OV7670 USB 上位机显示

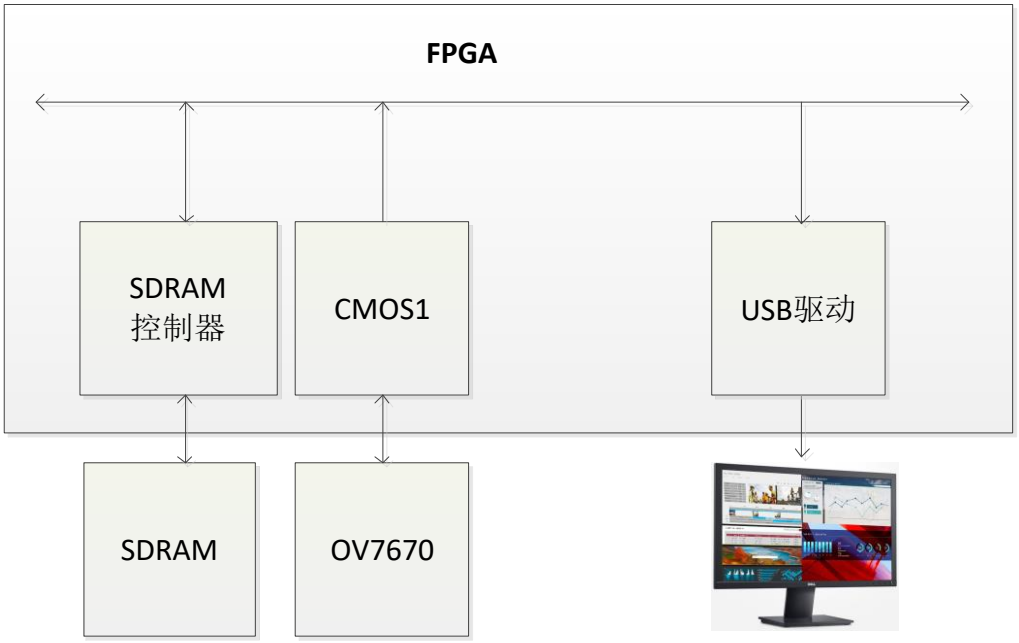
本章将 OV7670 摄像头视频数据通过 USB 发送给 PC 显示，帧率为 30 帧/秒。

16.1. 硬件介绍

本章硬件在前面章节都介绍过，这里不再重复。

16.2. HDL 设计

FPGA 将采集摄像头 OV7670 数据存放在 SDRAM 中，然后 USB 驱动模块再将 SDRAM 中的视频数据发送给 USB2.0 接口芯片，上位机实时显示。



具体代码可以参看具体工程。

16.3. 管脚约束

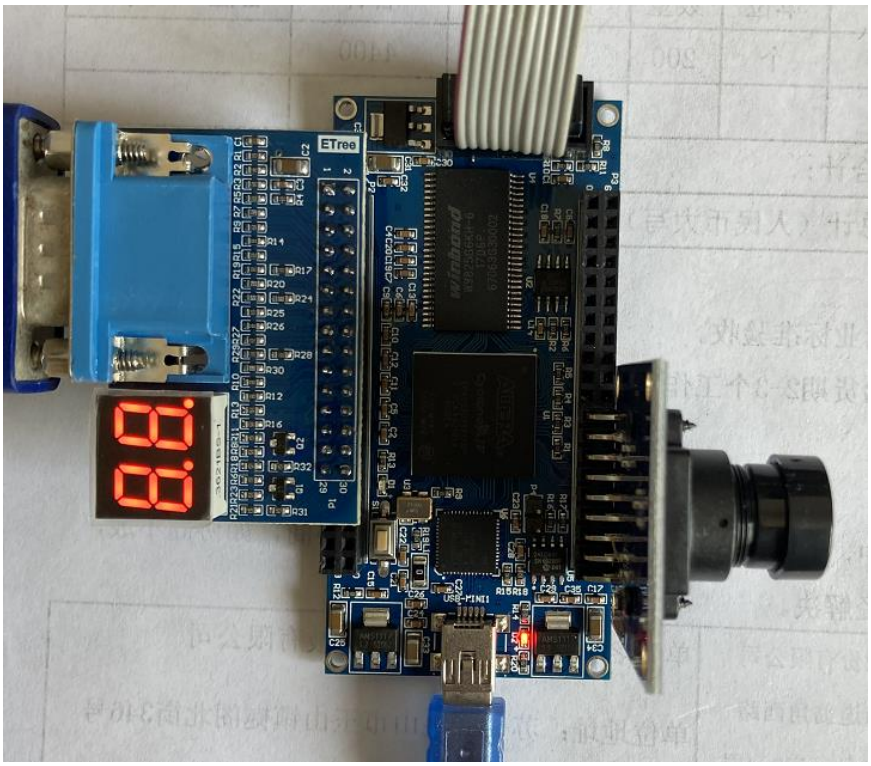
选择菜单 Assignment -> Pin Planner，添加管脚约束。

cam_data[7]	Input	PIN_A13	7
cam_data[6]	Input	PIN_B12	7
cam_data[5]	Input	PIN_A12	7
cam_data[4]	Input	PIN_B11	7
cam_data[3]	Input	PIN_A11	7
cam_data[2]	Input	PIN_B10	7
cam_data[1]	Input	PIN_A10	7
cam_data[0]	Input	PIN_B9	7
cam_hsync	Input	PIN_B13	7
cam_pclk	Input	PIN_F11	7
cam_pdown	Output	PIN_C9	7
cam_reset	Output	PIN_A9	7
cam_scl	Output	PIN_A15	7
cam_sda	Bidir	PIN_B14	7
cam_vsync	Input	PIN_A14	7
cam_xclk	Output	PIN_D12	7
clk	Input	PIN_M16	5
led	Output	PIN_L12	5
rst_n	Input	PIN_M12	5
sdram_addr[12]	Output	PIN_N2	2
sdram_addr[11]	Output	PIN_N1	2
sdram_addr[10]	Output	PIN_P3	3
sdram_addr[9]	Output	PIN_L2	2
sdram_addr[8]	Output	PIN_L1	2
sdram_addr[7]	Output	PIN_K2	2
sdram_addr[6]	Output	PIN_K1	2
sdram_addr[5]	Output	PIN_J2	2
sdram_addr[4]	Output	PIN_J1	2
sdram_addr[3]	Output	PIN_B3	8
sdram_addr[2]	Output	PIN_A3	8
sdram_addr[1]	Output	PIN_B4	8
sdram_addr[0]	Output	PIN_A4	8
sdram_ba[1]	Output	PIN_C3	8
sdram_ba[0]	Output	PIN_N6	3
sdram_cas_n	Output	PIN_N5	3
sdram_cke	Output	PIN_P1	2
sdram_clk	Output	PIN_P2	2

sdram_dq[7]	Bidir	PIN_M7	3
sdram_dq[6]	Bidir	PIN_L3	2
sdram_dq[5]	Bidir	PIN_K8	3
sdram_dq[4]	Bidir	PIN_D6	8
sdram_dq[3]	Bidir	PIN_K6	2
sdram_dq[2]	Bidir	PIN_D3	8
sdram_dq[1]	Bidir	PIN_K5	2
sdram_dq[0]	Bidir	PIN_J6	2
sdram_dqm[1]	Output	PIN_R1	2
sdram_dqm[0]	Output	PIN_L6	2
sdram_ras_n	Output	PIN_N3	3
sdram_we_n	Output	PIN_L4	2
usb_dq[7]	Output	PIN_L11	4
usb_dq[6]	Output	PIN_J14	5
usb_dq[5]	Output	PIN_G11	6
usb_dq[4]	Output	PIN_K11	5
usb_dq[3]	Output	PIN_F14	6
usb_dq[2]	Output	PIN_J11	5
usb_dq[1]	Output	PIN_F13	6
usb_dq[0]	Output	PIN_C16	6
usb_flaga	Input	PIN_D15	6
usb_flagb	Input	PIN_D16	6
usb_flagc	Input	PIN_F15	6
usb_pa0	Output	PIN_F16	6
usb_pa1	Output	PIN_G15	6
usb_pa2	Output	PIN_G16	6
usb_pa4	Output	PIN_J16	5
usb_pa5	Output	PIN_J15	5
usb_pa6	Output	PIN_K16	5
usb_pa7	Output	PIN_K15	5
usb_pclk	Output	PIN_B16	6
usb_pd0	Input	PIN_L16	5
usb_pd1	Input	PIN_L15	5
usb_pd2	Input	PIN_N16	5
usb_pd3	Input	PIN_M11	4
usb_pd4	Input	PIN_L14	5
usb_pd5	Input	PIN_N15	5
usb_pd6	Input	PIN_L13	5
usb_pd7	Input	PIN_N14	5
usb_rd	Output	PIN_N12	4
usb_wr	Output	PIN_K12	5

16.4. 实验操作

插上 JTAG 下载器、MINI USB 电源线、OV7670 摄像头模块(注意 1 脚对应 P3 的 1 脚)，如下图所示。



下载 output_file.jic 文件到 SPI Flash 中，断电后重启。

打开“Tools”目录下的“USB_Demo”软件，点击“Play”按钮即可，实时视频显示效果如下图所示：

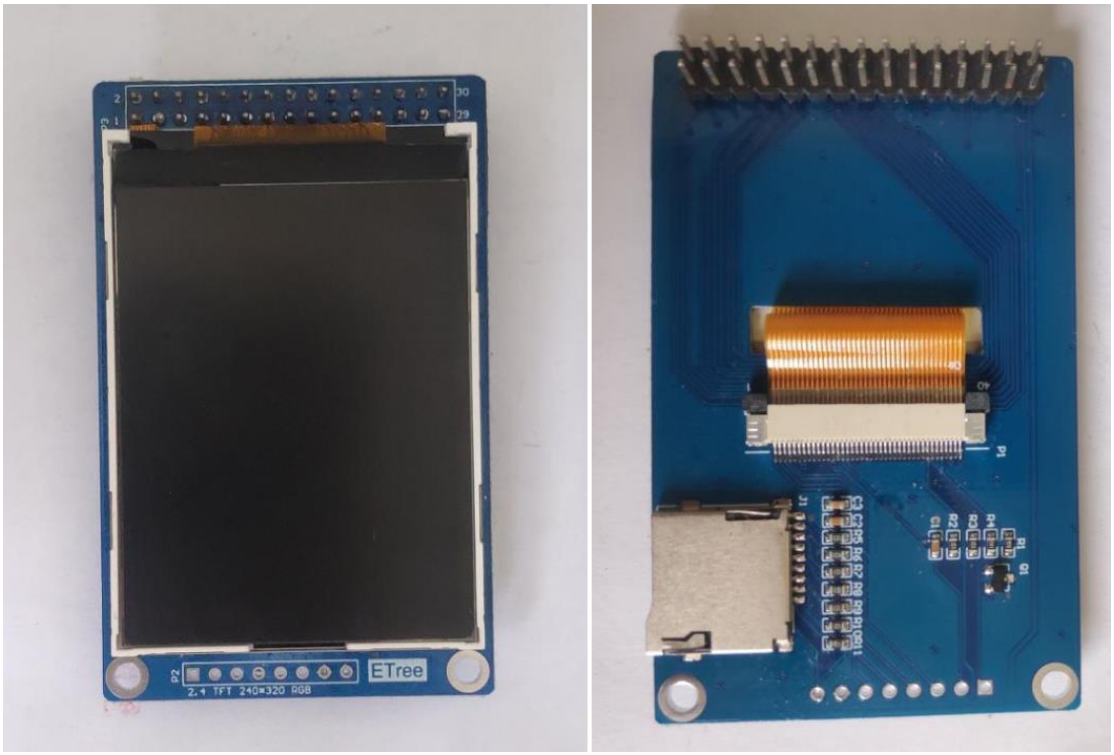


第十七章 液晶屏彩条显示[2.4 寸]

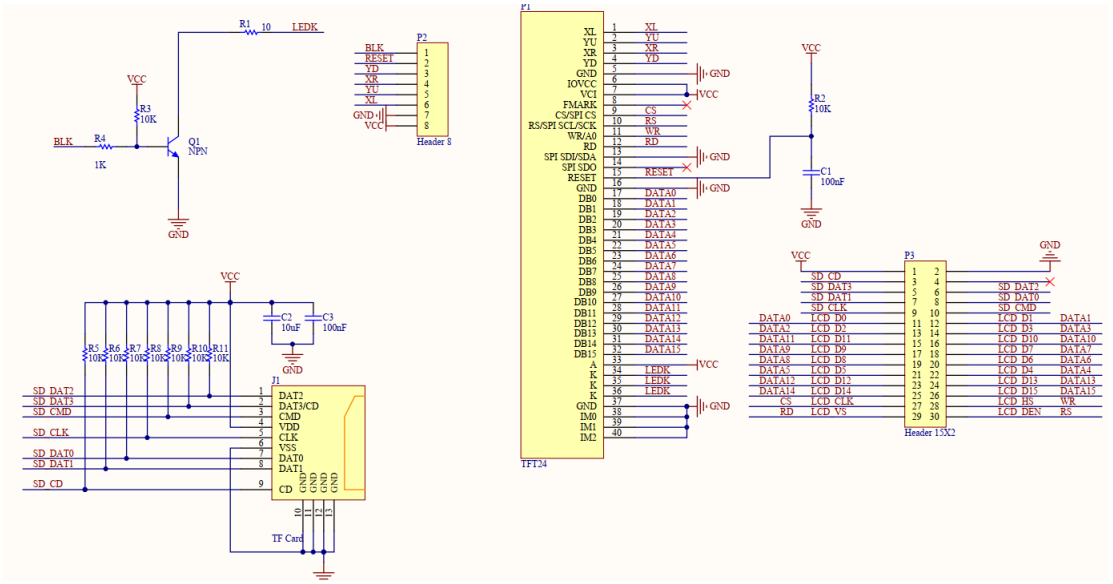
本章基于 2.4 寸液晶屏模块，实现 2.4 寸液晶屏彩条显示。

17.1. 硬件介绍

TFT 液晶屏模块分辨率为 320*240，驱动芯片为 ILI9341，可以显示 16 位色的真彩图，接口为 MCU 并口，该模块外观图如下所示：



模块硬件原理图如下所示：



ILI9341 液晶控制器自带显存，大小为 172800(240*320*18bit)。在 16 位模式下，ILI9341 采用 RGB565 格式存储像素数据，ILI9341 的 18 位数据线与 LCD GRAM 的对应关系如下图所

示。

9341 总线	D17	D16	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
GRAM (16bit)	R[4]	R[3]	R[2]	R[1]	R[0]	NC	G[5]	G[4]	G[3]	G[2]	G[1]	G[0]	B[4]	B[3]	B[2]	B[1]	B[0]	NC

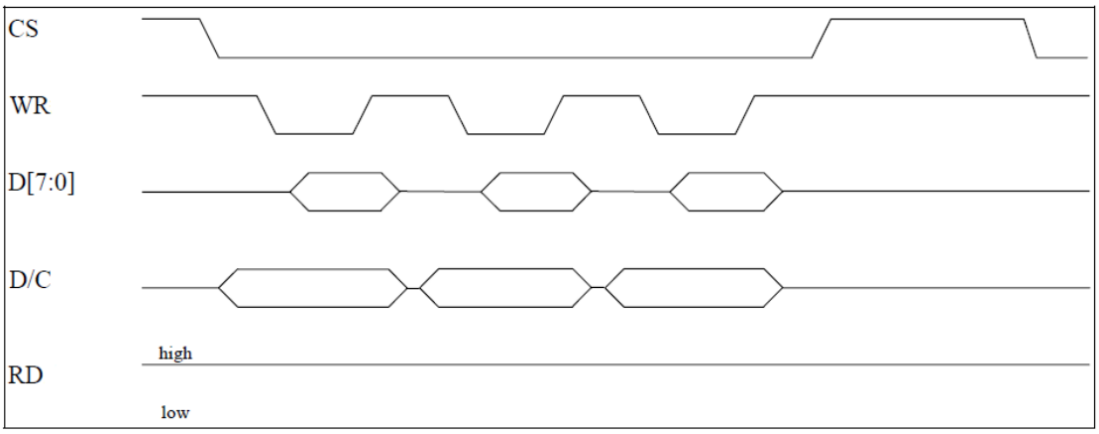
17.2. TFT-LCD 简介

TFT-LCD(Thin Film Transistor-Liquid Crystal Display)通过 MCU 并口(8080)来和其他主控通信。MCU 并口总线广泛的应用于各类液晶屏显示器上，主要包含一下信号线：

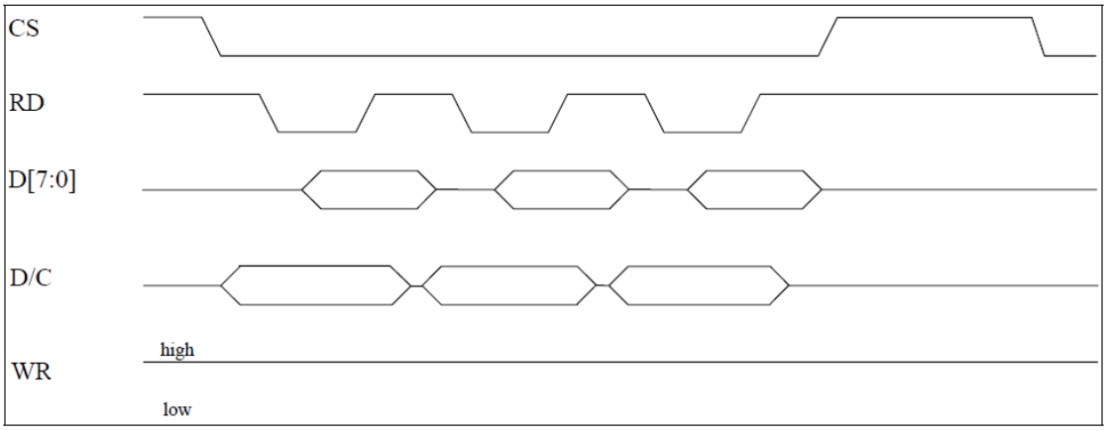
- CS: 片选信号；
- WR: 写入信号；
- RD: 读取信号；
- DATA[15:0]: 双向数据信号；
- RST: 复位信号；
- DC: 命令/数据标志(0 读，1 写)。

MCU 并口读写流程：先根据读写的数据类型，设置 DC，然后拉低 CS，接着设置 RD/WR 为低，在 RD 上升沿读取 DATA 数据，在 WR 上升沿写入 DATA 数据。

下面为写时序图：



下面为读时序图：



本章我们通过 STM32 的普通 IO 模拟 MCU 总线来驱动液晶屏显示，其中 PB0~PB15 用于 16 位数据通道，提高速率。

液晶屏的驱动芯片 ILI9341 自带显存，大小为 172800(240*320*18bit)。在 16 位模式下，ILI9341 采用 RGB565 格式存储像素数据，ILI9341 的 18 位数据线与 LCD GRAM 的对应关系如下图所示。

9341 总线	D17	D16	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
GRAM (16bit)	R[4]	R[3]	R[2]	R[1]	R[0]	NC	G[5]	G[4]	G[3]	G[2]	G[1]	G[0]	B[4]	B[3]	B[2]	B[1]	B[0]	NC

下面介绍 ILI9341 的几个重要命令，关于完整的命令可以参考 ILI9341 的数据手册 datasheet。

0x36 指令：存储访问控制指令，用于控制读写的方向。

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	1	0	1	1	0	36H
参数	1	1	↑	XX	MY	MX	MV	ML	BGR	MH	0	0	0

0x36 指令后面跟一个参数，其中 MY/MX/MV 可以控制 ILI9341 的扫描方向。

控制位			效果
MY	MX	MV	LCD 扫描方向（GRAM 自增方式）
0	0	0	从左到右, 从上到下
1	0	0	从左到右, 从下到上
0	1	0	从右到左, 从上到下
1	1	0	从右到左, 从下到上
0	0	1	从上到下, 从左到右
0	1	1	从上到下, 从右到左
1	0	1	从下到上, 从左到右
1	1	1	从下到上, 从右到左

0x2A 列地址设置指令，从上到下扫描(默认)下，用于设置横坐标 x，其中 SC 表示起始值，EC 表示结束值。

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	0	1	0	1	0	2AH
参数 1	1	1	↑	XX	SC15	SC14	SC13	SC12	SC11	SC10	SC9	SC8	SC
参数 2	1	1	↑	XX	SC7	SC6	SC5	SC4	SC3	SC2	SC1	SC0	
参数 3	1	1	↑	XX	EC15	EC14	EC13	EC12	EC11	EC10	EC9	EC8	EC
参数 4	1	1	↑	XX	EC7	EC6	EC5	EC4	EC3	EC2	EC1	EC0	

0x2B 页地址设置指令，从左到右，从上到下扫描(默认)下，用于设置纵坐标 y，其中 SP 表示开始值，EP 表示结束值。

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	0	1	0	1	0	2BH
参数 1	1	1	↑	XX	SP15	SP14	SP13	SP12	SP11	SP10	SP9	SP8	SP
参数 2	1	1	↑	XX	SP7	SP6	SP5	SP4	SP3	SP2	SP1	SP0	
参数 3	1	1	↑	XX	EP15	EP14	EP13	EP12	EP11	EP10	EP9	EP8	EP
参数 4	1	1	↑	XX	EP7	EP6	EP5	EP4	EP3	EP2	EP1	EP0	

0x2C 写 GRAM 指令，在发完该指令后，便可以往 LCD 的 GRAM 中写入颜色数据，支持连续写。

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	0	1	1	0	0	2CH
参数 1	1	1	↑	D1[15：0]									XX
……	1	1	↑	D2[15：0]									XX
参数 n	1	1	↑	Dn[15：0]									XX

0x2E 读 GRAM 指令，用于读取 ILI9341 的显存。

顺序	控制			各位描述												HEX
	RS	RD	WR	D15~D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX				0	0	1	0	1	1	1	0	2EH
参数 1	1	↑	1	XX												dummy
参数 2	1	↑	1	R1[4:0]		XX		G1[5:0]						XX		R1G1
参数 3	1	↑	1	B1[4:0]		XX		R2[4:0]					XX			B1R2
参数 4	1	↑	1	G2[5:0]			XX	B2[4:0]					XX			G2B2
参数 5	1	↑	1	R3[4:0]		XX		G3[5:0]						XX		R3G3
参数 N	1	↑	1	按以上规律输出												

17.3. HDL 设计

这里只需要实现 2.8 寸液晶屏的并口写，实现“红、绿、蓝、黄”彩条显示，由于液晶屏是 16 位 RGB565，颜色定义如下：

```
parameter RED       = 16'hf800;
parameter GREEN     = 16'h07e0;
parameter BLUE      = 16'h001f;
parameter YELLOW    = 16'hffe0;
```

关于液晶屏的具体寄存器配置这里不做介绍，可以参看芯片手册，配置代码如下：

```
always @(*)
begin
    case(lut_index)
    0   :   lut_data    <= 8'hCF;   //
    1   :   lut_data <= 8'h00;
    2   :   lut_data <= 8'hD9;
    3   :   lut_data <= 8'h30;

    4   :   lut_data <= 8'hED;  //
```

```
5   :   lut_data <= 8'h64;
6   :   lut_data <= 8'h03;
7   :   lut_data <= 8'h12;
8   :   lut_data <= 8'h81;

9   :   lut_data <= 8'hE8;  //
10  :   lut_data <= 8'h85;
11  :   lut_data <= 8'h10;
12  :   lut_data <= 8'h78;

13  :   lut_data <= 8'hCB;  //
14  :   lut_data <= 8'h39;
15  :   lut_data <= 8'h2C;
16  :   lut_data <= 8'h00;
17  :   lut_data <= 8'h34;
18  :   lut_data <= 8'h02;

19  :   lut_data <= 8'hF7;  //
20  :   lut_data <= 8'h20;

21  :   lut_data <= 8'hEA;  //
22  :   lut_data <= 8'h00;
23  :   lut_data <= 8'h00;

24  :   lut_data <= 8'hC0;  //Power control
25  :   lut_data <= 8'h21;

26  :   lut_data <= 8'hC1;  //Power control
27  :   lut_data <= 8'h12;

28  :   lut_data <= 8'hC5;  //VCM control
29  :   lut_data <= 8'h32;
30  :   lut_data <= 8'h3C;

31  :   lut_data <= 8'hC7;  //VCM control2
32  :   lut_data <= 8'hC1;

33  :   lut_data <= 8'h36;  //Memory Access Control
34  :   lut_data <= 8'h08;

35  :   lut_data <= 8'h3A;  //
36  :   lut_data <= 8'h55;

37  :   lut_data <= 8'hB1;  //
```

```
38  :   lut_data <= 8'h00;
39  :   lut_data <= 8'h18;

40  :   lut_data <= 8'hB6;  //
41  :   lut_data <= 8'h0A;
42  :   lut_data <= 8'hA2;

43  :   lut_data <= 8'hF2;  //
44  :   lut_data <= 8'h00;

45  :   lut_data <= 8'h26;  //
46  :   lut_data <= 8'h01;

47  :   lut_data <= 8'hE0;  //SET Gamma
48  :   lut_data <= 8'h0F;
49  :   lut_data <= 8'h20;
50  :   lut_data <= 8'h1E;
51  :   lut_data <= 8'h09;
52  :   lut_data <= 8'h12;
53  :   lut_data <= 8'h0B;
54  :   lut_data <= 8'h50;
55  :   lut_data <= 8'hBA;
56  :   lut_data <= 8'h44;
57  :   lut_data <= 8'h09;
58  :   lut_data <= 8'h14;
59  :   lut_data <= 8'h05;
60  :   lut_data <= 8'h23;
61  :   lut_data <= 8'h21;
62  :   lut_data <= 8'h00;

63  :   lut_data <= 8'hE1;  //Set Gamma
64  :   lut_data <= 8'h00;
65  :   lut_data <= 8'h19;
66  :   lut_data <= 8'h19;
67  :   lut_data <= 8'h00;
68  :   lut_data <= 8'h12;
69  :   lut_data <= 8'h07;
70  :   lut_data <= 8'h2D;
71  :   lut_data <= 8'h28;
72  :   lut_data <= 8'h3F;
73  :   lut_data <= 8'h02;
74  :   lut_data <= 8'h0A;
75  :   lut_data <= 8'h08;
76  :   lut_data <= 8'h25;
```

```
    77  :   lut_data <= 8'h2D;
    78  :   lut_data <= 8'h0F;

    79  :   lut_data <= 8'h11;  //EXIT SLEEP
//  80  :   lut_data <= 8'h29;  //Display on
//  81  :   lut_data <= 8'h29;  //Display on
    82  :   lut_data <= 8'h29;  //Display on

    83  :   lut_data <= 8'h2A;  //X region
    84  :   lut_data <= 8'h00;
    85  :   lut_data <= 8'h00;
    86  :   lut_data <= 8'h00;  //239-->0x00EF
    87  :   lut_data <= 8'hEF;

    88  :   lut_data <= 8'h2B;  //Y region
    89  :   lut_data <= 8'h00;
    90  :   lut_data <= 8'h00;
    91  :   lut_data <= 8'h01;  //319-->0x013F
    92  :   lut_data <= 8'h3F;

    93  :   lut_data <= 8'h2C;  //
    default: lut_data <= 0;
    endcase;
end
```

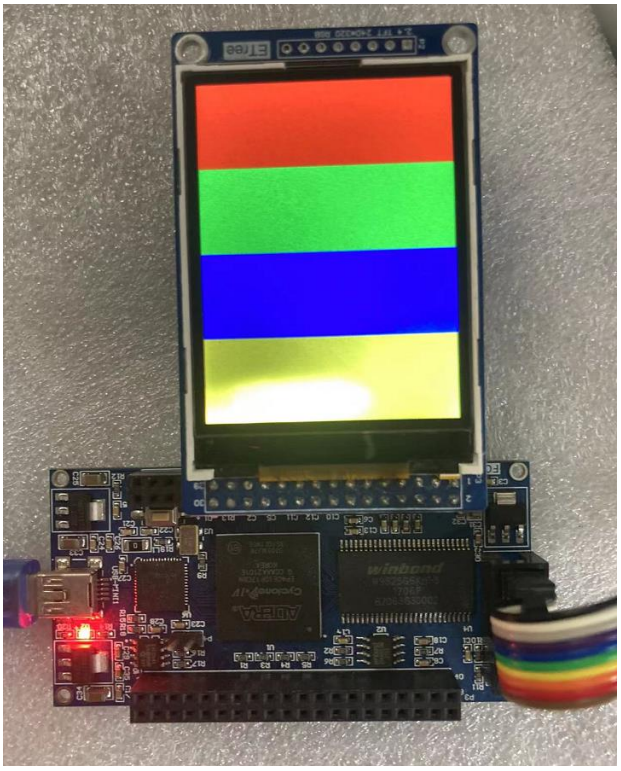
17.4. 管脚分配

选择菜单 Assignment -> Pin Planner，添加管脚约束。

Node Name	Direction	Location	I/O Bank
altera_reserved_tms	Input		
clk	Input	PIN_M16	5
led	Output	PIN_L12	5
rst_n	Input	PIN_M12	5
tft_cs	Output	PIN_T11	4
tft_data[15]	Output	PIN_R9	4
tft_data[14]	Output	PIN_T10	4
tft_data[13]	Output	PIN_T8	3
tft_data[12]	Output	PIN_T9	4
tft_data[11]	Output	PIN_N9	4
tft_data[10]	Output	PIN_M10	4
tft_data[9]	Output	PIN_P11	4
tft_data[8]	Output	PIN_R7	3
tft_data[7]	Output	PIN_N11	4
tft_data[6]	Output	PIN_L10	4
tft_data[5]	Output	PIN_R8	3
tft_data[4]	Output	PIN_T7	3
tft_data[3]	Output	PIN_P9	4
tft_data[2]	Output	PIN_M9	4
tft_data[1]	Output	PIN_L9	4
tft_data[0]	Output	PIN_K10	4
tft_rd	Output	PIN_R14	4
tft_rs	Output	PIN_P14	4
tft_wr	Output	PIN_R10	4

17.5. 实验操作

将 2.4 寸液晶屏模块插入 P2 接插件(注意 1 脚对应关系)，插入 USB-Blaster 下载器，再插入 USB 线缆给板卡供电后，下载 FPGA 配置文件，实际效果图如下所示：



第十八章 OV7670 液晶屏彩条显示[2.4 寸]

本章基于 OV7670 摄像头模块和 2.4 寸液晶屏，实现摄像头数据的实时显示。

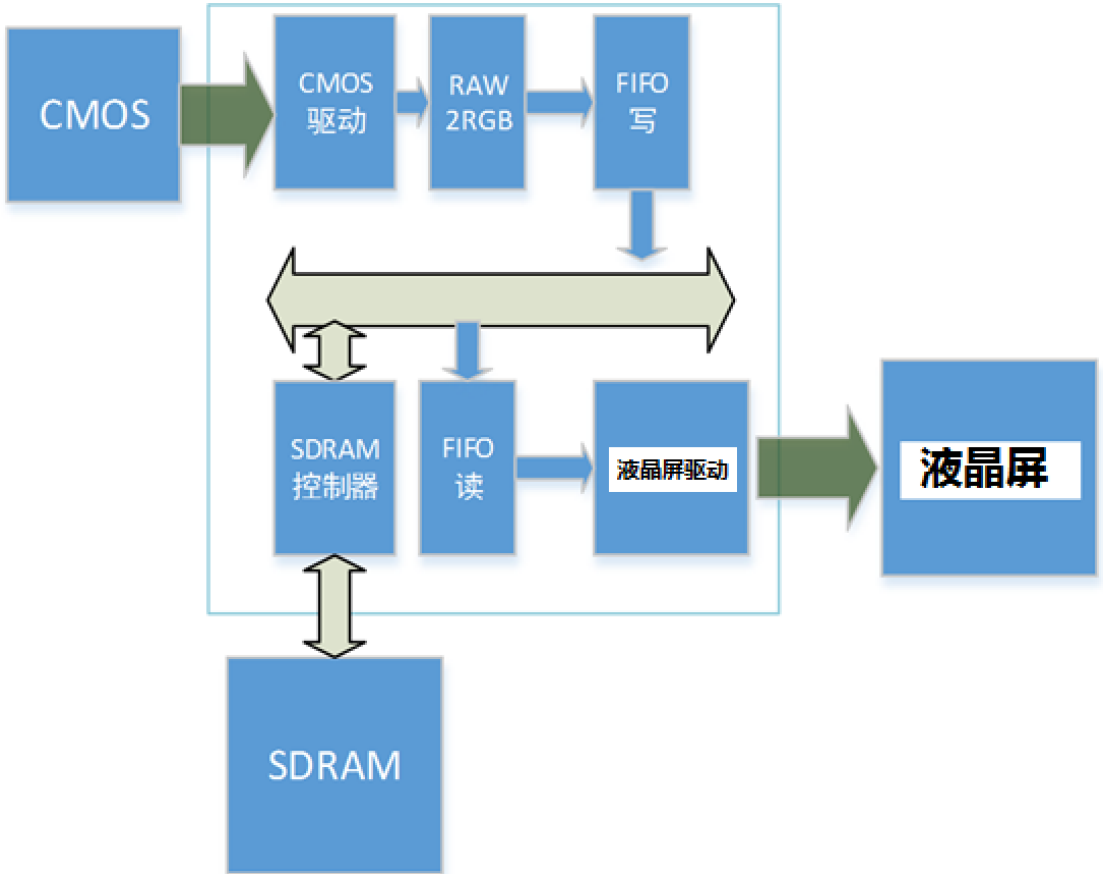
18.1. 硬件介绍

<略>

18.2. HDL 设计

➢ 系统框架

- CMOS 驱动：负责 CMOS 图像采集与裁剪；
- RAW2RGB：负责将原始的 RAW 数据转换成 RGB656 数据，这部功能由摄像头芯片内部完成，FPGA 直接接受的便是 RGB565 数据；
- FIFO 写负责将 RAW2RGB 转换的的 RGB 数据发送给 SDRAM 控制器；
- FIFO 读负责将 SDRAM 中的视频帧数据读出给液晶屏驱动模块；
- 液晶屏模块负责将视频信号发送给外部液晶屏显示。



➢ SDRAM 驱动

本章的 SDRAM 驱动修改于“友晶”开源的 SDRAM 控制器，实现 SDRAM 读写的控制。有兴趣的读者可以研究一下，这里不作说明。

18.3. 管脚约束

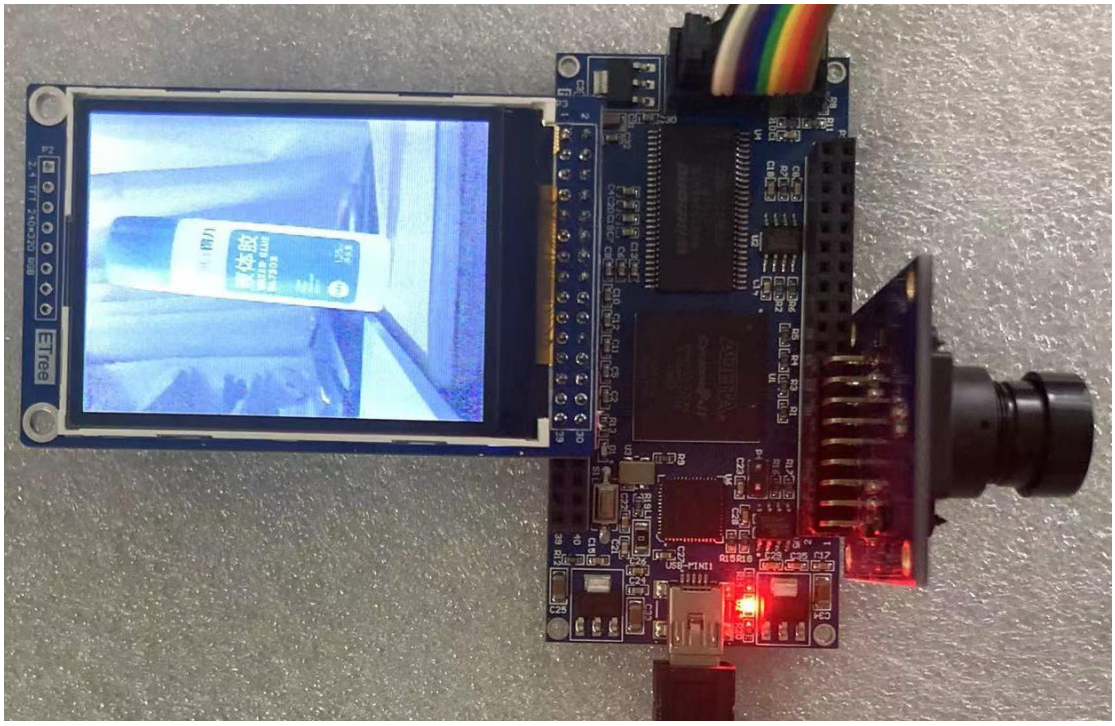
选择菜单 Assignment -> Pin Planner，添加管脚约束。

Node Name	Direction	Location	I/O Bank
altera_reserved_tms	Input		
clk	Input	PIN_M16	5
led	Output	PIN_L12	5
rst_n	Input	PIN_M12	5
tft_cs	Output	PIN_T11	4
tft_data[15]	Output	PIN_R9	4
tft_data[14]	Output	PIN_T10	4
tft_data[13]	Output	PIN_T8	3
tft_data[12]	Output	PIN_T9	4
tft_data[11]	Output	PIN_N9	4
tft_data[10]	Output	PIN_M10	4
tft_data[9]	Output	PIN_P11	4
tft_data[8]	Output	PIN_R7	3
tft_data[7]	Output	PIN_N11	4
tft_data[6]	Output	PIN_L10	4
tft_data[5]	Output	PIN_R8	3
tft_data[4]	Output	PIN_T7	3
tft_data[3]	Output	PIN_P9	4
tft_data[2]	Output	PIN_M9	4
tft_data[1]	Output	PIN_L9	4
tft_data[0]	Output	PIN_K10	4
tft_rd	Output	PIN_R14	4
tft_rs	Output	PIN_P14	4
tft_wr	Output	PIN_R10	4

cam_data[7]	Input	PIN_A13	7
cam_data[6]	Input	PIN_B12	7
cam_data[5]	Input	PIN_A12	7
cam_data[4]	Input	PIN_B11	7
cam_data[3]	Input	PIN_A11	7
cam_data[2]	Input	PIN_B10	7
cam_data[1]	Input	PIN_A10	7
cam_data[0]	Input	PIN_B9	7
cam_hsync	Input	PIN_B13	7
cam_pclk	Input	PIN_F11	7
cam_pdown	Output	PIN_C9	7
cam_reset	Output	PIN_A9	7
cam_scl	Output	PIN_A15	7
cam_sda	Bidir	PIN_B14	7
cam_vsync	Input	PIN_A14	7
cam_xclk	Output	PIN_D12	7
clk	Input	PIN_M16	5
led	Output	PIN_L12	5
rst_n	Input	PIN_M12	5
sdram_addr[12]	Output	PIN_N2	2
sdram_addr[11]	Output	PIN_N1	2
sdram_addr[10]	Output	PIN_P3	3
sdram_addr[9]	Output	PIN_L2	2
sdram_addr[8]	Output	PIN_L1	2
sdram_addr[7]	Output	PIN_K2	2
sdram_addr[6]	Output	PIN_K1	2
sdram_addr[5]	Output	PIN_J2	2
sdram_addr[4]	Output	PIN_J1	2
sdram_addr[3]	Output	PIN_B3	8
sdram_addr[2]	Output	PIN_A3	8
sdram_addr[1]	Output	PIN_B4	8
sdram_addr[0]	Output	PIN_A4	8
sdram_ba[1]	Output	PIN_C3	8
sdram_ba[0]	Output	PIN_N6	3
sdram_cas_n	Output	PIN_N5	3
sdram_cke	Output	PIN_P1	2
sdram_clk	Output	PIN_P2	2

18.4. 实验操作

将 2.4 寸液晶屏模块插入 P2 接插件(注意 1 脚对应关系)，将 OV7670 摄像头插入 P3 接插件(注意 1 脚对应关系)，插入 USB-Blaster 下载器，再插入 USB 线缆给板卡供电后，下载 FPGA 配置文件，实际效果图如下所示：



第十九章 USB 液晶屏图片显示[2.4 寸]

上一章，我们实现了 2.4 寸液晶屏的摄像头数据显示，本章通过 USB2.0 芯片将上位机发送过来的图像数据显示在 2.4 寸液晶屏上。

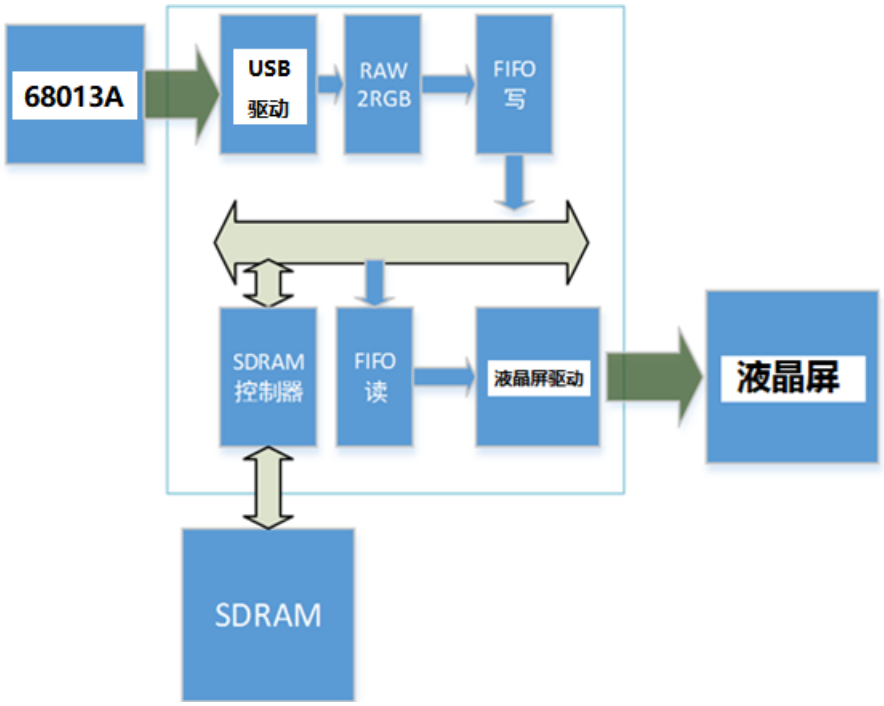
19.1. 硬件介绍

<略>

19.2. HDL 设计

➢ 系统框架

- USB 驱动：负责接受 68013A 发送过来的数据，具体原理可以参看“USB SLAVE FIFO 读”章节；
- RAW2RGB：负责将原始的 RAW 数据转换成 RGB656 数据，这部功能由图像提取软件”Img2Lcd.exe”完成，FPGA 直接接受的便是 RGB565 数据；
- FIFO 写负责将 RAW2RGB 转换的的 RGB 数据发送给 SDRAM 控制器；
- FIFO 读负责将 SDRAM 中的视频帧数据读出给液晶屏驱动模块；
- 液晶屏模块负责将视频信号发送给外部液晶屏显示。



➢ SDRAM 驱动

本章的 SDRAM 驱动修改于“友晶”开源的 SDRAM 控制器，实现 SDRAM 读写的控制。有兴趣的读者可以研究一下，这里不作说明。

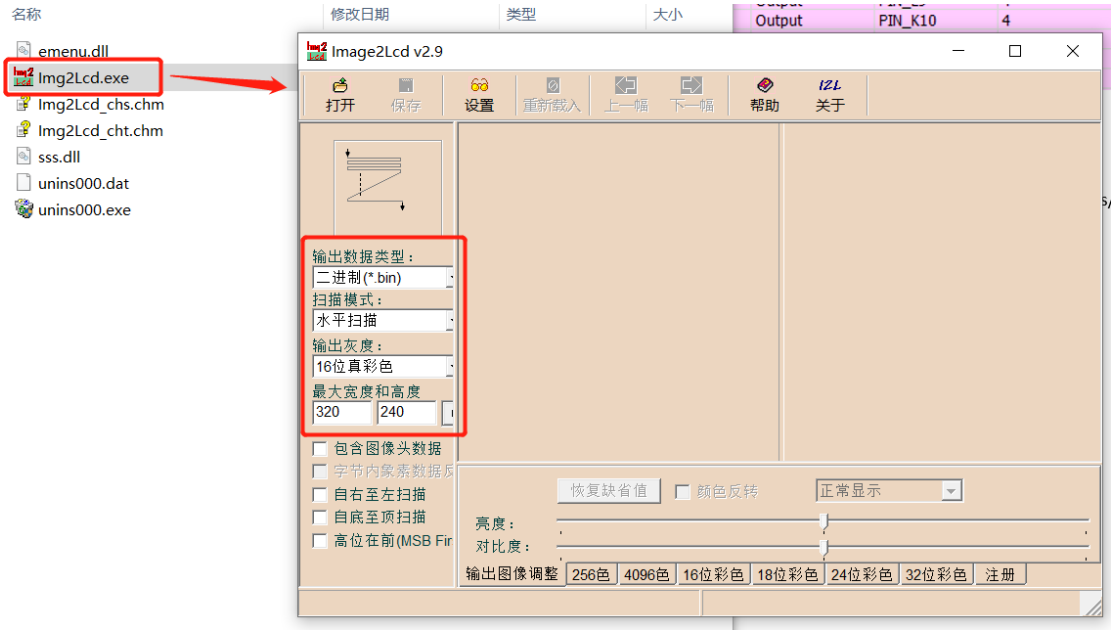
19.3. 管脚分配

选择菜单 Assignment -> Pin Planner，添加管脚约束。

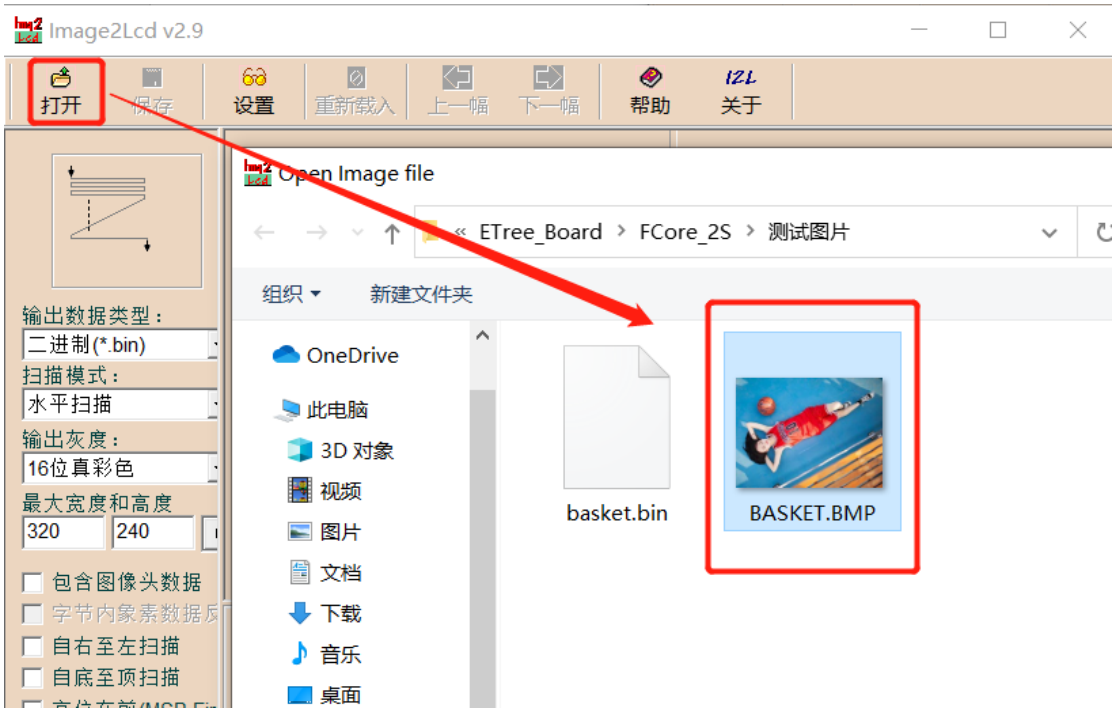
Node Name	Direction	Location	I/O Bank
altera_reserved_tms	Input		
clk	Input	PIN_M16	5
led	Output	PIN_L12	5
rst_n	Input	PIN_M12	5
tft_cs	Output	PIN_T11	4
tft_data[15]	Output	PIN_R9	4
tft_data[14]	Output	PIN_T10	4
tft_data[13]	Output	PIN_T8	3
tft_data[12]	Output	PIN_T9	4
tft_data[11]	Output	PIN_N9	4
tft_data[10]	Output	PIN_M10	4
tft_data[9]	Output	PIN_P11	4
tft_data[8]	Output	PIN_R7	3
tft_data[7]	Output	PIN_N11	4
tft_data[6]	Output	PIN_L10	4
tft_data[5]	Output	PIN_R8	3
tft_data[4]	Output	PIN_T7	3
tft_data[3]	Output	PIN_P9	4
tft_data[2]	Output	PIN_M9	4
tft_data[1]	Output	PIN_L9	4
tft_data[0]	Output	PIN_K10	4
tft_rd	Output	PIN_R14	4
tft_rs	Output	PIN_P14	4
tft_wr	Output	PIN_R10	4

19.4. 实验操作

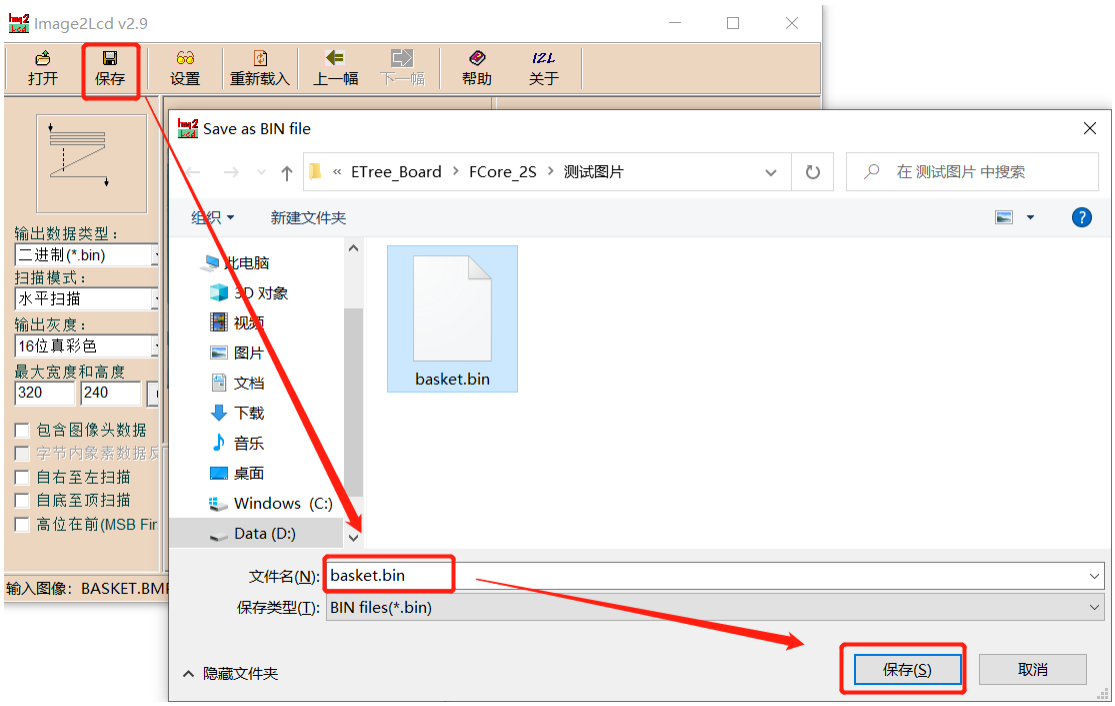
在做实验之前需要生成需要的图片的二进制文件，打开“Tools/ Image2Lcd 2.9/”目录下的“Img2Lcd.exe”软件，如下图所示：设置输出格式为二级制 bin，水平扫描，16 位真彩色，最大宽度和高度为 320 和 240。



打开“测试图片”目录下的 basket.bmp 图片，如下图所示：

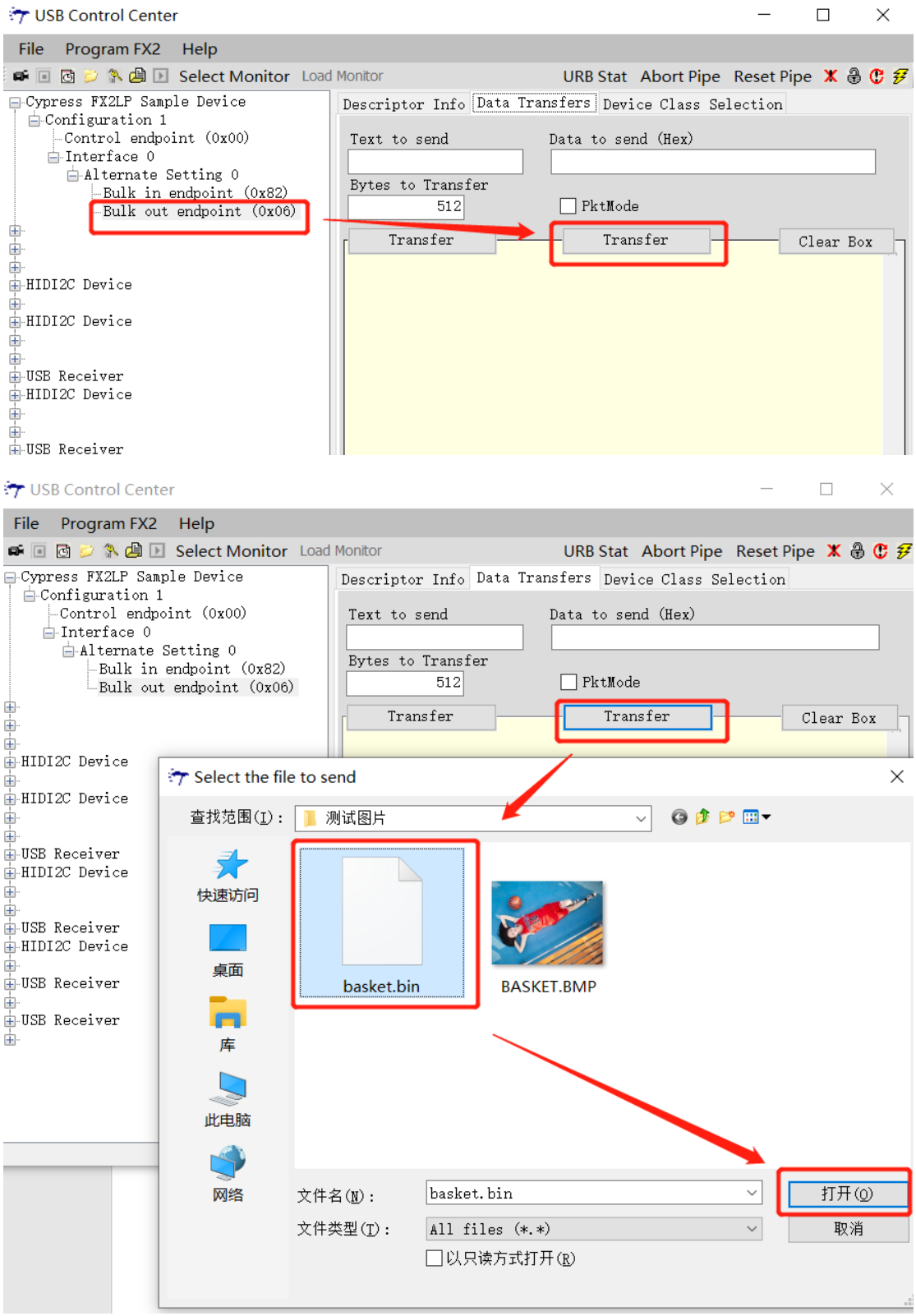


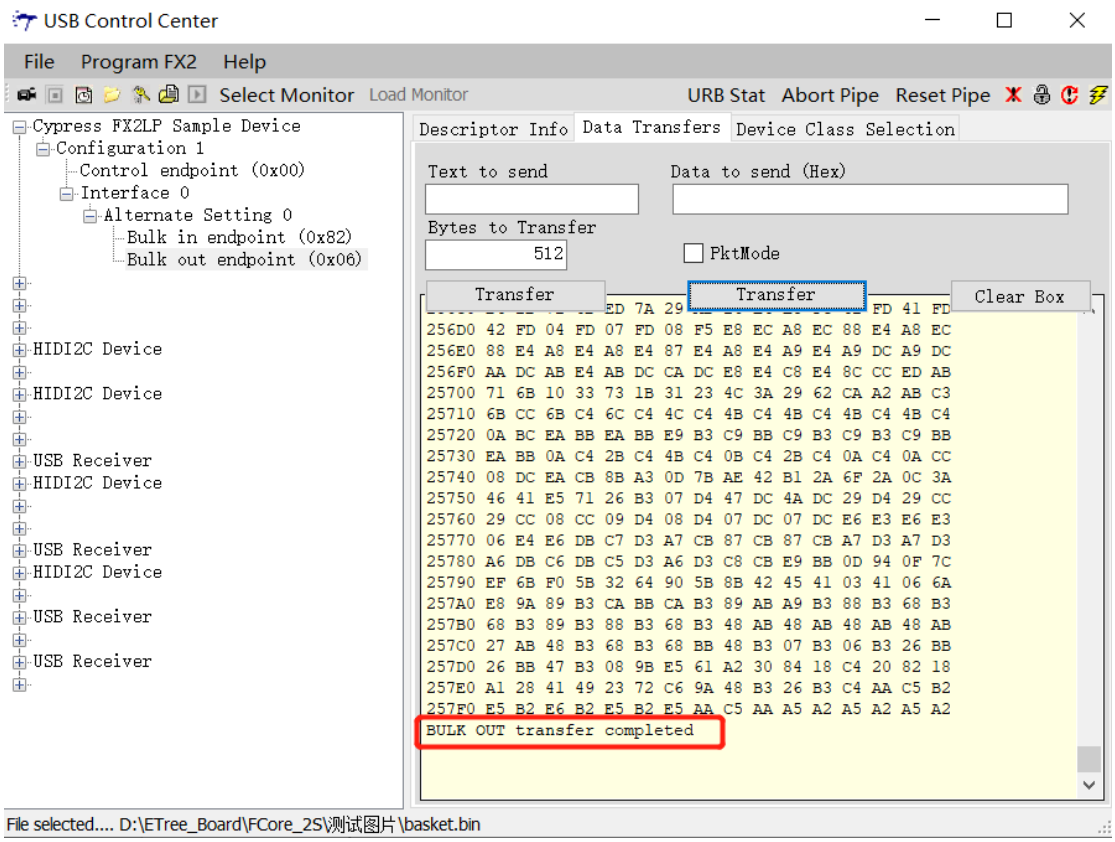
点击“保存”二进制数据，如下图所示：取名为 basket.bin。



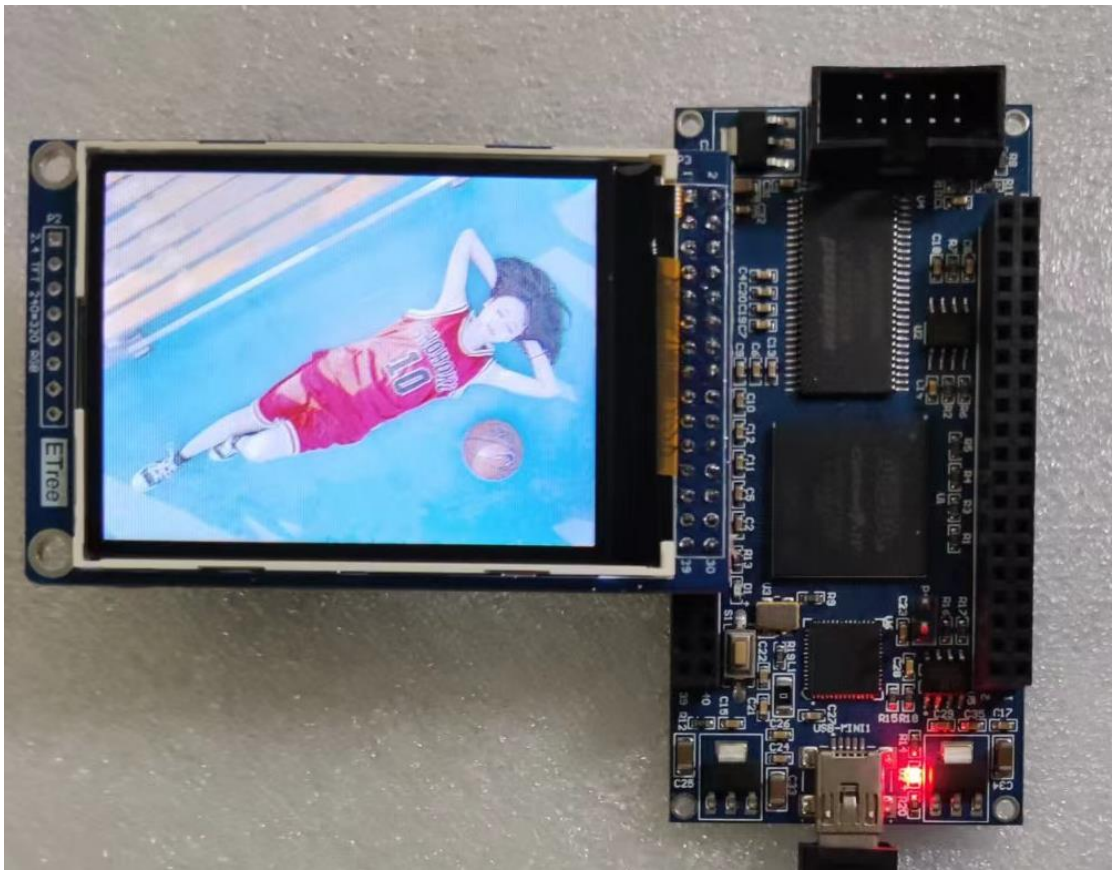
将 2.4 寸液晶屏模块插入 P2 接插件(注意 1 脚对应关系)，插入 USB-Blaster 下载器，再插入 USB 线缆给板卡供电后，下载 FPGA 配置文件。

打开“USB2.0\USB Control Center\”目录下的“CyControl.exe”，选中“Bulk out endpoint(0x06)”节点，点击中间的“Transfer”按钮，选择刚刚生成的 basket.bin 文件，如下图所示：


USB Control Center
File
Program FX2
Help
Select Monitor
Load Monitor
URB Stat
Abort Pipe
Reset Pipe
Cypress FX2LP Sample Device
Configuration 1
Control endpoint (0x00)
Interface 0
Alternate Setting 0
Bulk in endpoint (0x82)
Bulk out endpoint (0x06)
HIDI2C Device
HIDI2C Device
USB Receiver
HIDI2C Device
USB Receiver
Descriptor Info
Data Transfers
Device Class Selection
Text to send
Data to send (Hex)
Bytes to Transfer
512
PktMode
Transfer
Transfer
Clear Box
Select the file to send
查找范围(I):
测试图片
快速访问
桌面
库
此电脑
网络
basket.bin
BASKET.BMP
文件名(N):
basket.bin
文件类型(T):
All files (*.*)
打开(O)
取消
以只读方式打开(R)



从上图可以看出 BULK OUT 传输成功，具体显示效果如下图所示：和测试图片一样。



第二十章 USB 图片 VGA 显示

END